

Escuela Técnica Superior de Ingeniería La Salle

Trabajo Fin de Grado

Grado en Ingeniería Multimedia

Unity, Godot y GameMaker:
Una comparativa del desarrollo de juegos en 2D.

Alumno

Gabriel Sánchez López

Profesor Ponente

Jordi Roig Vilaseca

ACTA DEL EXAMEN
DEL TRABAJO FINAL DE GRADO

Reunido el Tribunal calificador en el día de la fecha, el alumno

D.

expuso su Trabajo Final de Grado, el cual trató sobre el tema siguiente:

Acabada la exposición y contestadas por parte del alumno las objeciones formuladas por los Sres. miembros del tribunal, este valoró el mencionado Trabajo con la calificación de

Barcelona,

VOCAL DEL TRIBUNAL VOCAL DEL TRIBUNAL

PRESIDENTE DEL TRIBUNAL

Resumen

Para hacer videojuegos, la gran mayoría de desarrolladores emplean motores de videojuegos. Estos son programas que actúan como frameworks, estableciendo una base para el desarrollo de videojuegos mediante el uso de librerías y paquetes. La principal ventaja que aportan es que permiten no tener que reinventar la rueda y agilizan el proceso de creación de juegos; es por esto por lo que son tan utilizados en el ámbito del desarrollo de videojuegos.

Ahora bien, hoy en día existe una amplia selección de motores de videojuegos entre los que escoger. Algunos se especializan en 3D, otros en 2D y algunos en ambas modalidades. Además, cada motor de videojuegos tiene sus puntos a favor y en contra, así como factores que lo definen. Esto hace que escoger un motor sea una tarea compleja, ya que, debido a la gran variedad disponible y a los requisitos de cada proyecto, convendrá más uno que otro.

En este documento voy a analizar, dentro del ámbito de la creación de juegos en 2D, cuál de los motores de videojuegos más populares es el más adecuado para esta labor. Compararé Unity con Godot y GameMaker. Estos son algunos de los motores de videojuegos más utilizados para el desarrollo de juegos en 2D, aunque Godot y Unity también permiten crear juegos en 3D, siendo GameMaker el único exclusivamente orientado al 2D.

He elegido el desarrollo de un juego en 2D, ya que la mayoría de desarrolladores indie deciden optar por este formato debido a su sencillez y accesibilidad. El proyecto consistirá en desarrollar el mismo juego en 2D en los tres motores de videojuegos y, posteriormente, comparar el proceso de creación en cada motor, el rendimiento de cada uno y otros factores adicionales.

Resum

Per crear videojocs, la gran majoria de desenvolupadors fan servir motors de videojocs. Aquests són programes que actuen com a frameworks i estableixen una base per al desenvolupament de videojocs mitjançant l'ús de llibreries i paquets. El principal avantatge que aporten és que permeten no haver de reinventar la roda i agilitzen el procés de creació de jocs; és per això que són tan utilitzats en l'àmbit del desenvolupament de videojocs.

Ara bé, avui dia hi ha una àmplia selecció de motors de videojocs per escollir. Alguns s'especialitzen en 3D, d'altres en 2D i alguns en ambdues modalitats. A més, cada motor de videojocs té els seus punts forts i febles, així com factors que el defineixen. Això fa que escollir un motor de videojocs sigui una tasca complexa, ja que, a causa de la gran varietat disponible i dels requisits de cada projecte, n'hi haurà algun que s'adapti millor que un altre.

En aquest document analitzaré, dins l'àmbit de la creació de jocs en 2D, quin dels motors de videojocs més populars és el més adequat per a aquesta tasca. Compararé Unity amb Godot i GameMaker. Aquests són alguns dels motors de videojocs més utilitzats per al desenvolupament de jocs en 2D, tot i que Godot i Unity també permeten crear jocs en 3D, mentre que GameMaker és l'únic exclusivament orientat al 2D.

He triat el desenvolupament d'un joc en 2D perquè la majoria de desenvolupadors indie opten per aquest format a causa de la seva senzillesa i accessibilitat. El projecte consistirà a desenvolupar el mateix joc en 2D en els tres motors de videojocs i, posteriorment, comparar el procés de creació en cada motor, el rendiment de cadascun i altres factors addicionals.

Abstract

To create video games, the vast majority of developers use game engines. These are software tools that act as frameworks, providing a foundation for video game development through the use of libraries and packages. Their main advantage is that they allow developers to avoid reinventing the wheel and speed up the game development process, which is why they are so widely used in the field of game development.

Nowadays, however, there is a wide variety of game engines to choose from. Some specialize in 3D, others in 2D, and some support both. In addition, each game engine has its own strengths, weaknesses, and defining characteristics. This makes choosing a game engine a complex task, since, depending on the wide range of options available and the specific requirements of a project, one engine may be more suitable than another.

In this document, I will analyze which of the most popular game engines is best suited for developing a 2D game. I will compare Unity, Godot, and GameMaker. These are among the most widely used game engines for 2D game development, although Unity and Godot also support 3D game development, while GameMaker is exclusively focused on 2D.

I have chosen 2D game development because many indie developers opt for this format due to its simplicity and accessibility. The project will consist of developing the same 2D game using the three game engines and then comparing the development process in each engine, the performance of each one, and other additional factors.

Agradecimientos

Quisiera expresar mi más sincero agradecimiento a mi familia por su apoyo constante y por comprender el esfuerzo y el tiempo invertidos en este proyecto.

También quiero agradecer a Jordi Roig los valiosos comentarios que me brindó durante la realización de este trabajo, ya que fueron de gran ayuda para mejorarlo.

Índice

1 Introducción 16
2 Motores de videojuegos 17
2.1 Historia de los motores de videojuegos 17
2.2 Otras maneras de desarrollar juegos 20
2.3 Sobre los motores de videojuegos 24
2.4 Sobre Unity 27
2.4.1 Características de Unity 32
2.5 Sobre Godot 34
2.5.1 Características de Godot 38
2.6 Sobre GameMaker 40
2.6.1 Características de GameMaker 43
3 El juego 45
3.1 Desarrollo en Unity 47
3.1.1 Diagrama de clases 47
3.1.2 Lógica del jugador 49
3.1.3 Lógica del enemigo 55
3.1.4 Lógica del spawner y de la puntuación 59
3.1.5 Lógica de las vidas del jugador, timer y menú de pausa 63
3.1.6 Audio, fondo y power-ups 68
3.1.7 Escenas game over y main menu 69
3.1.8 Build del juego 71
3.1.9 Sobre los shaders 71
3.2 Desarrollo en Godot 72
3.2.1 Diagrama de clases 72
3.2.2 Lógica del jugador 74
3.2.3 Lógica del enemigo y power-up 78
3.2.4 Lógica del timer, vidas del jugador y puntuación 82
3.2.5 Lógica del spawner, menú de pausa y música 85
3.2.6 Escenas game over y main menu 89
3.2.7 Build del juego 91
3.2.8 Sobre los shaders 92
3.3 Desarrollo en GameMaker 93
3.3.1 Diagrama de clases 93
3.3.2 Lógica del jugador 95
3.3.3 Lógica del enemigo, spawner, power-up y puntuación 98
3.3.4 Lógica del timer, vidas del jugador y fondo de la escena 102
3.3.5 Lógica del menú de pausa 104
3.3.6 Escena main menu y game over 106

3.3.7 Sobre los shaders 108
4 Conclusiones del Proyecto 109
5 Uso de herramientas de Inteligencia Artificial 112
6 Bibliografía 113
7 Anexo 114

Acrónimos

VR: Virtual Reality (realidad virtual).

UI: User Interface (interfaz de usuario).

Indie: abreviatura de independiente, equipo de desarrollo independiente que crea videojuegos sin el respaldo financiero directo de grandes editoras, generalmente con mayor libertad creativa.

2D: Dos dimensiones.

3D: Tres dimensiones.

Arcade: estilo de videojuego de mecánicas simples, acción rápida y enfoque en la diversión inmediata, sin gran complejidad ni realismo basado en las maquinas del mismo nombre.

Framework: es la estructura base reutilizable que contiene herramientas y reglas para construir aplicaciones de una forma más ágil y ordenada.

Script: conjunto de instrucciones escritas en un lenguaje de programación para ejecutar tareas específicas de forma automática.

Bullet hell: subgénero de videojuegos de disparos donde el jugador debe esquivar enormes cantidades de proyectiles que llenan la pantalla.

Motor lightweight: es un software diseñado para crear juegos de forma eficiente y flexible, con bajo consumo de recursos y sin la complejidad ni el peso de motores más completos.

Open source: es un modelo de desarrollo en el que el código de un software se hace público para que cualquiera pueda usarlo, estudiarlo, modificarlo y compartirlo libremente.

1 Introducción

Si uno decide adentrarse en el mundo del desarrollo de videojuegos, se topará con la palabra game engine muy rápidamente. Esto es debido a que estas herramientas se emplean con frecuencia para desarrollar juegos debido a su capacidad de acelerar y facilitar el proceso de hacer juegos. Los motores de videojuegos son un software que actúa como framework, estableciendo unas bases, empleando paquetes y librerías que permiten crear los juegos de manera más rápida, aligerando el proceso de la creación de elementos frecuentes como las físicas de los juegos, por poner un ejemplo. Por estas razones es que los motores de videojuegos son herramientas muy potentes para el desarrollo de videojuegos. Si bien hay motores que tienen muchas similitudes entre sí, hay motores de videojuegos que están mejor diseñados para ciertas tareas que otros y hay algunos que poseen cualidades que pueden hacer que los elijas a ellos por encima de otros. Ejemplo de esto sería cómo GameMaker es un motor de videojuegos diseñado para hacer 2D exclusivamente y, por ende, está optimizado para dicha tarea, por lo que resulta atractivo para alguien que quiera hacer un juego en 2D, mientras que otro motor de videojuegos como Unreal está diseñado para hacer 3D y suele ser el motor predilecto para hacer juego 3D a mayor escala. Otros factores para elegir un motor por encima de otro serían el lenguaje de programación; Unity emplea C#, por lo que si uno tiene experiencia con este lenguaje y/o le resulta más cómodo, elegiría Unity, mientras que Godot emplea su propio lenguaje de programación llamado GDScript, aunque Godot permite usar otros lenguajes de programación. Si alguien busca diseñar juegos y no se le da bien o no quiere programar tanto, quizá le resulte atractivo Unreal, ya que este tiene una función llamada Blueprints que permite crear la lógica del juego sin programar ninguna línea de código, todo mediante un sistema de visual scripting.

Estas son unas de las razones que nos harían escoger un motor de videojuegos por encima de otro; como podemos ver, hay muchos factores que condicionan nuestra decisión. Para este proyecto, he decidido limitar los factores a hacer un juego en 2D. He pensado en hacer un juego 2D debido a que es la opción más escogida por los desarrolladores indies y también por la sencillez y accesibilidad que conlleva desarrollar un juego en 2D. La idea será crear el mismo juego 2D de género bullet hell en cada motor y luego comparar el proceso de creación, el rendimiento y otros factores. He decidido hacer un bullet hell, ya que este tipo de género suele tener varios elementos en pantalla y suele tener un mayor coste de recursos. En cuanto a la elección de los motores de videojuegos, he escogido los más populares para el desarrollo en 2D en la escena indie; estos son: Unity, Godot y GameMaker. Los dos primeros motores de videojuegos pueden hacer tanto 2D como 3D y el último está optimizado para el 2D y se usa frecuentemente para esa función. Unity y Godot son bastante similares, solo que Unity usa objects y Godot nodes, y Unity usa C# mientras que Godot usa GDScript. GameMaker es el más diferente, siendo exclusivo 2D y usando su propio lenguaje llamado GML. Estos motores son los que verás más frecuentemente en el desarrollo de juegos 2D indies y son los que más documentación tienen.

Dicho esto, el proyecto se centrará en crear un bullet hell en estos motores y compararé las diferencias entre ellos, como es el desarrollo en cada motor, su documentación, comunidad y muchos otros factores. Daré mi opinión de la experiencia con cada motor y, por último, diré cuál considero el mejor para la tarea de desarrollar un juego en 2D.

2 Motores de videojuegos

2.1 Historia de los motores de videojuegos

Si uno decide investigar la historia del desarrollo de videojuegos, rápidamente se dará cuenta de que los motores de videojuegos no siempre han existido tal y como los conocemos hoy en día. En los inicios de la industria del videojuego, los juegos se desarrollaban completamente desde cero, tanto el código como las herramientas necesarias para mostrar gráficos, reproducir sonido o gestionar la lógica del juego. Cada videojuego era un proyecto único, creado específicamente para el hardware en el que iba a ejecutarse, lo que implicaba un gran esfuerzo técnico y una enorme cantidad de tiempo invertido en tareas repetitivas.

Durante las décadas de los años setenta y ochenta, cuando los videojuegos comenzaron a popularizarse en máquinas arcade y consolas domésticas, el concepto de reutilización de código era muy limitado. Los desarrolladores programaban directamente sobre el hardware, optimizando cada instrucción para obtener el mejor rendimiento posible. En esta etapa, no se podía hablar todavía de motores de videojuegos como tal, sino de rutinas y sistemas internos creados para un único juego. Sin embargo, este periodo sentó las bases técnicas necesarias para que, con el paso del tiempo, surgiera la necesidad de herramientas más reutilizables y estructuradas. [1]



Ilustración 1. Máquinas arcade de los 80. Fuente: https://mercapixels.com/modules/ph_simpleblog/covers/94.jpg

Con la llegada de los ordenadores personales en los años noventa y el aumento de la complejidad de los videojuegos, comenzó a hacerse evidente que desarrollar cada juego desde cero no era sostenible. Los equipos de desarrollo empezaron a reutilizar partes del código de proyectos anteriores, especialmente sistemas como el renderizado gráfico, la detección de colisiones o el manejo de la entrada del usuario. De esta práctica surgieron los primeros motores de videojuegos primitivos que, aunque no eran productos independientes, sí funcionaban como una base común sobre la que se construían varios juegos.

Un ejemplo claro de esta etapa es el uso de motores internos por parte de estudios grandes. Motores como el de Doom, desarrollado por id Software en 1993, marcaron un antes y un después en la industria. El Doom Engine separaba claramente el motor del contenido del juego, permitiendo reutilizar la tecnología para crear nuevos títulos. Este enfoque no solo aceleraba el desarrollo, sino que también abría la puerta a modificaciones creadas por la comunidad, lo que aumentó enormemente la vida útil del motor. Este concepto fue clave para la evolución posterior de los motores de videojuegos.



Ilustración 2. Doom 1993 por id Software. Fuente: https://m.media-amazon.com/images/M/MV5BymUzN2Y0N2MtNjU5NC00YWM0LTkxNWMtNDIyZWYyODU0Otc5XkEyXkFqcGc@._V1_.jpg

A medida que los juegos en 3D comenzaron a ganar protagonismo a finales de los años noventa, la necesidad de motores más avanzados se volvió aún más evidente. El desarrollo de gráficos tridimensionales, cámaras, iluminación y físicas requería un nivel técnico elevado. En este contexto aparecieron motores como Unreal Engine y Quake Engine, que no solo eran usados internamente por sus creadores, sino que empezaron a licenciarse a otros estudios. Esto marcó un punto de inflexión importante, ya que los motores de videojuegos comenzaron a convertirse en productos comerciales independientes.



Ilustración 3. Unreal Engine logo 2004. Fuente: https://cdn.mobygames.com/promos/2071856-unreal-tournament-2004-logo.jpg

Durante los años 2000, los motores de videojuegos evolucionaron rápidamente, incorporando cada vez más funcionalidades integradas. Ya no se limitaban únicamente al renderizado, sino que incluían editores visuales, sistemas de animación, físicas avanzadas y herramientas para el diseño de niveles. Esta evolución permitió que los desarrolladores se centraran más en el diseño y la creatividad, dejando en manos del motor gran parte del trabajo técnico. En esta etapa, los motores de videojuegos se consolidaron como una pieza fundamental en el desarrollo de videojuegos modernos.

Paralelamente, comenzó a surgir una escena de desarrollo independiente más accesible. Herramientas como GameMaker, lanzado inicialmente a finales de los años noventa, facilitaron la creación de juegos 2D sin necesidad de tener conocimientos profundos de programación. Este tipo de motores estaba pensado para usuarios con menos experiencia técnica, ofreciendo lenguajes simplificados y entornos más amigables. Gracias a esto, muchas personas pudieron dar sus primeros pasos en el desarrollo de videojuegos.

Con la llegada de la década de 2010, los motores de videojuegos experimentaron una democratización sin precedentes. Motores como Unity y Godot adoptaron modelos de

negocio más accesibles, permitiendo su uso gratuito o con costes muy reducidos. Unity, en particular, tuvo un impacto enorme en la escena indie gracias a su facilidad de uso, su compatibilidad con múltiples plataformas y su amplio ecosistema de recursos y documentación. Godot, por su parte, apostó por el código abierto, ofreciendo un motor completamente gratuito y modificable por la comunidad.

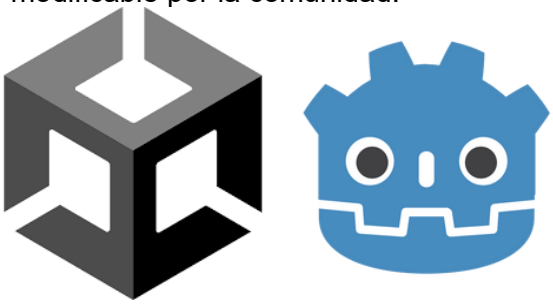


Ilustración 4. Logo de Unity y Godot. Fuentes: https://encrypted-tbn0.gstatic.com/images?q=tbn:And9GcRMEpvhQOuCkjQFYaax3e8Bma9yk4Y6xt_HUQ&s https://static.cdnlogo.com/logos/g/54/godot_thumb.png

En esta etapa, los motores de videojuegos no solo se convirtieron en herramientas para crear videojuegos, sino también en plataformas de aprendizaje y experimentación. Su uso se extendió a otros ámbitos como la simulación, la visualización arquitectónica y la educación. Además, la integración de lenguajes de programación más accesibles y sistemas de scripting visual permitió que personas sin formación técnica avanzada pudieran crear experiencias interactivas complejas.

En la actualidad, los motores de videojuegos continúan evolucionando de manera constante. Incorporan tecnologías modernas como el renderizado en tiempo real avanzado, la inteligencia artificial, la realidad virtual y aumentada, y sistemas de físicas cada vez más realistas. Motores como Unreal Engine 5 han elevado el nivel gráfico a cotas muy altas, mientras que otros como Godot y Unity siguen enfocándose en la accesibilidad y versatilidad para distintos tipos de proyectos.

En conclusión, la historia de los motores de videojuegos está estrechamente ligada a la evolución de la industria del videojuego. Lo que comenzó como simples rutinas de código reutilizadas ha terminado convirtiéndose en herramientas complejas y potentes que permiten crear videojuegos de forma más rápida, eficiente y accesible. Gracias a los motores de videojuegos, el desarrollo de videojuegos ha dejado de ser exclusivo de grandes estudios, permitiendo que desarrolladores independientes y pequeños equipos puedan materializar sus ideas y contribuir al crecimiento constante de esta industria.

2.2 Otras maneras de desarrollar juegos

Antes de seguir hablando sobre los motores de videojuegos, me gustaría comentar otra manera de desarrollar juegos que es menos habitual, pero también se usa; me refiero al uso de librerías o frameworks. Hay veces que los desarrolladores eligen desarrollar su juego usando una librería o un framework en vez de usar un motor de videojuegos; esto es porque los frameworks y librerías permiten crear tu juego exactamente como tú quieres. Esto tiene sus pros y sus contras, ya que el crear un juego desde cero significa reinventar la rueda y hacer todas las físicas desde cero, al igual que todas las funciones. Hay frameworks que tienen muchas herramientas para hacer este proceso más rápido y menos complejo, pero, por regla general, el desarrollar un juego empleando un framework o librería implica construir el mundo y su lógica desde cero. Esto significa que puedes customizarlo como tú quieras y sabes su funcionamiento a la perfección, pero, por otro lado, es mucho más trabajo y, si no tienes mucho interés en ello, los motores de videojuegos ya dan todo esto hecho.

Otra razón para usar un framework por encima de un motor de videojuegos es que no exista un motor de videojuegos que tenga funciones para hacer lo que quieres hacer. Un ejemplo de este caso es el puzzle plataformero llamado Fez, que es un juego en 2D que tiene la mecánica de rotar el mundo. Esta función fue añadida posteriormente en motores como Unity, pero en el momento de su creación no existía motor de videojuegos que pudiese hacer esa función, por lo que el desarrollador, Phil Fish, creó su propio motor llamado Trixel.

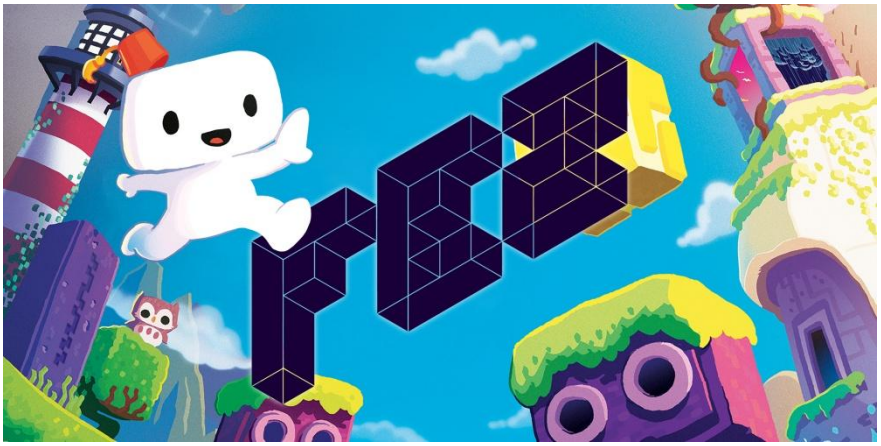


Ilustración 5. Videojuego Fez por Phil Fish. Fuente: https://www.nintendo.com/eu/media/images/10_share_images/games_15/nintendo_switch_download_software_1/H2x1_NswitchDS_Fez.jpg

También hay gente que emplea frameworks por temas de aprendizaje. Al diseñar un juego usando frameworks o librerías, tienes que crear todo de cero, por lo que te da un mayor entendimiento sobre desarrollar videojuegos y amplía tus conocimientos sobre game design. Asimismo, mejoras tus habilidades como programador, ya que para hacer juegos con un framework o librería se suele hacer todo con código y no hay elemento visual alguno, forzando al creador a hacer el juego todo mediante código.

Como podemos ver, los frameworks y librerías tienen sus usos y pueden ser preferibles a los motores de videojuegos dependiendo de la situación. Aunque, por otro lado, hay gente que considera que, como tienes que hacer todo desde cero, esto lleva a un desarrollo de juegos más lento y, como se tarda más en dar resultados, no es óptimo. Al final, si quieres aprender y/o crear mecánicas que no te aporten los motores de videojuegos, es recomendable usar frameworks o librerías, pero si este no es el caso, usar un motor de videojuegos es más simple y te permite crear resultados de manera más rápida, cosa que los desarrolladores indies valoran mucho.

Dicho esto, esto no significa que no haya juegos exitosos que no hayan empleado ningún motor de videojuegos. El caso más reciente y popular de un juego que ha tenido mucho éxito y ha sido diseñado empleando un framework es el caso de Balatro, un roguelike de cartas que ha vendido mucho y se ha hecho muy popular.



Ilustración 6. Balatro juego hecho por LocalThunk. Fuente: https://upload.wikimedia.org/wikipedia/en/thumb/8/89/Balatro_cover.jpg/422px-Balatro_cover.jpg

Este juego fue diseñado con el framework llamado LÖVE o Love2D. Este framework está pensado para crear juegos en 2D empleando el lenguaje de programación conocido como Lua. Una cosa curiosa de los juegos diseñados empleando Love2D es que el ejecutable .exe puede ser extraído y se pueden ver los archivos de código y todos los archivos empleados para la creación del proyecto. Esto es una vulnerabilidad que puede llevar a plagios de los juegos diseñados con este framework, cosa que ha pasado con Balatro, pero también ha llevado a poder ver los archivos y código creados para hacer el juego de Balatro, por lo que es una cosa interesante de ver. Si uno inspecciona el código de Balatro, puede ver que no es óptimo y resulta un poco amateur; de hecho, mucha gente calificó el código de Balatro como spaghetti code. Al final, lo que cuenta es el resultado y nadie puede negar que Balatro es un éxito y el juego funciona correctamente, sea el código más o menos óptimo.

El éxito de Balatro trajo consigo un incremento en la popularidad del framework Love2D; esto ha inspirado a muchos desarrolladores a usarlo y a adentrarse en el uso de los frameworks. Un ejemplo de un juego por venir que emplea Love2D es el juego Moonshire desarrollado por Challacade. Este desarrollador tiene un canal de YouTube con el mismo nombre en el que publica sus devlogs de Moonshire y también sube tutoriales de Love2D, por lo que es interesante consultarlo para ver cómo es el desarrollo de videojuegos empleando frameworks, en concreto usando Love2D.



Ilustración 7. Juego Moonshire por Challacade. Fuente: https://shared.fastly.steamstatic.com/store_item_assets/steam/apps/2542170/capsule_616x353.jpg?t=1751322127

Aparte de los frameworks, también se pueden emplear librerías como Kaplay para, en este caso, desarrollar juegos en JavaScript. Esta tiene menos herramientas fuera de la caja que un framework como Love2D, pero tiene los mismos puntos positivos y negativos que usar un framework. En el caso de Kaplay, esta cuenta con una documentación muy sencilla y fácil de entender y la librería en sí no es difícil de usar, por lo que, para los que estén interesados en desarrollar juegos web, es una buena elección. Una persona que ha ayudado a popularizar esta librería y a usar librerías para el desarrollo de juegos web es el YouTuber y desarrollador de videojuegos JSLegendDev.



Ilustración 8. Logo de JSLegendDev. Fuente: https://yt3.googleusercontent.com/BE0KjxKYBjzbDMDVXIilDNdtGcr5TlG6VBrLrkjt8KB71lIFZoVAH0a6pbH-kvJbyNK6GrjtA0U=s160-c-k-c0x00ffffff-no-rj

Existen muchos otros frameworks y librerías para los lenguajes de programación mencionados y para muchos otros; otra librería muy conocida es Pygame, que emplea el lenguaje Python y se basa en la librería SDL. Esta Pygame está pensada para el desarrollo de juegos en 2D y está orientada al manejo de sprites. Debido a la sencillez de Python, la librería Pygame permite prototipar y desarrollar rápidamente; es tan buena en este aspecto que cada vez es más usada en game jams donde el plazo es muy reducido. También es muy usada, ya que permite crear otros programas multimedia o interfaces gráficas de usuario.

No es lo mismo desarrollar un juego usando un framework que una librería que un motor de videojuegos. Desarrollar un juego con una librería implica tener control total del código y construir casi todo desde cero, mientras que un framework proporciona una estructura base que agiliza el desarrollo, pero limita algo la libertad. Un motor de videojuegos va un paso más allá, ofreciendo además herramientas completas como editores visuales, sistemas de físicas y gestión de escenas, lo que permite desarrollar juegos de forma mucho más rápida y accesible.

En conclusión, si bien el uso de los frameworks y librerías permite aprender más cómo desarrollar videojuegos y te deja un mayor nivel de personalización, no suelen ser usados, ya que es más rápido y eficiente usar un motor de videojuegos. No obstante, los frameworks y librerías son siempre una opción que merece ser considerada para hacer juegos.

2.3 Sobre los motores de videojuegos

Ya he hablado sobre la historia de los motores de videojuegos, métodos alternativos para desarrollar videojuegos y he comentado un poco qué es un motor de videojuegos. En este apartado, voy a indagar más sobre qué son y cómo funcionan.

Un motor de videojuegos en esencia es una plataforma para crear experiencias digitales interactivas. Estas experiencias pueden ser videojuegos, simulaciones, aplicaciones o realidad virtual. Los motores de videojuegos emplean assets o data para generar la experiencia digital; los motores tienen herramientas para generar dichos assets, aunque también se pueden generar mediante programas externos y luego importarlos. Estos datos que los motores de videojuegos leen runtime y emplean se suelen dividir en tres categorías distintas: los gráficos, que serían los elementos que vemos en la pantalla; el audio, que es todo sonido que escuchamos; y la lógica, que es el código que realizamos. Cada motor también permite convertir la experiencia digital a otras plataformas; algunos permiten diseñar para PC, Android, consolas y más. Algunos de los motores se especializan en ciertas funciones, como Unreal en 3D y GameMaker en 2D, mientras que otros hacen ambas, como Unity y Godot, aunque no igual que uno especializado para una sola como otros. Los motores de videojuegos actúan como framework para agilizar la creación de las experiencias digitales, realizando internamente muchas de las funciones comúnmente asociadas al desarrollo de estas experiencias. Ejemplo de las acciones que los motores de videojuegos facilitan o automatizan: el renderizado de los elementos, la lógica de las físicas y los scripts. El desarrollo de videojuegos empleando un motor de videojuegos se podría resumir como enseñarle al motor de videojuegos qué es lo que quieres hacer y no cómo es, es decir, no hace falta que hagas todo desde cero y explicarle cómo funciona todo, sino cómo debe comportarse lo que quieres hacer, ya que al usar un motor de videojuegos partes de una base establecida. [2]

Cabe mencionar que no todos los motores de videojuegos usados para el desarrollo de videojuegos están disponibles para el uso público. La mayoría de las empresas grandes de videojuegos, conocidas como triple A, emplean sus propios motores de videojuegos creados por ellas para crear sus juegos. El crear tu propio motor de videojuegos te permite hacerlo óptimo para los tipos de juegos que quieres crear. Si bien un buen motor para 3D es Unreal, si quieres crear un juego de mundo abierto donde las decisiones del jugador cambien el mundo y quieres que el mundo sea amplio y tenga buen rendimiento en ciudades, es beneficioso crear tu propio motor que esté pensado y optimizado para crear un juego de tales características, ya que con uno público como Unreal, quizá tengas problemas de rendimiento o no puedas ejecutar el juego que tienes pensado correctamente. Podemos nombrar el motor de videojuegos llamado REDengine de la empresa CD Projekt RED. Este motor fue usado para la creación de la trilogía de juegos de The Witcher; está especializado y optimizado para juegos en 3D no lineales de mundo abierto. La creación y uso de REDengine permitió a los estudios crear unos videojuegos más optimizados y de mejor calidad que no hubiese sido el caso si hubiesen usado algo como Unreal, ya que al ser este más genérico no podría hacer la trilogía de The Witcher igual que un motor pensado y optimizado exclusivamente para esa labor.



Ilustración 9. Logo del motor de videojuegos llamado REDengine. Fuente: https://upload.wikimedia.org/wikipedia/commons/thumb/b/b2/REDengine_logo.svg/500px-REDengine_logo.svg.png

El crear tu propio motor de videojuegos es una inversión; inviertes tiempo y recursos para crear una plataforma optimizada para crear un juego o juegos con ciertas características que ningún otro motor esté optimizado para hacer. Esto resulta en unos juegos de mayor calidad y rendimiento, ya que tienes herramientas diseñadas exactamente para eso, y también te permite crear juegos similares de una manera más rápida y eficiente, por lo que sale rentable a largo plazo. Por otro lado, el uso de un motor de videojuegos público como Unreal permite de una manera rápida hacer juegos sin invertir tiempo en crear tu propio motor y, aunque no esté diseñado exactamente para lo que quieres hacer en concreto, son buenas herramientas genéricas que permiten crear cualquier tipo de juego (en este caso en 3D principalmente), aunque no tan eficientemente como un motor de videojuegos pensado para lo que quieres hacer en concreto.

Profundicemos un poco más en el funcionamiento de los motores de videojuegos. La mayoría de los motores funcionan mediante un bucle principal que se ejecuta constantemente mientras el juego está en marcha. En este bucle se actualiza la lógica del juego, se procesan las entradas del jugador, se calculan las físicas y finalmente se renderiza la imagen que se muestra en pantalla. Este proceso se repite decenas o incluso cientos de veces por segundo. El desarrollador no suele tener que implementar este bucle manualmente, ya que el motor se encarga de ello, permitiendo que el creador del juego se centre en definir el comportamiento de los elementos del juego.

Siguiendo con esta idea, también es importante entender cómo los motores de videojuegos gestionan el ciclo de vida de un juego. La mayoría de los motores funcionan mediante un bucle principal que se ejecuta constantemente mientras el juego está en marcha. En este bucle se actualiza la lógica del juego, se procesan las entradas del jugador, se calculan las físicas y finalmente se renderiza la imagen que se muestra en pantalla. Este proceso se repite decenas o incluso cientos de veces por segundo. El desarrollador no suele tener que implementar este bucle manualmente, ya que el motor se encarga de ello, permitiendo que el creador del juego se centre en definir el comportamiento de los elementos del juego.

Otro aspecto relevante es la optimización del rendimiento. Los motores de videojuegos incluyen numerosas técnicas para mejorar el rendimiento de los juegos, como la gestión eficiente de la memoria, el uso de sistemas de renderizado optimizados o la posibilidad de desactivar elementos que no están siendo utilizados. Estas optimizaciones son especialmente importantes en juegos con muchos elementos en pantalla o en proyectos destinados a dispositivos con recursos limitados, como móviles. Gracias a estas herramientas, es posible crear juegos más complejos sin que el desarrollador tenga que dominar todos los detalles técnicos de bajo nivel.

También cabe destacar la importancia de las herramientas de depuración que ofrecen los motores de videojuegos. Estas herramientas permiten detectar errores, analizar el

rendimiento del juego y entender mejor qué está ocurriendo internamente durante la ejecución. Funciones como los depuradores visuales, los perfiles de rendimiento o los sistemas de registro de errores facilitan enormemente el proceso de desarrollo. Sin estas herramientas, encontrar y corregir errores sería una tarea mucho más lenta y complicada.

En el ámbito del trabajo en equipo, los motores de videojuegos modernos están pensados para facilitar la colaboración entre distintos perfiles profesionales. Programadores, diseñadores, artistas y diseñadores de sonido pueden trabajar sobre el mismo proyecto utilizando el motor como punto de encuentro. Los editores visuales permiten a los diseñadores crear niveles sin necesidad de modificar código, mientras que los artistas pueden importar y ajustar sus assets directamente dentro del motor. Esta separación de roles mejora la eficiencia del desarrollo y es clave en proyectos de mayor tamaño.

Otro punto a considerar es la posibilidad de ampliar y personalizar un motor de videojuegos mediante plugins o extensiones. Muchos motores permiten añadir nuevas funcionalidades creadas por la propia comunidad o por el desarrollador. Esto hace que el motor sea más flexible y adaptable a las necesidades específicas de cada proyecto. Gracias a este sistema, es posible integrar herramientas adicionales sin modificar directamente el núcleo del motor.

Por último, es importante mencionar que el aprendizaje de un motor de videojuegos no solo sirve para crear un único proyecto, sino que las habilidades adquiridas suelen ser transferibles a otros motores. Muchos conceptos, como el uso de escenas, sistemas de físicas, scripting o gestión de assets, son comunes entre distintos motores. Por ello, aprender a usar un motor de videojuegos no solo permite desarrollar un juego concreto, sino que también proporciona una base sólida de conocimientos dentro del desarrollo de videojuegos en general.

2.4 Sobre Unity

Unity es un motor de videojuegos multiplataforma creado por Unity Technologies, anunciado y lanzado por primera vez el 8 de junio de 2005. La empresa Unity Technologies fue fundada en 2004 por David Helgason, Nicholas Francis y Joachim Ante en Copenhague, Dinamarca. Lanzaron un juego llamado GooBall, el cual no tuvo mucho éxito, pero los tres creadores reconocieron el valor del motor de videojuegos, por lo que decidieron lanzar Unity al mercado para que sea un motor con un precio asequible que cualquiera pueda usar. El éxito de Unity se debe principalmente a que desde sus inicios se centró en las necesidades de los desarrolladores independientes. Estos desarrolladores, en la mayoría de los casos, no disponen de los recursos necesarios para crear su propio motor de videojuegos ni podían permitirse, en el momento en que Unity apareció, pagar el acceso a otros motores de videojuegos que resultaban mucho más costosos. La filosofía de la empresa se basa en democratizar el desarrollo de videojuegos, buscando que la creación de contenido interactivo en 2D y 3D sea accesible para la mayor cantidad de personas posible en todo el mundo. [3]



Ilustración 10. Foto de Nicholas, Joachim y David respectivamente. Fuente: https://connect-mediagw.unity.com/h1/20200923/learn/images/8162ccbc-ecec-4168-84ea-8037ª7214d8a_Founders_photo_V3.jpg

En el año 2005, en la Conferencia Mundial de Desarrolladores de Apple, se presentó la primera versión de Unity. El motor de videojuegos en sus comienzos fue programado únicamente para trabajar en computadoras Mac, lo cual posibilitaba crear proyectos en esta plataforma. El éxito de esta primera etapa fue suficiente para impulsar el desarrollo posterior del motor y la ampliación de sus herramientas.

Con el lanzamiento de Unity 3 en septiembre de 2010, el motor tomó un paso importante al comenzar a incluir herramientas más avanzadas, similares a las que utilizan estudios de desarrollo más grandes. La intención era captar a desarrolladores más grandes, pero a la vez seguir ofreciendo un conjunto de herramientas accesibles para los estudios indies, que de otra manera tendrían dificultades para acceder a este tipo de tecnologías de un modo asequible.

A principios del 2015 salió a la venta Unity 5, una versión que supuso una gran mejora. En la Game Developers Conference se anunció esta actualización que incluía muchas novedades como el sistema de animación Mecanim, soporte para DirectX 11 y compatibilidad con el desarrollo de juegos en Linux, así como varias mejoras en rendimiento, corrección de errores y calidad de texturas. Así, Unity continuó con su filosofía de ser un motor creado por desarrolladores de juegos para desarrolladores de juegos, manteniéndose enfocado en las necesidades reales de la industria.

Unity es un motor de creación de videojuegos que se caracteriza por su gran compatibilidad con herramientas externas de creación de contenido. Permite trabajar con programas muy utilizados en la industria para modelar, diseñar y editar, facilitando que los desarrolladores puedan crear gráficos, animaciones o elementos visuales fuera del motor y luego integrar fácilmente en el proyecto. Además, Unity actualiza automáticamente los cambios en estos elementos, lo cual evita tener que reimportarlos manualmente y acelera el proceso de desarrollo.

En cuanto a gráficos, Unity ofrece un motor visual avanzado que se adapta a múltiples plataformas y dispositivos. Esto permite mostrar efectos visuales como luces, sombras, reflejos y otros elementos que mejoran la calidad visual del juego. Internamente, el motor se encarga de manejar todos estos aspectos técnicos, dejando al desarrollador que se preocupe más por el diseño del juego que por los detalles técnicos del renderizado.

Unity también cuenta con un sistema de físicas que permite simular el comportamiento de los objetos dentro del juego, como colisiones, movimientos o interacciones entre elementos. Este sistema es fundamental para crear experiencias más realistas y coherentes, ya que permite que los objetos reaccionen de manera natural al entorno y a las acciones del jugador, sin necesidad de programar estos comportamientos desde cero.

En el apartado de la programación, Unity permite definir la lógica del juego mediante scripts, utilizando principalmente el lenguaje C#. A través de estos scripts, el desarrollador puede controlar el comportamiento de personajes, enemigos, objetos y eventos del juego. Además, el motor ofrece herramientas para depurar errores y probar el funcionamiento del código, lo que facilita el desarrollo y la corrección de fallos durante el proceso de creación.

Por último, Unity cuenta con muchas herramientas más que ayudan a completar el desarrollo de un videojuego. Entre ellos se incluyen audio, animación, creación de escenarios, iluminación y soporte para trabajo en equipo. Un ejemplo de ello es Mecanim, el sistema de animación de Unity, que permite crear movimientos fluidos y volver a utilizar las animaciones en diferentes personajes de una forma sencilla. Estas herramientas integradas hacen que Unity sea una plataforma completa que permite desarrollar videojuegos de una manera más accesible, organizada y eficiente.

En el año 2023, Unity se vio envuelta en una fuerte polémica tras anunciar un cambio importante en su modelo de comisiones. La compañía comunicó que comenzaría a cobrar una tarifa por cada instalación de un juego desarrollado con su motor, siempre que se superara un determinado número de descargas y de ingresos. Esta medida generó un gran rechazo por parte de la comunidad de desarrolladores, especialmente entre los estudios independientes, ya que este nuevo coste reducía considerablemente los beneficios obtenidos, teniendo en cuenta otros gastos habituales como el desarrollo del juego o las comisiones de plataformas de distribución. [4]

La respuesta negativa no tardó en llegar y pronto se extendió por las redes sociales y los foros especializados. Muchos desarrolladores opinaron que esta decisión rompía con la filosofía de accesibilidad que Unity había mantenido durante años. La situación empeoró cuando se supo que el director ejecutivo de la compañía había vendido acciones poco antes de anunciar estas medidas, lo que aumentó la desconfianza y el malestar en la comunidad. Algunos desarrolladores, como los de Cult of the Lamb, amenazaron con quitar su juego de la tienda si Unity no revertía sus políticas.



Ilustración 11. Juego Cult of the Lamb por Massive Monster. Fuente: https://shared.fastly.steamstatic.com/store_item_assets/steam/apps/1313140/19f32bb477a6e9d058e05193b4e282149bd70644/header.jpg?t=1769836674

El impacto de esta polémica fue tan grande que incluso algunas oficinas de Unity tuvieron que cerrar temporalmente debido a amenazas recibidas. Tras una investigación, se determinó que dichas amenazas procedían de un empleado de la propia empresa. Este episodio evidenció la magnitud del conflicto generado y marcó un punto crítico en la relación entre Unity y muchos de sus usuarios, afectando seriamente a la imagen pública del motor. Actualmente, la reputación de Unity se ha visto dañada por esto y ha perdido confianza por parte de los desarrolladores, que ahora muchos de ellos favorecen el uso de otros motores en vez de Unity. Si bien Unity ya no goza de la popularidad y la confianza de antes, sigue siendo una opción muy usada para desarrollar videojuegos, sobre todo en la escena indie.

Unity es un motor intuitivo y fácil de aprender. Tiene soporte para desarrollar juegos tanto en 2D como en 3D y VR. Unity usa escenas, las cuales contienen los entornos y menús del juego. Cada archivo de escena es como un nivel único. En cada escena, se encuentran los entornos, obstáculos y decoraciones, lo que hace que el juego se diseñe de manera segmentada empleando dichas escenas. Para construir los elementos de las escenas en Unity, usamos GameObjects.

En Unity, todos los elementos que forman parte de un videojuego se representan mediante GameObjects. Estos pueden ser personajes, enemigos, objetos interactivos, luces, cámaras o efectos visuales.

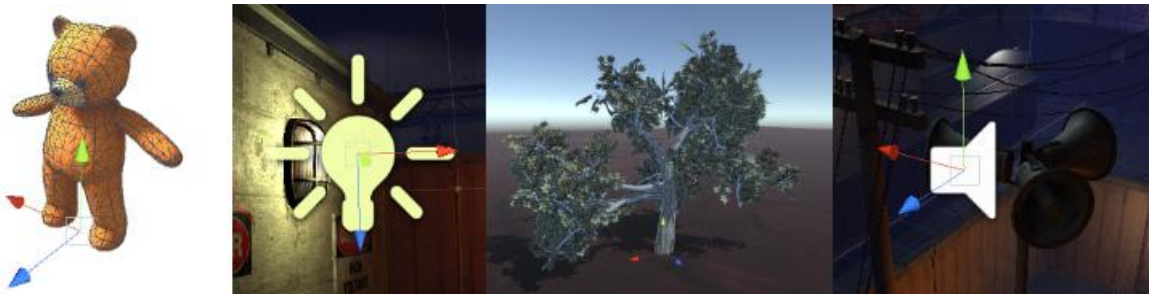


Ilustración 12. Tipos de GameObjects: personaje, luz, árbol y fuente de audio. Fuente: https://docs.unity3d.com/es/2018.4/uploads/Main/GameObjectsExamples.jpg

Por sí solo, un GameObject no tiene ningún comportamiento ni función concreta, sino que actúa como una base sobre la cual se construyen los distintos elementos del juego. Para que un GameObject pueda cumplir una función específica dentro del juego, es necesario añadirle componentes. Los componentes definen las propiedades y comportamientos del objeto, como su posición, su apariencia visual, su sonido o su lógica de funcionamiento. Dependiendo del tipo de elemento que se quiera crear, se combinarán distintos componentes sobre un mismo GameObject, lo que permite una gran flexibilidad a la hora de diseñar el juego. Unity ofrece

una amplia variedad de componentes integrados que cubren las necesidades más comunes del desarrollo de videojuegos. Además, el desarrollador puede crear sus propios componentes mediante programación, adaptándolos a las necesidades específicas del proyecto. De esta manera, los GameObjects, los componentes y el sistema de scripting trabajan de forma conjunta, permitiendo construir y controlar todos los elementos del juego de manera ordenada y eficiente.

Otro importante elemento para el desarrollo de juegos en Unity son los prefabs. Estos permiten guardar un GameObject junto con todos sus componentes, valores y objetos hijos como un recurso reutilizable dentro del proyecto. Un Prefab actúa como una plantilla a partir de la cual se pueden crear múltiples copias de un mismo objeto dentro de una escena o en distintas escenas. Esto resulta especialmente útil cuando se necesitan utilizar los mismos elementos varias veces a lo largo de un videojuego.

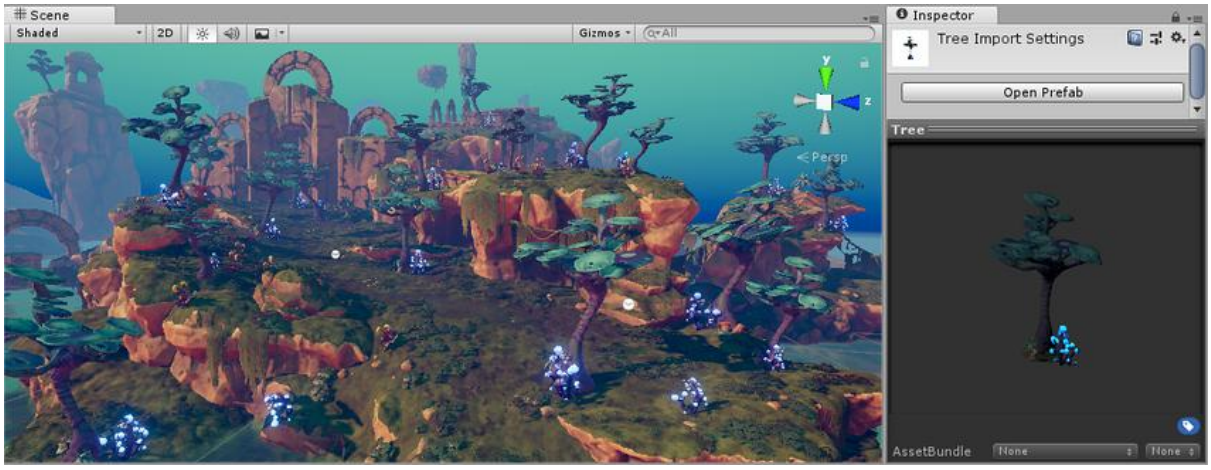


Ilustración 13. Ejemplo de prefab en una escena. Fuente: https://docs.unity3d.com/es/2018.4/uploads/Main/PrefabsIntroScene.png

Los prefabs se emplean cuando se quiere reutilizar un objeto configurado de una forma concreta, como un personaje secundario, un elemento del escenario o un objeto interactivo. A diferencia de copiar y pegar un GameObject, el sistema de Prefabs permite mantener todas las copias sincronizadas, de modo que cualquier cambio realizado en el Prefab original se aplica automáticamente a todas sus instancias. Esto facilita realizar modificaciones globales en el proyecto sin tener que repetir el mismo cambio en cada objeto individual. Unity también permite crear estructuras más complejas mediante el uso de prefabs anidados, es decir, prefabs que contienen otros prefabs en su interior. Esta funcionalidad facilita la organización de elementos complejos y permite editar diferentes niveles de un mismo objeto de manera más ordenada y eficiente. A pesar de que los prefabs funcionan como plantillas, no todas sus instancias tienen por qué ser idénticas. Unity permite modificar ciertos valores en instancias individuales sin afectar al Prefab original, lo que se conoce como sobrescrituras. Además, es posible crear variantes de un prefab, que permiten definir diferentes versiones de un mismo objeto manteniendo una base común. Por último, los Prefabs también se utilizan cuando es necesario crear objetos de forma dinámica durante la ejecución del juego. Esto incluye elementos como proyectiles, efectos visuales, objetos coleccionables o personajes que aparecen en momentos concretos de la partida. Gracias al sistema de Prefabs, Unity facilita la reutilización, organización y gestión de estos elementos, contribuyendo a un desarrollo más eficiente y estructurado del videojuego.

Si nos fijamos bien, Unity emplea escenas para segmentar los juegos, usa los GameObjects para poner los elementos en las escenas, se emplean los scripts y las animaciones para aplicar la lógica y movimiento a los GameObjects, respectivamente, y se emplean los prefabs para poder crear múltiples instancias de un GameObject y poder modificarlos sin tener que ir

haciéndolo uno a uno. Pero hay un elemento que Unity emplea que es muy útil y mucha gente decide usar Unity por esta razón; ese elemento es la Unity Asset Store.

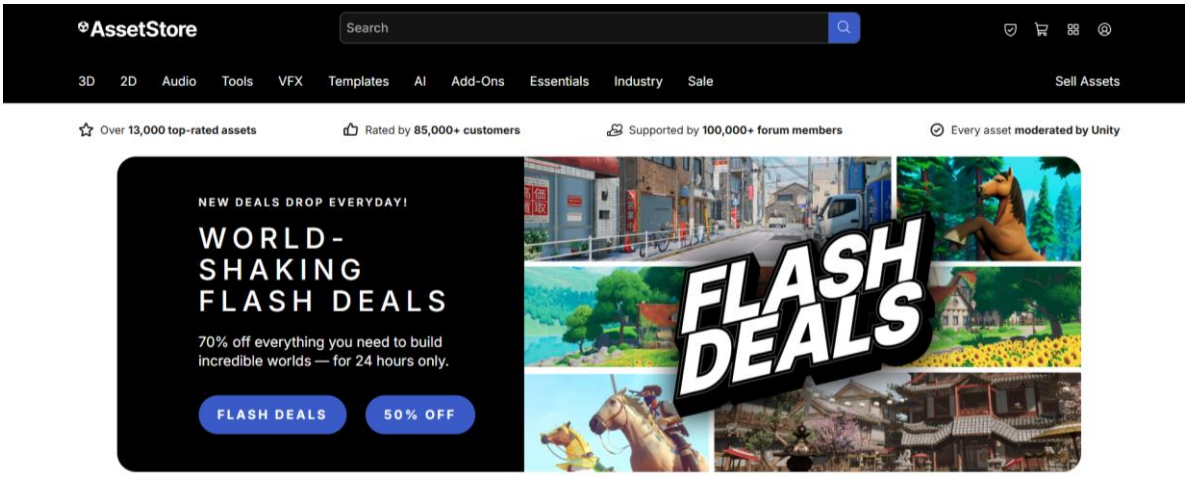


Ilustración 14.Captura de la Unity Asset Store. Fuente: https://assetstore.unity.com/

La Unity Asset Store es una plataforma integrada al motor que permite a los desarrolladores acceder a una gran cantidad de recursos ya creados como son modelos, sonidos, scripts, animaciones y herramientas. Estos recursos, que pueden ser gratuitos o de pago, permiten acelerar el desarrollo de los videojuegos pues permiten reutilizar contenido y centrarse en otros aspectos del proyecto. La Unity Asset Store es muy útil para agilizar el desarrollo de videojuegos, ya que te ofrece herramientas y assets que te permiten crear juegos de forma más eficiente y rápida. También es muy buena para gente que quiera aprender a desarrollar videojuegos, ya que simplifica partes del proceso de desarrollo de videojuegos o directamente hace por ti ciertos elementos del proceso.

2.4.1 Características de Unity

Unity es un motor de videojuegos potente que ofrece todo tipo de herramientas para desarrollar todo tipo de juegos. Si bien no haré uso de la Unity Asset Store para este proyecto, tengo experiencia previa con Unity y en proyectos anteriores sí que la he usado. He de decir que la Asset Store es muy útil; se requiere salir de la aplicación de Unity y se requiere internet, pero en ella se encuentra todo tipo de assets y extensiones para poder realizar tus juegos de manera más rápida.

Es bastante fácil aprender a usar Unity; la documentación está en bastantes idiomas, es muy clara y, aparte de tener contenido para funciones y sentencias, incluye muchos tutoriales, recursos y ejemplos de proyecto de todo lo que uno pueda querer para aprender a desarrollar videojuegos. Ha sido de gran ayuda el manual de Unity para el desarrollo de juegos en 2D.

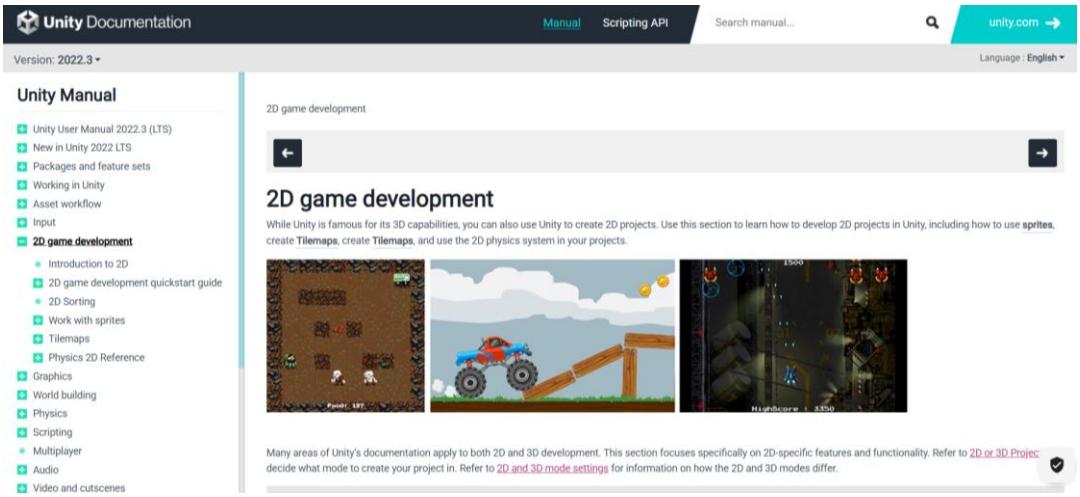


Ilustración 15. Captura del Unity Manual de desarrollo de juegos 2D. Fuente: https://docs.unity3d.com/2022.3/Documentation/Manual/Unity2D.html

La comunidad de Unity es muy activa y extensa; si tienes cualquier duda, puedes pedirla por los foros de discusiones de Unity. Esto es un gran recurso, aunque no he hecho uso de él para este proyecto.

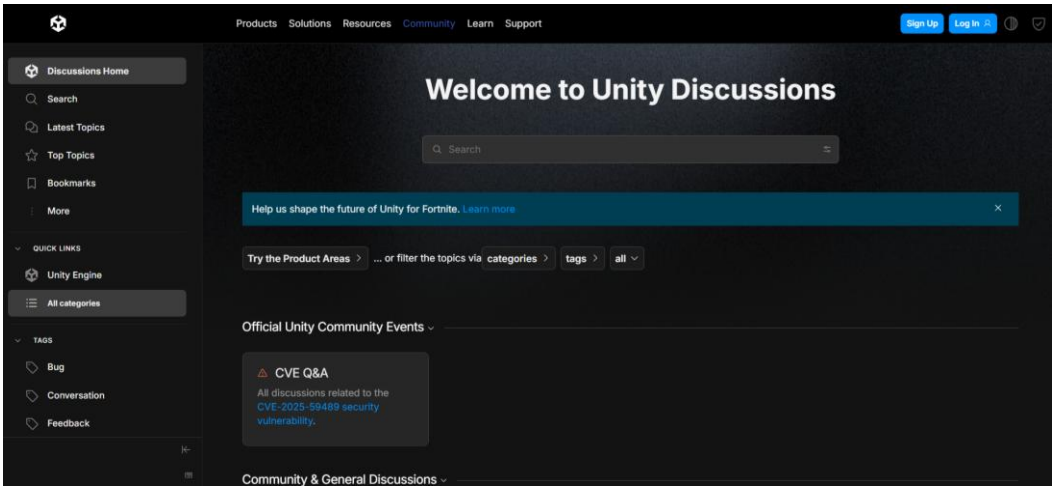


Ilustración 16. Captura del foro de discusiones Unity Discussions. Fuente: https://discussions.unity.com/

Otra cosa muy buena que tiene Unity y que me será de gran ayuda para el desarrollo del juego es la cantidad de tutoriales de desarrollo de cualquier tipo de juego que se encuentran en internet sobre Unity. Da igual el tipo de juego que quieras hacer; si buscas un tutorial de cómo hacerlo en Unity en una plataforma como YouTube, es muy probable que encuentres varios. Esto es debido a que, al ser un motor de videojuegos tan popular y con tantos años en el mercado, ha llevado a que exista un amplio repertorio de tutoriales online hechos por la comunidad.

Por todas las razones mencionadas anteriormente, Unity es un motor de videojuegos idóneo para aprender a desarrollar videojuegos. Tiene muchos tutoriales, gran documentación y una comunidad activa. Si bien C# es un lenguaje que puede llegar a ser complicado y laborioso, el programa en este lenguaje es una tarea que se puede llevar a cabo, ya que tienes infinidad de recursos a tu disposición para lograr tu objetivo. Es por esto que Unity es una opción muy sólida para el desarrollo de videojuegos en 2D, pero esto no significa que no tenga puntos negativos. Para proyectos sencillos como el juego que desarrollaré, la potencia de Unity es demasiado y no compensa el tiempo que tarde en crear un proyecto y compilar los códigos. Para proyectos de menor tamaño, quizá sea recomendable usar un motor de videojuegos menos potente, pero que no tarde tanto en crear un proyecto, compilar código y no genere proyectos tan pesados. Para los indies, quizá sea más acertado un motor de videojuegos más lightweight.

Hablando de los indies, el precio es algo que también hay que tener en cuenta; este será mayor cuanto más unidades vendas, por lo que para un indie es algo agradable, aunque igualmente tendría que pagar una comisión, mientras que algunos motores no piden nada. Otro elemento importante es la reputación; últimamente Unity ha tomado una serie de malas decisiones, desde cambios injustos en el método de pago hasta problemas dentro de la comunidad. Esto es otro factor a tener en cuenta al elegir un motor, ya que si planeas hacer un juego que normalmente lleva años o meses completar, quieres un motor de videojuegos estable que no haga cambios radicales en elementos cruciales que puedan perjudicar tu proyecto, cosa que con Unity hay peligro de que suceda.

Por otro lado, el rendimiento de los juegos desarrollados en el motor es excelente; por muchos GameObjects con scripts que haya en los momentos más concurridos, el juego será jugable y rendirá fenomenalmente, generalmente.

En conclusión, Unity es un motor de videojuegos potente, perfecto para aprender gracias a su extensa documentación, tutoriales y comunidad. También tiene un modelo de negocio basado en la cantidad de unidades vendidas, lo cual ayuda a los indies. No es tan lightweight como otros y tiene un cierto coste, pero debido a su popularidad y sencillez, es un motor de videojuegos muy sólido para el desarrollo de juegos 2D.

2.5 Sobre Godot

Godot es un motor de videojuegos 2D y 3D multiplataforma, libre y de código abierto, publicado bajo la Licencia MIT (permite reutilizar software dentro de software propietario). Fue desarrollado inicialmente en Buenos Aires por los argentinos Ariel Manzur y Juan Linietsky y luego por la comunidad de Godot. El motor funciona en sistemas Linux, Windows, macOS y BSD, además de tener soporte experimental para Android y HTML. [5]

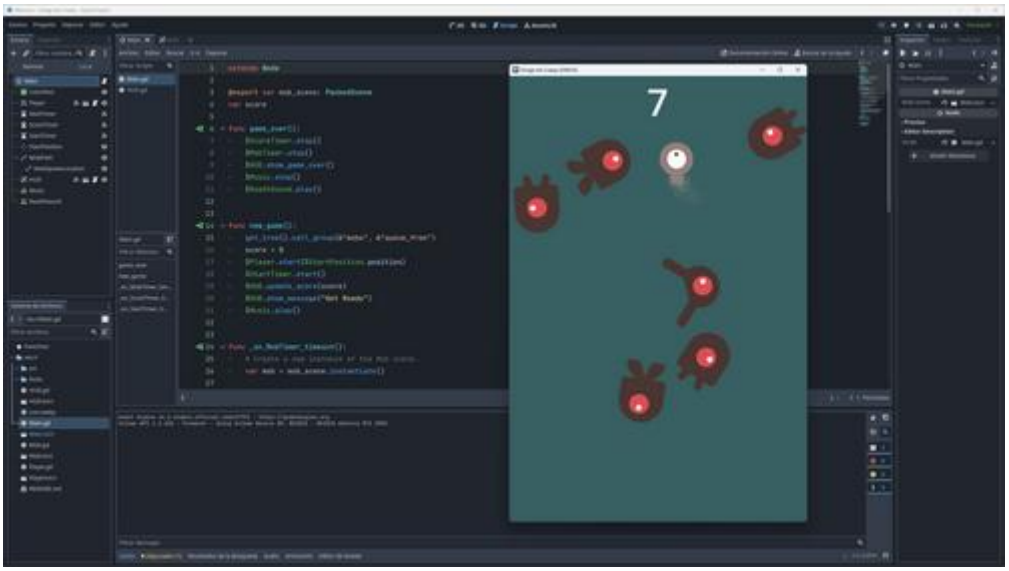


Ilustración 17. Fotografía de la interfaz de Godot. Fuente: https://upload.wikimedia.org/wikipedia/commons/thumb/d/d4/Godot_4.2_Spanish.webp/500px-Godot_4.2_Spanish.webp.png

En el año 2001 aproximadamente, la empresa argentina OKAM Studios comenzó a desarrollar y usar Godot internamente. Durante muchos años fue una herramienta privada, hasta que en febrero de 2014 su código fuente fue publicado de manera abierta en la plataforma GitHub bajo la Licencia MIT. Esta decisión permitió que la comunidad pudiera acceder al motor, modificarlo y participar en su desarrollo, un hito fundamental en su evolución.

Godot 1.0, la primera versión estable del motor, fue lanzada a finales de 2014. En esta versión se incluyeron características importantes como sistemas de iluminación precalculada, soporte para navegación de personajes y mejoras en los shaders, por lo que era una opción viable para el desarrollo de videojuegos completos. A partir de entonces, Godot empezó a ganar adeptos, en especial, entre los desarrolladores independientes.

Las versiones Godot 3.x contaron con soporte a largo plazo, recibiendo actualizaciones centradas en el mantenimiento, la corrección de errores y la estabilidad del motor. Esto permitió que muchos proyectos se desarrollaran sobre esta versión durante varios años sin problemas de compatibilidad.

Más adelante, ya en marzo de 2023, salió Godot 4.0, una actualización importante que trajo mejoras técnicas relevantes, como un nuevo sistema de renderizado basado en Vulkan, iluminación y reflejos en tiempo real y una iluminación global más avanzada. En la actualidad, Godot sigue evolucionando y ya se encuentra en su versión 4.4, publicada en marzo de 2025, afianzándose como una alternativa moderna, potente y de código abierto dentro del desarrollo de videojuegos.

En cuanto a las características de Godot, el motor cuenta con un amplio conjunto de herramientas que buscan simplificar el desarrollo de videojuegos en 2D y 3D. Una de las

principales ventajas que ofrece es que permite exportar los proyectos a múltiples plataformas, como Windows, Linux, macOS, Android, iOS o navegadores web mediante HTML. Godot no tiene soporte oficial para consolas, pero existen soluciones de terceros para llevar a plataformas como PlayStation, Xbox o Nintendo Switch proyectos desarrollados en Godot.

Sobre el apartado gráfico, Godot cuenta con un motor de renderizado moderno que soporta tecnologías como OpenGL, Vulkan o Direct3D. Esto permite ajustar el rendimiento y la calidad visual para la plataforma de destino. Además el motor incluye su propio sistema de sombreadores escrito en GLSL, que se utiliza tanto en objetos como en partículas y materiales. Estos sombreadores pueden crearse visualmente con herramientas del editor o mediante código, por lo que son accesibles tanto para artistas como para programadores.

Godot también cuenta con un editor de animaciones integrado que permite trabajar con distintos tipos de animación, como animación por fotogramas, animación esquelética y cinemáticas. El sistema de animación no se limita únicamente al movimiento de personajes, sino que permite animar propiedades de cualquier elemento del juego, como sonidos, llamadas a funciones o cambios visuales a lo largo del tiempo, ofreciendo una gran flexibilidad creativa.

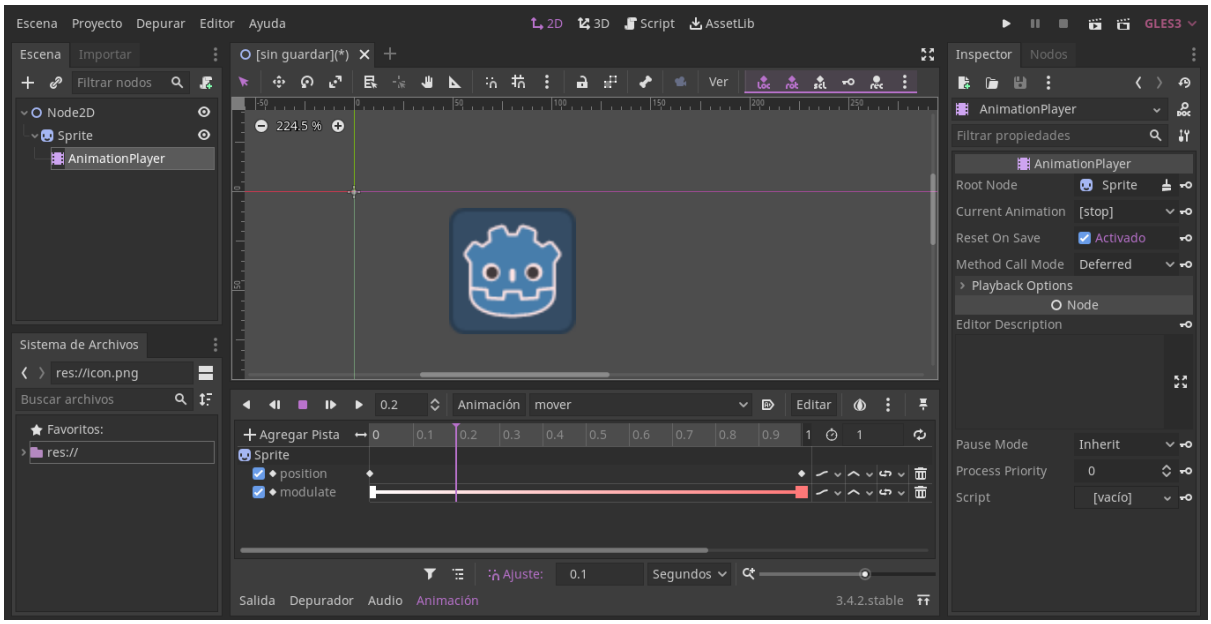


Ilustración 18. Captura del editor de animación de Godot. Fuente: https://upload.wikimedia.org/wikipedia/commons/6/61/Editor_de_animaci%C3%B3n_de_Godot_3.4.2.png

En el apartado de físicas, Godot incorpora un motor propio para juegos en 2D y 3D que gestiona colisiones, cuerpos físicos, personajes, vehículos y detección mediante raycasts. Este sistema permite simular comportamientos realistas sin necesidad de herramientas externas. Actualmente, el desarrollo del motor avanza hacia la integración del motor de físicas Jolt, con el objetivo de mejorar el rendimiento y la precisión de las simulaciones.

Respecto a la programación, Godot soporta varios lenguajes de scripting. El principal es GDScript, un lenguaje creado específicamente para el motor, con una sintaxis similar a Python y diseñado para facilitar el desarrollo rápido de videojuegos. Además, el motor permite utilizar C# y C++ mediante extensiones, y la comunidad ha desarrollado soporte adicional para otros lenguajes como Rust o JavaScript, así como opciones de programación visual.

El motor contiene muchas otras características adicionales que hacen de él una completa herramienta para el desarrollo de videojuegos. Estas incluyen el manejo de varios tipos de

entrada como teclado, ratón, mando o pantallas táctiles, herramientas para crear interfaces gráficas, sistemas de navegación y búsqueda de rutas, ejecución en múltiples hilos, capacidades de red para juegos en línea, y soporte para dispositivos de realidad extendida. Todo esto hace de Godot un motor versátil y adaptable a una gran variedad de proyectos.

Godot es un motor bastante lightweight, pesa menos y tiene una gran velocidad para crear y abrir proyectos. Asimismo, la Asset Store de Godot es muy potente y se encuentra dentro del mismo motor, pudiendo acceder a ella sin salir de este, lo cual la hace bastante práctica de usar.

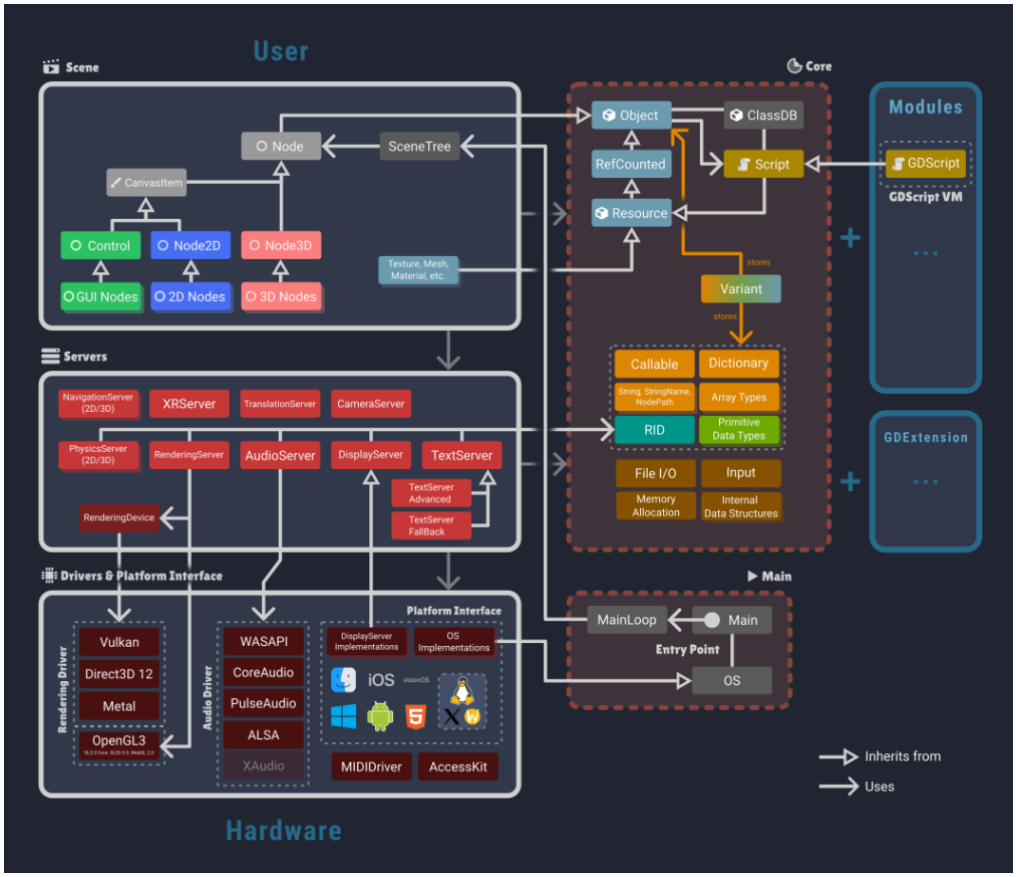


Ilustración 19. Fotografía de la arquitectura de Godot. Fuente: https://docs.godotengine.org/en/stable/_images/godot-architecture-diagram.webp

Una de las mejores funciones de Godot es su versatilidad, ya que el motor tiene soporte para desarrollo 2D y 3D, sistemas de físicas, animación, scripting y exportación a múltiples plataformas. Otro factor que hace muy atractivo a Godot es que es un motor de código abierto y gratuito; debido a su naturaleza de motor open source, el beneficio que pueda generar un juego de un desarrollador indie irá todo directamente a él; Godot no se lleva ninguna parte.

El carácter open source de Godot también significa que se pueden hacer forks del motor; de hecho, existen muchos forks creados. Un fork es la creación de una copia o una versión derivada de un proyecto existente para desarrollarlo de manera independiente a partir de ese punto. Uno de los forks más recientes y más populares de Godot es Redot. Tras los cambios en la monetización que implementó Unity, muchos usuarios se pasaron a Godot, por lo que el motor ganó más popularidad que nunca. Seguido de este evento, los community managers de Godot fueron acusados de seguir prácticas excluyentes en Discord y sus otras redes sociales. Aparte de esto, Godot se politizó por culpa de los community managers que llevaban un tiempo creando sus cámaras de eco en Discord y en otras redes, excluyendo a cualquiera que tuviese una opinión contraria a la suya. Esto llevó a un gran descontento en la comunidad

de Godot y de desarrolladores de videojuegos en general. Las cosas empeoraron cuando los community managers empezaron a bannear en masa a gente en redes sociales y hasta a contribuidores en el GitHub del motor de videojuegos. Se banearon usuarios por tener opiniones distintas, por cosas triviales e incluso se baneó a gente que pedía que se centraran en el motor de videojuegos y no en política. Todo esto llevó a la creación de Redot, un fork de Godot que está en contra de la censura de opiniones y que prioriza el desarrollo del motor y no la política. Redot quiere mantener la comunidad unida, sin importar la ideología de la gente; además, con Redot pretenden que el motor de videojuegos vuelva a ser impulsado por la comunidad. El canal de YouTube de Redot tiene un video en el que se explica la situación y hablan sobre sus valores e ideales. [6]



Ilustración 20. Logo de Redot, fork de Godot. Fuente: https://img.itch.zone/aW1nLzE5MDQ3MDIyLnBuZw==/original/Oh3%2BoK.png

2.5.1 Características de Godot

Godot es un motor de videojuegos muy completo que ofrece muchas herramientas para el desarrollo de videojuegos en 2D y 3D. Una de las características más importantes de Godot es que es completamente gratuito y de código abierto, lo que significa que cualquier desarrollador puede usarlo sin pagar licencias ni comisiones por ventas. Esto lo convierte en una opción muy buena para desarrolladores independientes que no disponen de grandes recursos económicos. Además, al ser open source, la comunidad puede contribuir directamente a su desarrollo, lo que hace que evolucione constantemente.

El editor de Godot es muy intuitivo y ligero en comparación con otros motores. Crear un proyecto nuevo es rápido y el programa se inicia en poco tiempo, lo que facilita trabajar de forma ágil durante el desarrollo. La estructura del motor se basa en un sistema de nodos que se organizan en forma de árbol para crear escenas. Este sistema permite estructurar los elementos del juego de una forma clara y modular, lo que facilita la organización del proyecto y la reutilización de elementos.

Otro aspecto importante de Godot es su documentación. El manual oficial del motor es muy completo y contiene explicaciones detalladas sobre prácticamente todas las funciones del motor. Además, incluye numerosos ejemplos y guías que ayudan a los desarrolladores a aprender cómo funciona el motor y cómo crear distintos tipos de mecánicas de juego. Este manual es especialmente útil para quienes están empezando en el desarrollo de videojuegos.

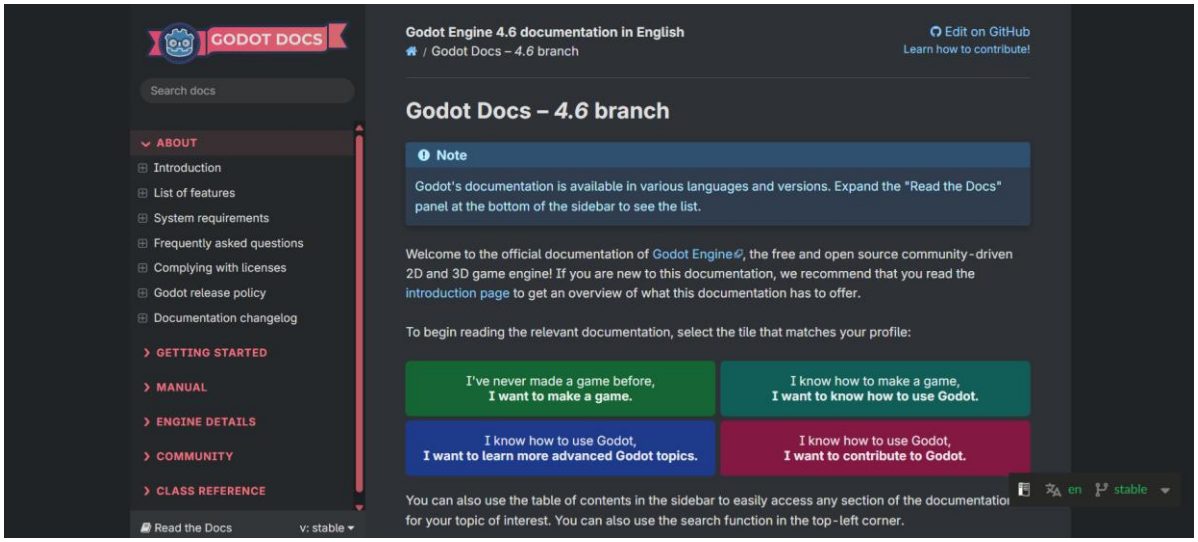


Ilustración 21. Captura del Godot Documentation. Fuente: https://docs.godotengine.org/en/stable/

La comunidad de Godot también es muy activa y ofrece una gran cantidad de recursos para aprender a utilizar el motor. Existen foros oficiales, servidores de Discord y muchas páginas web donde los desarrolladores comparten soluciones a problemas, tutoriales y ejemplos de proyectos. Gracias a esta comunidad, es relativamente sencillo encontrar ayuda cuando surge alguna duda durante el desarrollo de un juego.

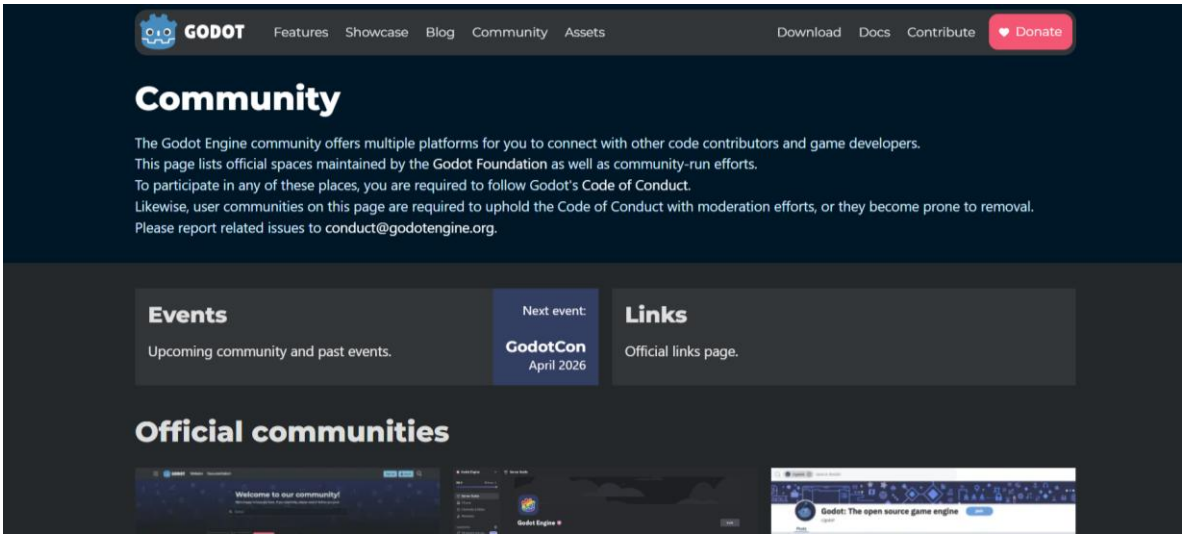


Ilustración 22. Captura del foro de Godot Community. Fuente: https://docs.godotengine.org/en/stable/

Además de la documentación oficial, también existe una gran cantidad de tutoriales disponibles en internet. En plataformas como YouTube es posible encontrar guías para desarrollar todo tipo de juegos en Godot, desde proyectos sencillos hasta sistemas más complejos. Aunque la cantidad de tutoriales no es tan grande como en otros motores, el número de recursos disponibles ha crecido mucho en los últimos años debido al aumento de popularidad del motor.

Otro punto fuerte de Godot es su lenguaje de programación principal, llamado GDScript. Este lenguaje fue diseñado específicamente para el motor y su sintaxis es similar a Python, lo que lo hace bastante sencillo de aprender. Gracias a esto, programar mecánicas de juego en Godot suele ser un proceso bastante rápido y accesible incluso para desarrolladores con poca experiencia en programación. Además, el motor también permite utilizar otros lenguajes como C# o C++, lo que proporciona mayor flexibilidad dependiendo de las necesidades del proyecto.

En cuanto al rendimiento, Godot funciona especialmente bien en proyectos 2D. El motor está muy optimizado para este tipo de juegos, por lo que es una herramienta muy adecuada para desarrolladores independientes que quieran crear juegos con gráficos 2D. Además, el motor es bastante ligero, por lo que los proyectos suelen ocupar menos espacio y compilarse más rápido que en otros motores más pesados.

Sin embargo, Godot también tiene algunos inconvenientes. Aunque su comunidad está creciendo rápidamente, todavía es más pequeña que la de otros motores más populares, lo que significa que en algunos casos puede ser más difícil encontrar soluciones específicas a ciertos problemas. Además, aunque el motor soporta desarrollo 3D, en este apartado todavía no alcanza el mismo nivel de madurez que otros motores más especializados.

En definitiva, Godot es un motor de videojuegos muy atractivo para el desarrollo de videojuegos, especialmente para desarrolladores independientes. Su naturaleza gratuita y de código abierto, junto con su editor ligero, su lenguaje de programación sencillo y su buen rendimiento en juegos 2D, lo convierten en una opción muy sólida. Aunque todavía tiene algunas limitaciones en comparación con otros motores más establecidos, su crecimiento constante y su comunidad activa hacen que sea una herramienta cada vez más utilizada en el desarrollo de videojuegos.

2.6 Sobre GameMaker

GameMaker es un motor de videojuegos pensado principalmente para facilitar el desarrollo de videojuegos a usuarios principiantes o con poca experiencia en programación. Fue creado originalmente por Mark Overmars y apareció por primera vez a finales de 1999 como una herramienta educativa, con el objetivo de permitir la creación de juegos de una forma más accesible y sencilla. Desde sus inicios, GameMaker se ha caracterizado por ofrecer un lenguaje propio y un entorno de trabajo que reduce la complejidad técnica, permitiendo centrarse más en el diseño y la lógica del juego que en los detalles de bajo nivel. Aunque durante muchos años fue un software de pago con versiones de prueba limitadas, en la actualidad es de uso gratuito tras la adquisición de la empresa desarrolladora YoYo Games por parte de Opera, lo que ha ampliado su accesibilidad y popularidad entre desarrolladores independientes y aficionados. [7]



Ilustración 23. Logo de GameMaker. Fuente: https://upload.wikimedia.org/wikipedia/commons/5/59/Logo_GameMaker_2022.png

Ahora, un poco sobre la historia de este motor de videojuegos. Su historia comienza en 1999 bajo el nombre de Animo, cuando Mark Overmars empezó a desarrollar una herramienta de animación con fines educativos para sus estudiantes. Con el paso del tiempo, este proyecto fue evolucionando hasta convertirse en una herramienta orientada al desarrollo de videojuegos, sentando las bases de lo que más tarde sería GameMaker. Desde sus primeras versiones, el objetivo principal del programa fue simplificar el proceso de creación de juegos y hacerlo accesible a personas sin grandes conocimientos técnicos.

A partir de 2004, con el lanzamiento de Game Maker 6.0, el motor comenzó a incorporar mejoras importantes, como el uso de Direct3D, lo que permitió añadir efectos visuales más avanzados e incluso trabajar con elementos en 3D. Las versiones posteriores continuaron ampliando sus posibilidades, introduciendo nuevas acciones, soporte para modelos externos y un sistema de extensiones que permitía ampliar el motor de forma relativamente sencilla. Durante estos años, GameMaker fue consolidándose como una herramienta cada vez más completa, manteniendo su enfoque en la facilidad de uso.

Con la llegada de GameMaker Studio en 2012, el motor dio un salto significativo al permitir la exportación de juegos a múltiples plataformas, como dispositivos móviles, navegadores web y distintos sistemas operativos de escritorio. Este cambio marcó una etapa clave en la evolución del motor, ya que abrió la puerta al desarrollo multiplataforma y al acceso a consolas mediante acuerdos con grandes compañías. Finalmente, el lanzamiento de GameMaker Studio 2 a partir de 2016 supuso una modernización del entorno de trabajo, adaptándolo a nuevas necesidades y consolidando a GameMaker como una herramienta ampliamente utilizada en el desarrollo de videojuegos independientes.

Sobre las características de GameMaker, es un motor de desarrollo de videojuegos diseñado para que tanto usuarios principiantes como desarrolladores avanzados puedan crear juegos de manera accesible y eficiente. Una de sus principales ventajas es que permite construir

videojuegos sin necesidad de conocer lenguajes de programación complejos como C++ o Java, gracias a su interfaz basada en un sistema de “arrastrar y soltar”. Este método permite a los usuarios organizar acciones mediante íconos y listas, facilitando la creación de movimientos, controles básicos y lógica del juego de forma intuitiva. Al mismo tiempo, los usuarios más experimentados pueden aprovechar Game Maker Language (GML), un lenguaje propio del motor, para acceder a todas sus funcionalidades y programar juegos más complejos, incluyendo proyectos en 3D o con mecánicas avanzadas.

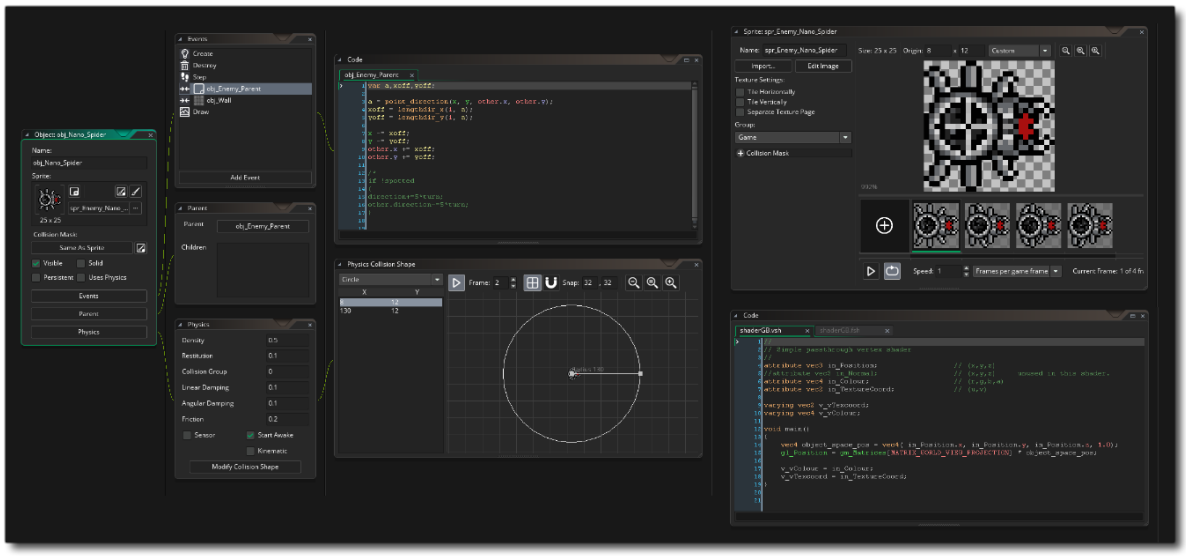


Ilustración 24. Interfaz de GameMaker. Fuente: https://manual.gamemaker.io/lts/es/assets/Images/Introduction/MainPage.png

El desarrollo en GameMaker se centra en tres elementos principales: los recursos, que incluyen gráficos, sonidos y fondos; los objetos, que representan los elementos activos del juego; y los eventos, que determinan cómo reaccionan los objetos ante acciones del jugador o del entorno. Cada uno de estos elementos puede configurarse mediante la interfaz visual o con GML, ofreciendo un equilibrio entre facilidad de uso y flexibilidad. Además, GameMaker permite ampliar sus capacidades mediante bibliotecas de acciones personalizadas o el uso de bibliotecas externas en lenguajes como C++, Delphi o Pascal, lo que ofrece aún más opciones de personalización y expansión de los proyectos.

Otro aspecto importante de GameMaker es su compatibilidad con distintos tipos de distribución. Los juegos creados pueden exportarse a archivos ejecutables para Windows, paquetes para Android o incluso scripts HTML5, permitiendo que los desarrolladores publiquen y vendan sus creaciones de manera sencilla. Gracias a estas características, GameMaker se ha consolidado como un motor versátil y accesible, capaz de cubrir las necesidades de principiantes que buscan crear su primer videojuego, así como de desarrolladores avanzados que desean proyectos más ambiciosos, incluyendo juegos multijugador, plataformas, shooters o simulaciones. La combinación de facilidad, flexibilidad y adaptabilidad ha hecho que GameMaker mantenga una base de usuarios amplia y activa a lo largo de los años.

GameMaker cuenta con varias ediciones o licencias distintas para el uso de su software. La edición Standard es gratuita y proporciona la mayoría de las funciones básicas del programa, lo que la hace ideal para principiantes o desarrolladores que quieren experimentar sin coste inicial. Esta versión no incluye licencia comercial alguna. Por otro lado, la edición Professional es de pago e incluye una licencia comercial que permite publicar juegos con fines comerciales. Además, amplía las opciones de exportación al incluir plataformas de escritorio, web y móviles como Android e iOS desde una misma licencia, lo que la convierte en una

opción adecuada para desarrolladores que quieran distribuir sus proyectos en distintos sistemas. Finalmente, la edición Enterprise incluye todas las funciones de la versión Professional y amplía las opciones de publicación al ofrecer acceso a plataformas adicionales orientadas al desarrollo profesional, especialmente consolas, mediante acuerdos específicos con sus fabricantes. Está pensada principalmente para estudios que necesitan distribuir sus juegos en un mayor número de sistemas.

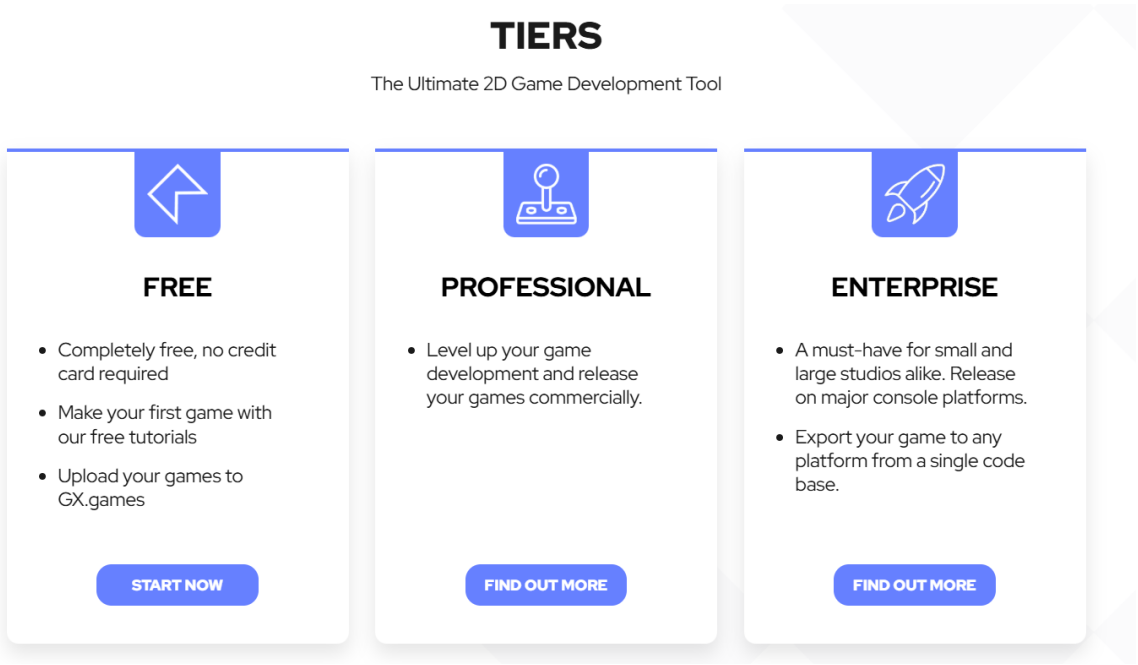


Ilustración 25. Captura de las versiones de GameMaker. Fuente: https://gamemaker.io/en

La popularidad de GameMaker es innegable, a pesar de no hacer 3D. Esto es debido a que es muy simple de aprender y usar y, al estar pensado exclusivamente para 2D, tiene más funciones y es más eficiente realizando juegos en este ámbito, por lo que muchos desarrolladores con un enfoque 2D lo usan.

2.6.1 Características de GameMaker

GameMaker es muy popular dentro del desarrollo de videojuegos independientes, especialmente para la creación de juegos en 2D. El motor está pensado principalmente para desarrollar videojuegos de forma sencilla y rápida, reduciendo la complejidad técnica y permitiendo que incluso personas con poca experiencia en programación puedan crear sus propios juegos.

Una de las características más importantes del motor es su sistema de desarrollo basado en "arrastrar y soltar". Este sistema permite crear la lógica del juego mediante acciones visuales sin necesidad de escribir código, lo que facilita mucho el aprendizaje del programa. Gracias a esta herramienta, los usuarios pueden crear mecánicas básicas organizando acciones dentro del editor de manera intuitiva. Sin embargo, para proyectos más complejos, GameMaker también ofrece su propio lenguaje de programación llamado Game Maker Language (GML), que permite tener un control mucho más detallado sobre el comportamiento del juego.

La documentación de GameMaker también es bastante completa y resulta de gran ayuda para aprender a utilizar el motor. En el manual oficial se pueden encontrar explicaciones detalladas de las funciones del programa, así como ejemplos de código y guías para desarrollar distintos tipos de juegos. Este manual es especialmente útil para comprender cómo funcionan los objetos, los eventos y los sistemas de scripting dentro del motor.

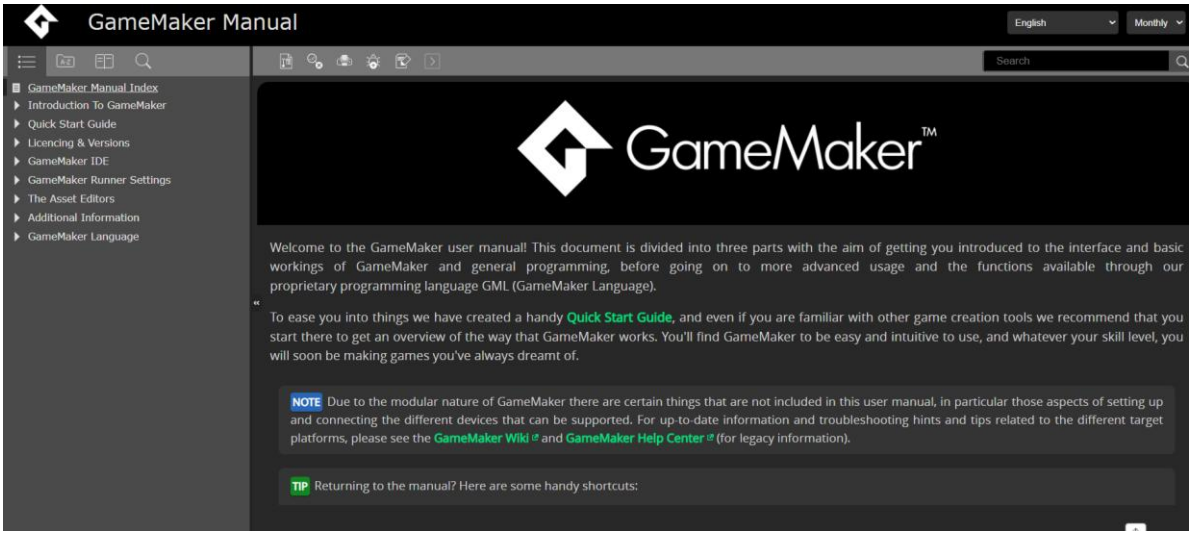


Ilustración 26. Captura del manual oficial de GameMaker. Fuente: https://manual.gamemaker.io/monthly/en/#t=Content.htm

La comunidad de GameMaker también es bastante activa y cuenta con numerosos recursos en internet. Existen foros oficiales, páginas de discusión y comunidades en distintas plataformas donde los desarrolladores comparten tutoriales, ejemplos de código y soluciones a problemas comunes. Esto facilita mucho el aprendizaje del motor, ya que es posible encontrar ayuda rápidamente cuando surge alguna dificultad durante el desarrollo de un proyecto.

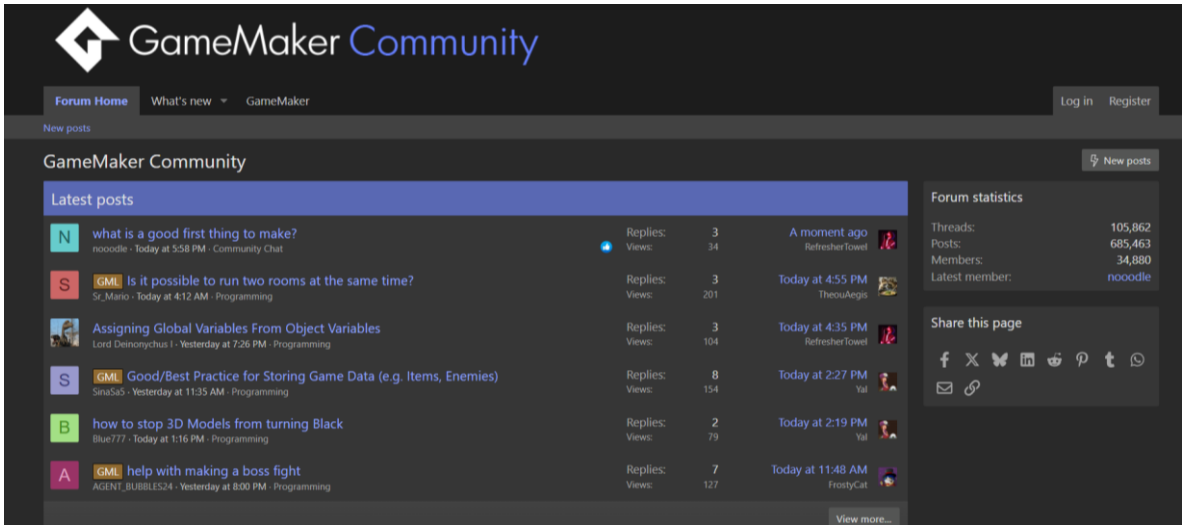


Ilustración 27. Captura del foro de la comunidad de GameMaker. Fuente: https://manual.gamemaker.io/monthly/en/#t=Content.htm

Otro punto muy positivo del motor es la gran cantidad de tutoriales disponibles en internet. Debido a que el motor lleva muchos años siendo utilizado por desarrolladores independientes, existe una gran cantidad de contenido educativo en plataformas como YouTube o páginas especializadas. Es posible encontrar tutoriales para desarrollar todo tipo de juegos en 2D, como plataformas, shooters, juegos de rol o puzzles, lo que facilita mucho el proceso de aprendizaje.

GameMaker funciona especialmente bien en proyectos 2D. El motor está optimizado para este tipo de juegos, por lo que es una herramienta muy adecuada para desarrolladores que quieren crear proyectos pequeños o medianos sin tener que lidiar con motores más complejos. Además, el entorno de desarrollo es bastante ligero y los proyectos suelen compilarse rápidamente.

Por otro lado, GameMaker también tiene algunas limitaciones. A diferencia de otros motores, sus capacidades para el desarrollo en 3D son bastante limitadas, ya que el motor está diseñado principalmente para trabajar en 2D. Esto hace que no sea la mejor opción para proyectos que requieran gráficos tridimensionales avanzados o sistemas más complejos.

En resumen, GameMaker es muy adecuado para el desarrollo de videojuegos en 2D, especialmente para desarrolladores independientes o principiantes. Su sistema de arrastrar y soltar facilita el aprendizaje, mientras que el lenguaje GML permite desarrollar proyectos más complejos cuando es necesario. Aunque tiene algunas limitaciones en comparación con otros motores más completos, su sencillez, rapidez y enfoque en el desarrollo 2D lo convierten en una herramienta muy sólida para crear videojuegos de este tipo.

3 El juego

Como he mencionado anteriormente, la idea del proyecto es crear el mismo juego 2D en tres motores de videojuegos distintos populares para compararlos. La parte práctica de este proyecto sería la creación del juego, que será un juego 2D top-down, es decir, un juego en dos dimensiones visto desde arriba, en el que el jugador controlará una nave espacial y deberá enfrentarse a otras naves espaciales enemigas. El juego estará inspirado en el juego Asteroids, aunque se controlará como un shooter top-down, donde con las teclas “wasd” se moverá el personaje y con el ratón se rotará, apuntando a los enemigos a los que con clic izquierdo podrás dispararles.

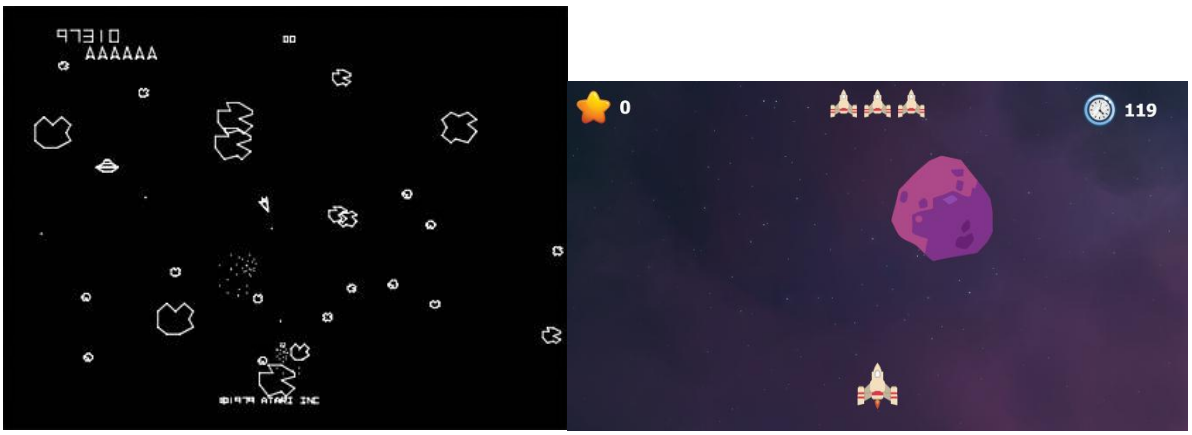


Ilustración 28. Fotografías de Asteroids original (izquierda) y juego inspirado en Asteroids (derecha). Fuentes: https://i0.wp.com/www.elpixelilustre.com/wp-content/uploads/2015/10/Asteroids.jpg?ssl=1 https://plays.org/screenshots/asteroids-game-start.png

Si bien me inspiro en varios elementos del juego Asteroids, el gameplay será más como un shooter top down genérico. El jugador contará con cinco vidas; tras ser golpeado, el jugador parpadeará durante 2 segundos y se le descontará una vida. Durante este periodo, el jugador no podrá recibir más daño hasta que deje de parpadear. El jugador se puede mover con las teclas “wasd”, con el ratón rotará, con el clic izquierdo disparará y con la tecla “p” podrá acceder al menú de pausa. Los enemigos aparecerán en oleadas cada vez más frecuentes, los enemigos mantendrán una cierta distancia con el jugador y se moverán de lado a lado para evadir sus ataques. Al morir, los enemigos tienen una probabilidad de soltar un power-up con forma de rayo que, al cogerlo, aumenta la cadencia de tiro del jugador por un tiempo limitado. El objetivo del juego es alcanzar el máximo tiempo sobrevivido posible matando al mayor número de enemigos posible. Los enemigos darán todos cien puntos que se podrán ver en un texto de puntuación durante la partida y en la pantalla de game over. En esta pantalla, también se podrá ver el tiempo total sobrevivido. Basándome en juegos antiguos que tenían diferencias en versiones para distintas consolas, el juego variará estéticamente un poco dependiendo de qué motor de videojuegos se haya creado. El funcionamiento será el mismo, pero tendrá diferencias visuales para hacer cada versión más única.

Usaré un asset pack de la página Kenney para aligerar el desarrollo y tener un apartado visual de calidad. El asset pack en concreto se llama *Space Shooter Redux*.

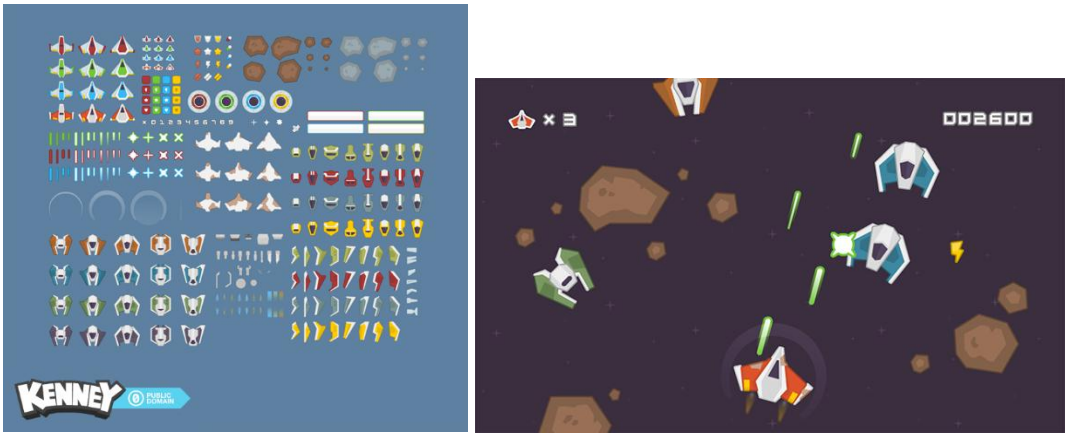


Ilustración 29. Fotografías del asset pack Space Shooter Redux. Fuente: https://www.kenney.nl/assets/space-shooter-redux

Aparte del asset pack de Kenney, usaré una fuente llamada Alien Invader sacada de la página web DaFont y he usado música proveniente de la página OpenGameArt. Estos mismos assets serán usados para el desarrollo en los otros motores de videojuegos.

3.1 Desarrollo en Unity

3.1.1 Diagrama de clases

Para el desarrollo en Unity, he creado un diagrama de clases del proyecto. El diagrama está compuesto de doce clases distintas. Estas clases estructuran la lógica del juego, organizando las diferentes funcionalidades en componentes independientes que interactúan entre sí dentro del motor Unity. La separación en distintas clases permite mantener un diseño modular, facilitando el mantenimiento del código y la reutilización de componentes.

Sobre las distintas clases, la clase PlayerController es la encargada de gestionar el comportamiento del jugador dentro de la escena de juego. Esta clase controla el movimiento del personaje en el espacio 2D, la rotación hacia la posición del cursor y la acción de disparo mediante el uso de un arma asociada. Además, se encarga de la gestión de las vidas del jugador, incluyendo la lógica de recibir daño, la activación de un breve periodo de invulnerabilidad tras recibir un impacto y la comprobación de la condición de muerte del jugador. Cuando el número de vidas llega a cero, se activa la transición hacia la escena de fin de partida.

EnemyController es una clase que define el comportamiento de los enemigos del juego. Esta clase implementa la lógica de movimiento hacia el jugador manteniendo una distancia mínima determinada, así como un desplazamiento lateral que genera un comportamiento más dinámico y orgánico. Además, se encarga de rotar continuamente hacia la posición del jugador para poder apuntar correctamente y ejecutar disparos a intervalos definidos utilizando un sistema de arma similar al del jugador.

La clase EnemyHealth gestiona la vida de los enemigos. Su función principal consiste en detectar si el laser del enemigo ha impactado. En caso afirmativo, el enemigo es destruido y se añaden cien puntos a la puntuación del jugador a través del sistema de gestión de puntuación. Esta clase también es usada para soltar power-ups que el player pude recoger.

Weapon representa el sistema de disparo utilizado tanto por el jugador como por los enemigos. Su responsabilidad principal es instanciar los proyectiles del juego y aplicar la fuerza necesaria para que estos se desplacen en la dirección adecuada. Además, esta clase permite modificar temporalmente la velocidad de disparo cuando se recoge un power-up.

Laser representa los proyectiles generados por las armas. Estos proyectiles se encargan de detectar colisiones con otros objetos del entorno, aplicar daño a los objetivos correspondientes y destruirse tras impactar o abandonar los límites visibles de la pantalla. De esta manera, se gestiona el ciclo completo de vida de cada disparo realizado en el juego.

La clase EnemySpawner contiene la lógica de generación de enemigos. Esta clase crea nuevas instancias de enemigos en posiciones aleatorias situadas fuera de los límites visibles de la cámara. Gracias a este sistema, se garantiza un flujo continuo de enemigos durante la partida, aumentando progresivamente la dificultad del juego mediante la aparición constante de nuevos adversarios.

PowerUp representa los objetos de mejora que el jugador puede recoger durante la partida. Estos elementos aparecen de forma aleatoria cuando un enemigo es eliminado y, al ser recogidos por el jugador, aplican un efecto temporal que incrementa la velocidad de disparo del arma del jugador.

La clase ScoreManager es la responsable de gestionar la puntuación obtenida durante la partida. Este componente sigue el patrón de diseño Singleton, lo que permite acceder a una única instancia del gestor de puntuación desde cualquier parte del código. Su función principal

es actualizar la puntuación cuando el jugador elimina enemigos añadiendo cien puntos a la puntuación que se verán en la interfaz de usuario.

La clase TimerManager controla el tiempo transcurrido desde el inicio de la partida. Este componente incrementa el contador de tiempo en cada actualización del juego y muestra el resultado en la interfaz de usuario en formato de minutos y segundos. También, dentro de TimerManager hay métodos que permiten detener o reiniciar el temporizador cuando sea necesario en otros componentes.

PauseMenu es la clase que gestiona el sistema de pausa del juego. Permite detener temporalmente la ejecución del juego mediante la modificación del factor de escala temporal del motor. Además, controla la visibilidad del panel de pausa de la interfaz de usuario y contiene funciones para reanudar la partida o regresar al menú principal.

GameOverManager es la clase que controla la escena de fin de partida. Su función consiste en mostrar al jugador los resultados obtenidos durante la partida, incluyendo la puntuación final y el tiempo total de supervivencia. Adicionalmente, proporciona opciones para reiniciar la partida o volver al menú principal.

Por último, la clase MainMenu gestiona la lógica del menú principal del juego. MainMenu se encarga de cargar la escena de juego cuando el jugador decide iniciar una nueva partida y de cerrar la aplicación cuando se selecciona la opción correspondiente. De esta forma, actúa como punto de entrada al sistema de juego.

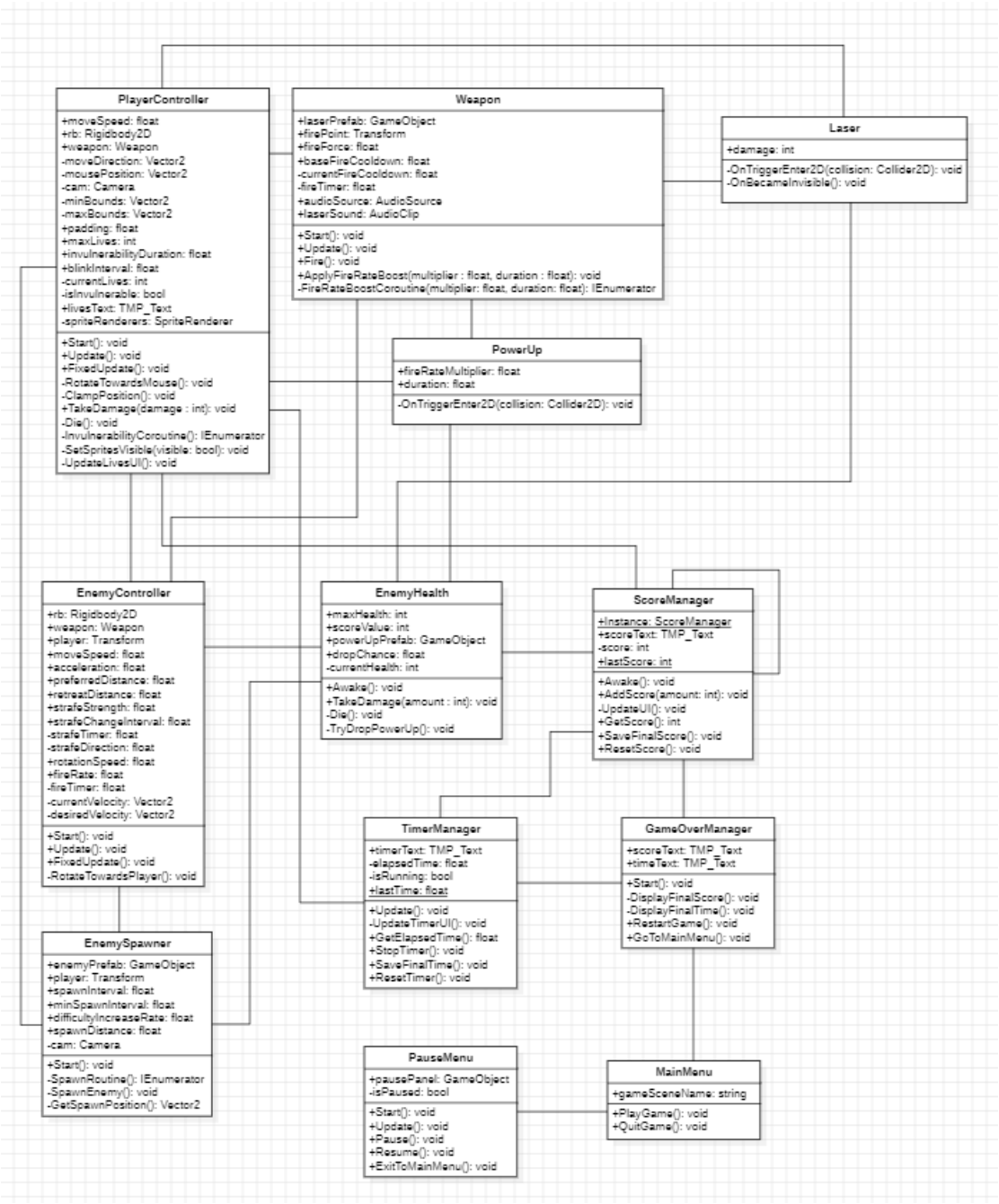


Ilustración 30. Diagrama de clases del proyecto en Unity.

3.1.2 Lógica del jugador

Comenzando con el desarrollo del juego en Unity, primero empiezo creando un proyecto en 2D. Lo primero que hice fue crear un sprite y ponerme con la lógica del player. Empecé con el movimiento en el eje x y el eje y; luego, me puse con la rotación para que siempre mire hacia donde apunta el mouse.

Para hacer el movimiento, creé un script llamado PlayerController, asociado al objeto que representa al jugador dentro de la escena. En Unity, los scripts se implementan como componentes que heredan de la clase MonoBehaviour, lo que permite que el motor ejecute automáticamente determinados métodos en momentos específicos del ciclo de vida del juego.

Primero de todo, definí las variables necesarias para controlar el movimiento del jugador, incluyendo la velocidad de desplazamiento y una referencia al componente físico Rigidbody2D, que es el encargado de gestionar la simulación de físicas en dos dimensiones. Es habitual en Unity trabajar con componentes físicos para gestionar el movimiento, ya que permite integrarlo fácilmente con el sistema de colisiones del motor.

```
6
                Unity Script (1 asset reference) | 2 references
7       public class PlayerController : MonoBehaviour
8       {
9           [Header("Movement")]
10          public float moveSpeed = 5f;
11          public Rigidbody2D rb;
12          public Weapon weapon;
13
14          Vector2 moveDirection;
15          Vector2 mousePosition;
16
```

Ilustración 31. Captura código variables movimiento.

Estas variables permiten almacenar tanto la velocidad de desplazamiento como la dirección de movimiento que se calculará posteriormente a partir de la entrada del usuario. El uso de Vector2 permite representar coordenadas en los ejes X e Y del plano.

Después, implemento el input del usuario dentro del método Update(). Este método se ejecuta una vez por cada frame renderizado, esto quiere decir que es adecuado para capturar la entrada del jugador porque se actualiza constantemente y permite responder de manera inmediata a las acciones del usuario.

```
                Unity Message | 0 references
50      void Update()
51      {
52          float moveX = Input.GetAxisRaw("Horizontal");
53          float moveY = Input.GetAxisRaw("Vertical");
54
55          if (Time.timeScale == 0f)
56              return;
57
58          if (Input.GetMouseButtonDown(0))
59          {
60              weapon.Fire();
61          }
62
63          moveDirection = new Vector2(moveX, moveY).normalized;
64          mousePosition = cam.ScreenToWorldPoint(Input.mousePosition);
65      }
```

Ilustración 32. Captura código del método Update().

En este fragmento de código empleo el sistema de entrada de Unity mediante la función Input.GetAxisRaw(), que permite obtener los valores asociados a los ejes de movimiento configurados en el sistema de input del motor. Estos ejes suelen corresponder, por defecto, a las teclas W, A, S, D o a las flechas del teclado. Los valores obtenidos se utilizan para construir un vector de dirección que posteriormente se normaliza, evitando que el jugador se desplace más rápido en diagonal.

Una característica particular del desarrollo en Unity es la separación entre el método Update() y el método FixedUpdate(). Mientras que Update() se ejecuta una vez por frame, FixedUpdate() se ejecuta a intervalos constantes y está específicamente diseñado para trabajar con el sistema de físicas del motor. Debido a esto, el cálculo de la velocidad del objeto que posee un Rigidbody2D se realiza dentro de FixedUpdate().

```
Unity Message | 0 references
67  void FixedUpdate()
68  {
69      rb.velocity = moveDirection * moveSpeed;
70
71      RotateTowardsMouse();
72
73      ClampPosition();
74  }
```

Ilustración 33. Captura código del método FixedUpdate().

He implementado el movimiento del jugador modificando directamente la propiedad velocity del componente Rigidbody2D. Esto hace que el sistema de físicas del motor se encargue de aplicar el desplazamiento correspondiente en cada actualización de la simulación.

Este método es muy empleado en Unity, donde el control del movimiento suele realizarse manipulando directamente las propiedades del Rigidbody2D, lo que permite una integración más directa con el sistema de físicas del motor.

Además del desplazamiento, el jugador también debe orientarse hacia la posición del cursor del ratón. Para ello calculo el ángulo entre la posición actual del jugador y la posición del cursor en el mundo, utilizando funciones matemáticas proporcionadas por la biblioteca Mathf.

```
1 reference
76  void RotateTowardsMouse()
77  {
78      Vector2 aimDirection = mousePosition - rb.position;
79      float aimAngle = Mathf.Atan2(aimDirection.y, aimDirection.x) * Mathf.Rad2Deg - 90f;
80      rb.rotation = aimAngle;
81  }
```

Ilustración 34. Captura código del método RotateTowardsMouse().

En este método, utilizo la función Mathf.Atan2() para calcular el ángulo entre ambos puntos, convirtiéndolo posteriormente a grados mediante el factor Mathf.Rad2Deg. Finalmente, el valor calculado se asigna a la rotación del Rigidbody2D, permitiendo que el sprite del jugador se oriente automáticamente hacia la posición del cursor.

Ahora, implemento un sistema que limita el movimiento del jugador dentro de los límites visibles de la cámara. Esto evita que el jugador pueda abandonar el área de juego.

```
1 reference
83  void ClampPosition()
84  {
85      float clampedX = Mathf.Clamp(rb.position.x, minBounds.x + padding, maxBounds.x - padding);
86      float clampedY = Mathf.Clamp(rb.position.y, minBounds.y + padding, maxBounds.y - padding);
87
88      rb.position = new Vector2(clampedX, clampedY);
89  }
```

Ilustración 35. Captura código del método ClampPosition().

Uso la función Mathf.Clamp() para restringir los valores de posición dentro de un rango determinado, en este caso los límites calculados previamente a partir de la cámara principal. De esta forma, el jugador permanece siempre dentro del área visible del escenario.

El movimiento del jugador se basa en la interacción entre el sistema de entrada del usuario, el componente físico Rigidbody2D y los métodos del ciclo de vida proporcionados por Unity (Update() y FixedUpdate()). Esto es una de las características distintivas del motor Unity y permite desarrollar mecánicas de juego de forma estructurada y fácilmente ampliable.

Una vez implementado el movimiento del jugador, el siguiente paso en el desarrollo del proyecto fue diseñar el sistema de disparo. Para ello, creé un prefab llamado Laser, que representa el proyectil reutilizable que será disparado por el jugador. El uso de prefabs permite reutilizar un mismo objeto con una configuración predefinida en múltiples instancias dentro del juego, facilitando la gestión de recursos y la consistencia del comportamiento de los proyectiles.

Para estructurar correctamente la lógica de disparo utilicé una jerarquía de objetos dentro del jugador. En primer lugar, el objeto principal del jugador contiene un GameObject hijo denominado Forward, el cual se encarga de gestionar la lógica del arma. Este objeto incluye el script Weapon, que controla la creación de proyectiles y la frecuencia de disparo. La separación de esta lógica en un objeto independiente permite mantener el código más modular.

Dentro de este objeto Forward añadí otro objeto hijo denominado FirePoint, cuya función consiste en definir el punto exacto desde el cual se generan los proyectiles. Este objeto actúa como referencia espacial para determinar la posición inicial y la orientación del disparo. De esta forma, el proyectil siempre se genera en la dirección hacia la que está apuntando el jugador.

La lógica principal del disparo se encuentra implementada en el script Weapon, que se encarga de instanciar el prefab del láser y aplicarle una fuerza inicial para que se desplace en la dirección correcta.

```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
    Unity Script (2 asset references) | 4 references
5   public class Weapon : MonoBehaviour
6   {
7       [Header("Laser")]
8       public GameObject laserPrefab;
9       public Transform firePoint;
10      public float fireForce = 20f;
11
12      [Header("Fire Rate")]
13      public float baseFireCooldown = 0.4f;
14
15      private float currentFireCooldown;
16      private float fireTimer = 0f;
```

Ilustración 36. Captura código de variables de disparo en script Weapon.

En este fragmento defino las variables necesarias para gestionar el disparo. La variable laserPrefab contiene la referencia al prefab del proyectil, mientras que firePoint indica el punto desde el cual se generará el disparo. Además, aquí establezco un sistema de control de cadencia de disparo mediante un temporizador (fireTimer) que limita la frecuencia con la que el arma puede disparar.

El método encargado de generar los proyectiles es Fire(), que se ejecuta cuando el jugador realiza la acción de disparo. Este método crea una instancia del prefab del láser y le aplica una fuerza inicial utilizando el componente Rigidbody2D.

```
2 references
public void Fire()
{
    if (fireTimer > 0f)
        return;

    GameObject laser = Instantiate(
        laserPrefab,
        firePoint.position,
        firePoint.rotation
    );

    laser.GetComponent<Rigidbody2D>()
        .AddForce(firePoint.up * fireForce, ForceMode2D.Impulse);

    fireTimer = currentFireCooldown;

    if (audioSource != null && laserSound != null)
    {
        audioSource.PlayOneShot(laserSound);
    }
}
```

Ilustración 37. Captura código de función Fire().

En este caso utilicé la función Instantiate() proporcionada por Unity para crear una nueva instancia del prefab del láser en la posición y orientación del objeto FirePoint. Posteriormente se obtiene el componente Rigidbody2D del proyectil y se aplica una fuerza en la dirección up del punto de disparo, lo que provoca que el proyectil se desplace hacia delante. Debido a la baja complejidad del juego, descarté el uso de Object Pooling para instanciar los objetos como láseres, enemigos o power-ups, ya que complicaría demasiado la lógica y no es necesario para un juego tan simple.

El prefab Laser incluye un script genérico denominado Laser, que se utiliza tanto para los proyectiles del jugador como para los de los enemigos. Este script se encarga de detectar colisiones con otros objetos del entorno y aplicar el daño correspondiente.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Unity Script (2 asset references) | 0 references
public class Laser : MonoBehaviour
{
    public int damage = 1;

    Unity Message | 0 references
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            PlayerController player = collision.GetComponent<PlayerController>();

            if (player != null)
            {
                player.TakeDamage(damage);
            }

            Destroy(gameObject);
            return;
        }

        EnemyHealth enemy = collision.GetComponent<EnemyHealth>();

        if (enemy != null)
        {
            enemy.TakeDamage(damage);
            Destroy(gameObject);
            return;
        }

        Destroy(gameObject);
    }
```

Ilustración 38. Captura código de script Laser.cs.

En este fragmento utilicé el método OnTriggerEnter2D(), que es ejecutado automáticamente por el motor cuando el proyectil entra en contacto con otro objeto que tenga un collider

configurado como trigger. El script comprueba si el objeto impactado corresponde al jugador o a un enemigo, aplicando el daño correspondiente y destruyendo posteriormente el proyectil.

Adicionalmente, implementé un sistema para eliminar automáticamente los proyectiles cuando abandonan el área visible de la cámara. Esto se realiza mediante el método OnBecameInvisible(), que es ejecutado por Unity cuando el objeto deja de ser visible para la cámara.

```
35
                Unity Message | 0 references
36          private void OnBecameInvisible()
37          {
38              Destroy(gameObject);
39          }
40      }
```

Ilustración 39. Captura código de función OnBecameInvisible().

Este mecanismo es especialmente importante desde el punto de vista del rendimiento, ya que evita que los proyectiles permanezcan indefinidamente en la escena una vez han abandonado el área visible del juego. De esta forma se reduce la acumulación innecesaria de objetos activos, optimizando el uso de recursos del sistema.

Una vez implementado el sistema de disparo del jugador, procedí a crear un objeto enemigo básico que inicialmente funcionaba como un objetivo estacionario. Este objeto me permitió comprobar el correcto funcionamiento de las colisiones entre proyectiles y enemigos, verificando que los disparos se destruían correctamente al impactar y que el sistema de daño funcionaba según lo previsto.

El sistema de disparo que he implementado se basa en la interacción entre prefabs reutilizables, componentes físicos y scripts de comportamiento.

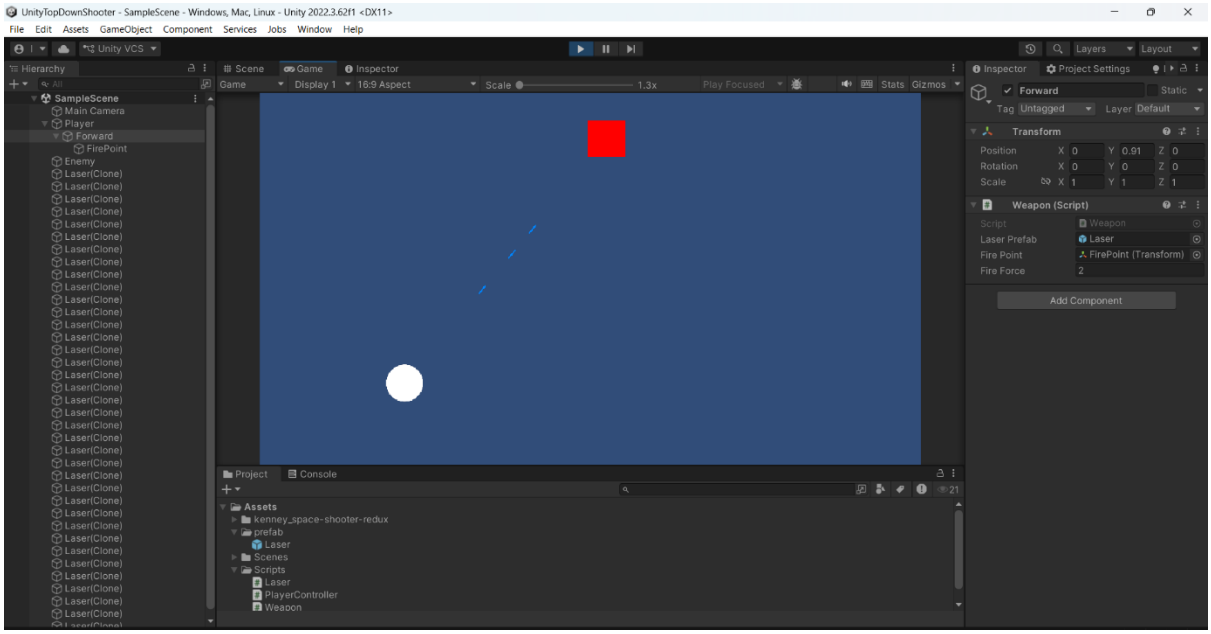


Ilustración 40. Captura de la lógica del player en Unity.

3.1.3 Lógica del enemigo

El siguiente paso en el desarrollo del proyecto fue diseñar el comportamiento de los enemigos presentes en la escena. Para ello creé un objeto enemigo que comparte una estructura jerárquica similar a la del jugador, lo que permite reutilizar gran parte de los sistemas previamente desarrollados.

En primer lugar, el objeto principal del enemigo contiene un GameObject hijo denominado Forward, cuya función es gestionar la lógica de disparo. Este objeto incluye el mismo script Weapon utilizado por el jugador, lo que permite reutilizar el sistema de disparo sin necesidad de implementar una lógica completamente nueva. Dentro de este objeto se encuentra otro GameObject hijo denominado FirePoint, que indica el punto exacto desde el cual se genera el proyectil. Esta estructura jerárquica permite separar claramente la lógica de comportamiento del enemigo de la lógica del arma, facilitando la modularidad del sistema.

Aunque el enemigo utiliza el mismo sistema de disparo que el jugador, se emplea un prefab de proyectil distinto, denominado LaserEnemy, que posee un sprite diferente para diferenciar visualmente los disparos enemigos de los del jugador. Sin embargo, ambos proyectiles utilizan el mismo script Laser, por lo que existe cierto elemento de reusabilidad.

Para gestionar el comportamiento del enemigo creé el script EnemyController, encargado de controlar tanto el movimiento como la rotación y el disparo del enemigo. Empecé definiendo las referencias necesarias para acceder a los componentes que intervienen en el comportamiento del enemigo.

```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
    Unity Script (1 asset reference) | 2 references
5   public class EnemyController : MonoBehaviour
6   {
7       [Header("References")]
8       public Rigidbody2D rb;
9       public Weapon weapon;
10      public Transform player;
```

Ilustración 41. Captura de variables de referencia de EnemyController.cs.

Estas variables permiten acceder al componente físico Rigidbody2D, al sistema de disparo Weapon y a la posición del jugador dentro de la escena. La referencia al jugador es especialmente importante, ya que el comportamiento del enemigo se basa en reaccionar dinámicamente a la posición de este.

Por otro lado, tenemos los parámetros que permiten definir la velocidad de desplazamiento del enemigo y la forma en la que este se aproxima al jugador. En lugar de dirigirse directamente hacia él, el enemigo intenta mantenerse a una distancia determinada. Si el jugador se encuentra demasiado lejos, el enemigo se aproxima, mientras que si el jugador se acerca demasiado, el enemigo retrocede para recuperar una distancia segura.

```
11
12      [Header("Movement")]
13      public float moveSpeed = 3f;
14      public float acceleration = 6f;
15
16      [Header("Distance Control")]
17      public float preferredDistance = 4f;
18      public float retreatDistance = 2.5f;
```

Ilustración 42. Captura de variables de movimiento de EnemyController.cs.

Además del desplazamiento directo, implementé un sistema de movimiento lateral denominado strafing, que permite al enemigo desplazarse de izquierda a derecha mientras se enfrenta al jugador. Este comportamiento genera una sensación de mayor dinamismo durante el combate y aumenta la dificultad para impactar a los enemigos.

```
20      [Header("Strafing")]
21      public float strafeStrength = 1.2f;
22      public float strafeChangeInterval = 2f;
23      private float strafeTimer;
24      private float strafeDirection;
```

Ilustración 43. Captura de variables de strafing de EnemyController.cs.

Este sistema alterna periódicamente la dirección del movimiento lateral, generando un comportamiento menos predecible.

La lógica principal que calcula el movimiento del enemigo se encuentra dentro del método Update(), donde se calcula la dirección hacia el jugador y la distancia existente entre ambos.

```
Unity Message | 0 references
45  void Update()
46  {
47      if (player == null)
48          return;
49
50      Vector2 enemyPosition = rb.position;
51      Vector2 playerPosition = player.position;
52
53      Vector2 toPlayer = playerPosition - enemyPosition;
54      float distance = toPlayer.magnitude;
55      Vector2 dirToPlayer = toPlayer.normalized;
```

Ilustración 44. Captura de variables de strafing de EnemyController.cs.

A partir de esta información se determina si el enemigo debe acercarse o alejarse del jugador. Luego combiné este desplazamiento con el movimiento lateral previamente descrito.

```
57      strafeTimer -= Time.deltaTime;
58      if (strafeTimer <= 0f)
59      {
60          strafeDirection *= -1f;
61          strafeTimer = strafeChangeInterval;
62      }
63
64      Vector2 strafe = new Vector2(-dirToPlayer.y, dirToPlayer.x) * strafeDirection;
65
66      Vector2 distanceMovement = Vector2.zero;
67
68      if (distance > preferredDistance)
69      {
70          distanceMovement = dirToPlayer;
71      }
72      else if (distance < retreatDistance)
73      {
74          distanceMovement = -dirToPlayer;
75      }
76
77      Vector2 finalDirection = distanceMovement + strafe * strafeStrength;
78      finalDirection.Normalize();
79
80      desiredVelocity = finalDirection * moveSpeed;
81
82      fireTimer -= Time.deltaTime;
83      if (fireTimer <= 0f)
84      {
85          weapon.Fire();
86          fireTimer = fireRate;
87      }
88  }
```

Ilustración 45. Captura de lógica de strafing en a función Update().

Este cálculo genera un vector de movimiento final que combina el desplazamiento hacia o desde el jugador con el movimiento lateral. El resultado es un comportamiento más complejo

y dinámico, en el que los enemigos se desplazan constantemente alrededor del jugador en lugar de seguir una trayectoria lineal.

Al igual que en el caso del jugador, el movimiento físico se aplica dentro del método FixedUpdate(), ya que este método está diseñado para trabajar con el sistema de físicas del motor Unity.

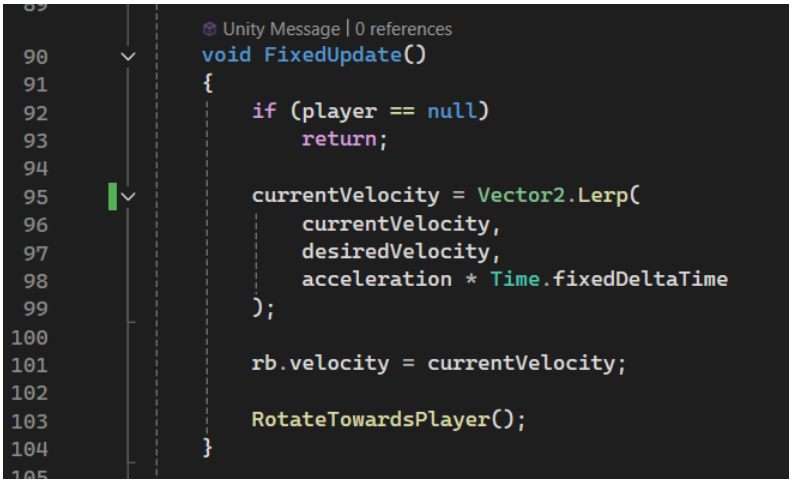


Ilustración 46. Captura de la función FixedUpdate().

En este fragmento utilicé la función Vector2.Lerp() para interpolar suavemente entre la velocidad actual y la velocidad deseada, lo que genera un movimiento más natural y evita cambios bruscos de dirección.

Además del movimiento, el enemigo también debe orientar su sprite hacia el jugador para poder disparar correctamente. Para ello calculé el ángulo entre ambos objetos utilizando funciones matemáticas proporcionadas por Unity.

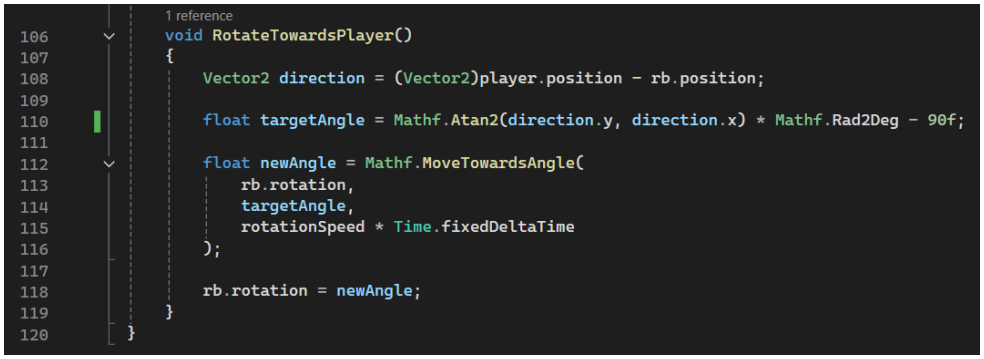


Ilustración 47. Captura de la función RotateTowardsPlayer().

Este cálculo permite que el enemigo rote progresivamente hacia la posición del jugador, lo que refuerza la sensación de que está apuntando activamente hacia él.

Finalmente, el enemigo también puede disparar proyectiles utilizando el mismo sistema de armas que el jugador. La frecuencia de disparo se controla mediante un temporizador que se reduce progresivamente en cada actualización.

```
81
82          fireTimer -= Time.deltaTime;
83          if (fireTimer <= 0f)
84          {
85              weapon.Fire();
86              fireTimer = fireRate;
87          }
```

Ilustración 48. Captura de la lógica de disparo en EnemyController.cs.

Este mecanismo permite que el enemigo dispare automáticamente a intervalos regulares, reutilizando el sistema de disparo implementado previamente.

Aparte del comportamiento del enemigo, implementé un sistema de gestión de vida mediante el script EnemyHealth, encargado de controlar la cantidad de daño que puede recibir el enemigo antes de ser destruido.

```
1 reference
23      public void TakeDamage(int amount)
24      {
25          currentHealth -= amount;
26
27          if (currentHealth <= 0)
28          {
29              Die();
30          }
31      }
```

Ilustración 49. Captura de la función TakeDamage().

Cuando la vida del enemigo llega a cero, se ejecuta el método Die(), que elimina el objeto de la escena y añade puntos a la puntuación del jugador.

```
1 reference
33      void Die()
34      {
35          if (ScoreManager.Instance != nul
36          {
37              ScoreManager.Instance.AddSco
38          }
39
40          TryDropPowerUp();
41
42          Destroy(gameObject);
43      }
```

Ilustración 50. Captura de la función Die().

Además, existe una probabilidad de que el enemigo genere un power-up al ser destruido, lo que introduce una mecánica adicional de recompensa dentro del juego.

El comportamiento del enemigo se basa en la combinación de un sistema de movimiento dinámico, un sistema de rotación orientado hacia el jugador y un sistema de disparo reutilizado a partir de la lógica implementada para el jugador.

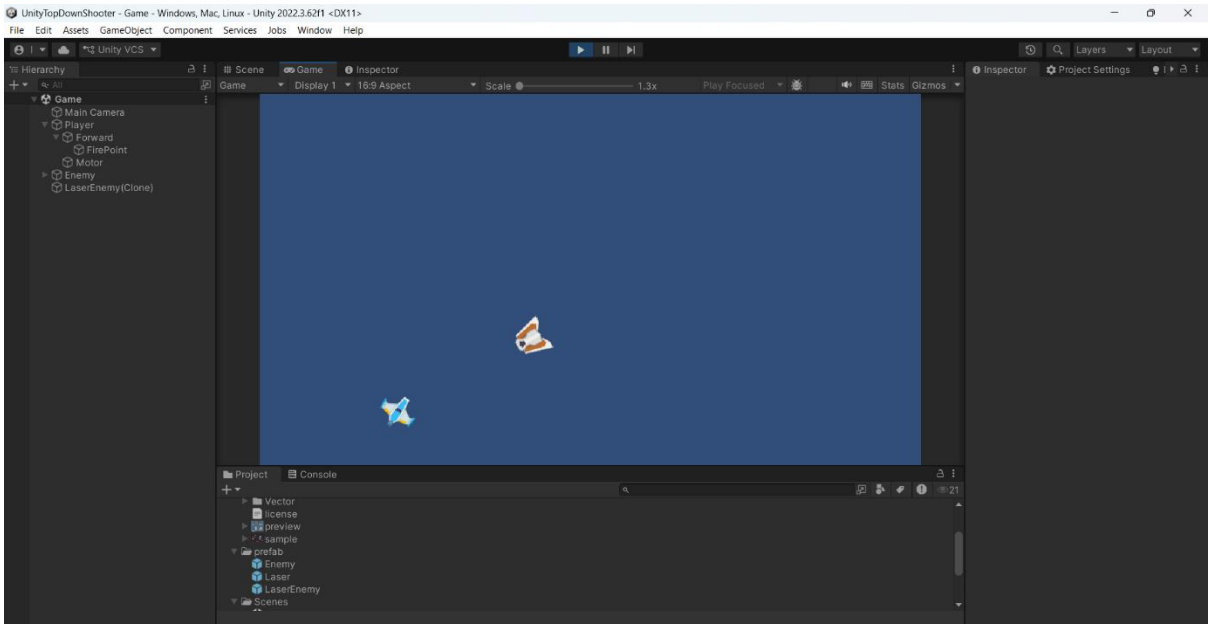


Ilustración 51. Captura de la lógica del enemigo en Unity.

3.1.4 Lógica del spawner y de la puntuación

Tras realizar la implementación de la lógica del enemigo, me puse con la implementación de un sistema que permitiera generar enemigos de forma dinámica durante la partida. Para ello convertí el objeto enemigo previamente creado en un prefab, lo que permite instanciar nuevas copias del mismo durante la ejecución del juego. Los prefabs permiten reutilizar configuraciones completas de objetos sin necesidad de reconstruirlas manualmente cada vez.

Después de crear el prefab del enemigo, añadí a la escena un GameObject vacío encargado de gestionar la generación de enemigos. A este objeto le asocié un script denominado EnemySpawner, cuya función consiste en instanciar enemigos en intervalos de tiempo determinados y en posiciones situadas fuera de la zona visible de la cámara.

En primer lugar, dentro del script definí las referencias necesarias para acceder al prefab del enemigo y al objeto del jugador dentro de la escena.

```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
        Unity Script (1 asset reference) | 0 references
5   public class EnemySpawner : MonoBehaviour
6   {
7       [Header("References")]
8       public GameObject enemyPrefab;
9       public Transform player;
```

Ilustración 52. Captura de referencias del prefab del enemigo.

Estas variables permiten que el spawner pueda crear nuevas instancias del enemigo y asignarles una referencia al jugador, lo cual es necesario para que el comportamiento del enemigo pueda orientarse correctamente hacia él.

Después, definí los parámetros que controlan la frecuencia de aparición de los enemigos y la progresión de la dificultad a lo largo de la partida.

```
11      [Header("Spawn Settings")]
12      public float spawnInterval = 2f;
13      public float minSpawnInterval = 0.5f;
14      public float difficultyIncreaseRate = 0.05f;
15      public float spawnDistance = 2f;
```

Ilustración 53. Captura de referencias de spawn.

El parámetro spawnInterval define el tiempo inicial entre apariciones de enemigos. A medida que la partida avanza, este intervalo se reduce progresivamente mediante el parámetro difficultyIncreaseRate, lo que provoca que los enemigos aparezcan con mayor frecuencia y aumente la dificultad del juego. Asimismo, se establece un valor mínimo (minSpawnInterval) para evitar que el intervalo llegue a ser demasiado pequeño.

La generación de enemigos la implementé mediante una corrutina, un mecanismo proporcionado por Unity que permite ejecutar procesos de manera asíncrona a lo largo del tiempo sin bloquear el flujo principal del programa.

```
1 reference
25  IEnumerator SpawnRoutine()
26  {
27      while (true)
28      {
29          SpawnEnemy();
30
31          spawnInterval = Mathf.Max(minSpawnInterval, spawnInterval - difficultyIncreaseRate);
32
33          yield return new WaitForSeconds(spawnInterval);
34      }
35  }
```

Ilustración 54. Captura de corrutina SpawnRoutine().

Esta corrutina ejecuta un bucle continuo en el que se genera un nuevo enemigo y posteriormente se espera un determinado intervalo de tiempo antes de repetir el proceso. Durante cada iteración se reduce el intervalo de aparición, lo que genera una progresión gradual de la dificultad.

El método encargado de crear el enemigo es SpawnEnemy(), que instancia una nueva copia del prefab en una posición determinada y asigna al enemigo una referencia al jugador.

```
1 reference
37  void SpawnEnemy()
38  {
39      Vector2 spawnPosition = GetSpawnPosition();
40
41      GameObject enemy = Instantiate(enemyPrefab, spawnPosition, Quaternion.identity);
42
43      EnemyController enemyController = enemy.GetComponent<EnemyController>();
44      enemyController.player = player;
45  }
```

Ilustración 55. Captura del método SpawnEnemy().

En este fragmento utilizo la función Instantiate() de Unity para crear una nueva instancia del prefab del enemigo. Luego, obtengo el componente EnemyController y establezco la referencia al jugador, permitiendo que el enemigo pueda seguirlo y atacarlo correctamente.

Para evitar que los enemigos aparezcan directamente dentro de la pantalla, implementé un método que calcula posiciones de generación situadas fuera de los límites visibles de la cámara.

```
1 reference
Vector2 GetSpawnPosition()
{
    Vector2 min = cam.ViewportToWorldPoint(new Vector2(0, 0));
    Vector2 max = cam.ViewportToWorldPoint(new Vector2(1, 1));

    int side = Random.Range(0, 4);

    switch (side)
    {
        case 0:
            return new Vector2(
                Random.Range(min.x, max.x),
                max.y + spawnDistance
            );

        case 1:
            return new Vector2(
                Random.Range(min.x, max.x),
                min.y - spawnDistance
            );

        case 2:
            return new Vector2(
                min.x - spawnDistance,
                Random.Range(min.y, max.y)
            );

        default:
            return new Vector2(
                max.x + spawnDistance,
                Random.Range(min.y, max.y)
            );
    }
}
```

Ilustración 56. Captura del método GetSpawnPosition().

Este método calcula las coordenadas de los límites de la cámara y selecciona aleatoriamente uno de los cuatro lados de la pantalla para generar el enemigo. De esta manera, los enemigos aparecen fuera del área visible y entran progresivamente en la escena, lo que mejora la experiencia visual del jugador.

Durante la implementación del sistema de combate también surgió un problema relacionado con las colisiones. Inicialmente utilicé el mismo prefab de proyectil tanto para el jugador como para los enemigos, lo que provocaba que los enemigos pudieran dañarse entre sí al colisionar con los disparos enemigos. Para solucionar este problema creé un prefab independiente llamado EnemyLaser, con un sprite diferente, y utilicé tags para distinguir entre jugador y enemigos. De esta forma, el sistema de colisiones puede identificar correctamente a qué tipo de objeto pertenece cada proyectil.

Aparte del sistema de generación de enemigos, implementé un sistema de puntuación que permite mostrar en pantalla los puntos obtenidos por el jugador al destruir enemigos. Para ello creé un script denominado ScoreManager, encargado de almacenar la puntuación actual y actualizar la interfaz de usuario.

El sistema utiliza un patrón Singleton, lo que permite acceder al gestor de puntuación desde cualquier parte del código del proyecto.

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

Unity Script (1 asset reference) | 5 references
public class ScoreManager : MonoBehaviour
{
    public static ScoreManager Instance;

    [Header("UI")]
    public TMP_Text scoreText;

    private int score = 0;

    public static int lastScore = 0;

    Unity Message | 0 references
    void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
        }
        else
        {
            Destroy(gameObject);
        }
    }
```

Ilustración 57. Captura del método Awake().

Este patrón garantiza que únicamente exista una instancia activa del gestor de puntuación durante la ejecución del juego.

Cuando un enemigo es destruido, se añaden puntos a la puntuación del jugador mediante el método AddScore().

```
1 reference
public void AddScore(int amount)
{
    score += amount;
    UpdateUI();
}
```

Ilustración 58. Captura del método AddScore().

Este método incrementa el valor de la puntuación y actualiza el texto mostrado en la interfaz de usuario. De esta manera, el jugador puede visualizar en todo momento la cantidad de enemigos eliminados y los puntos acumulados durante la partida.

Habiendo hecho todo esto, ya tendría creada la aparición constante de enemigos y el sistema de puntuación.

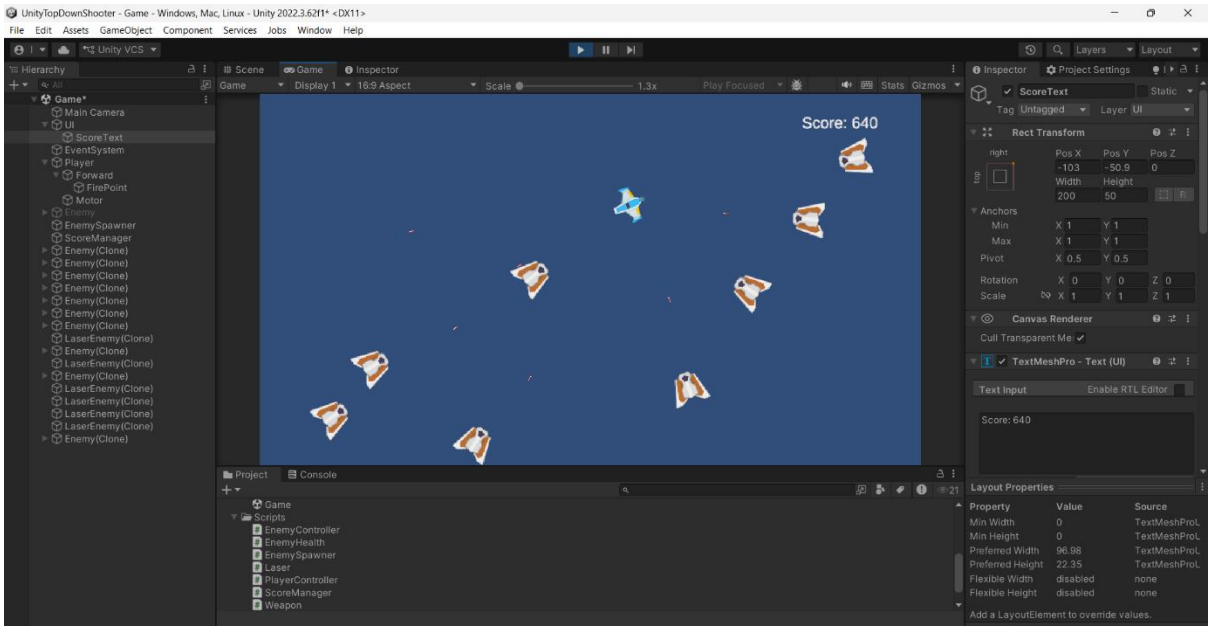


Ilustración 59. Captura de la lógica del spawner de enemigos y puntuación en Unity.

3.1.5 Lógica de las vidas del jugador, timer y menú de pausa

El siguiente paso en el desarrollo de la interfaz del juego fue ampliar los elementos de la UI con información adicional relevante para el jugador. Comencé implementando un sistema que muestra el tiempo transcurrido durante la partida, permitiendo al jugador conocer cuánto tiempo ha sobrevivido frente a los enemigos.

Para gestionar esta funcionalidad creé un script llamado TimerManager, encargado de contabilizar el tiempo desde el inicio de la partida y actualizar el texto correspondiente en la interfaz de usuario.

En primer lugar definí una referencia al elemento de texto que se encarga de mostrar el temporizador en pantalla.

```
1   using UnityEngine;
2   using TMPro;
3
    Unity Script (1 asset reference) | 3 references
4   public class TimerManager : MonoBehaviour
5   {
6       [Header("UI")]
7       public TMP_Text timerText;
8
9       private float elapsedTime;
10      private bool isRunning = true;
11
12      public static float lastTime = 0f;
13
```

Ilustración 60. Captura de la referencia del texto.

La variable elapsedTime se utiliza para almacenar el tiempo acumulado desde el inicio de la partida, mientras que isRunning permite controlar si el temporizador debe continuar actualizándose o no. Además, la variable estática lastTime se utiliza para guardar el tiempo final de la partida cuando el jugador pierde.

El cálculo del tiempo transcurrido es realizado dentro del método Update(), que se ejecuta

una vez por cada frame renderizado. Para ello utilicé la variable Time.deltaTime, proporcionada por Unity, que representa el tiempo transcurrido entre dos frames consecutivos.

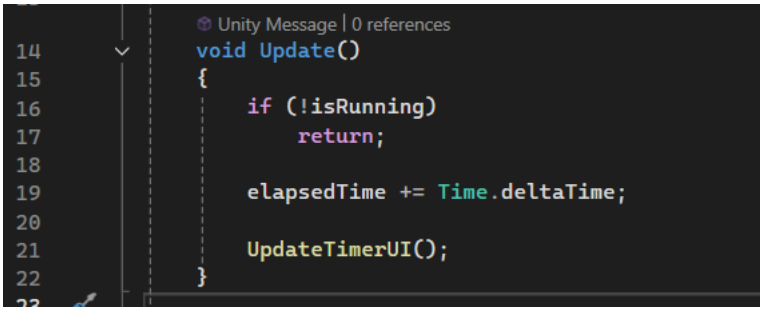


Ilustración 61. Captura del método Update().

Esta lógica permite calcular el tiempo de manera independiente del número de frames por segundo, garantizando que el temporizador funcione correctamente incluso si el rendimiento del juego varía.

Para mostrar el tiempo en un formato más comprensible para el jugador, implementé un método que convierte el tiempo acumulado en minutos y segundos antes de actualizar el texto de la interfaz.

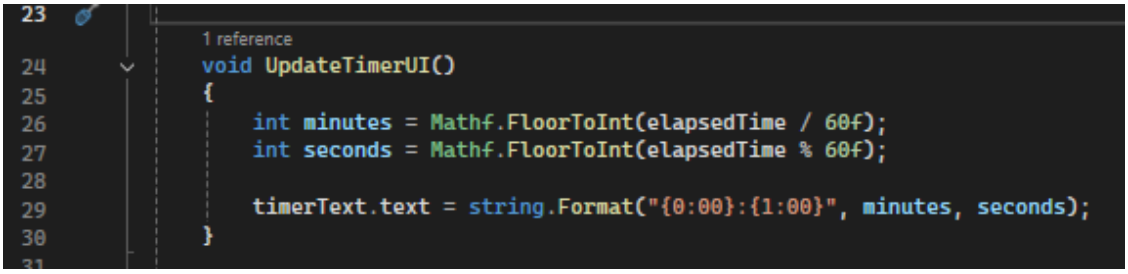


Ilustración 62. Captura del método UpdateTimerUI().

Este método divide el tiempo total en minutos y segundos utilizando funciones matemáticas proporcionadas por Unity y posteriormente actualiza el texto de la interfaz mediante el componente TextMeshPro.

Finalmente, cuando la partida termina, el temporizador guarda el valor final para poder mostrarlo posteriormente en la pantalla de Game Over.

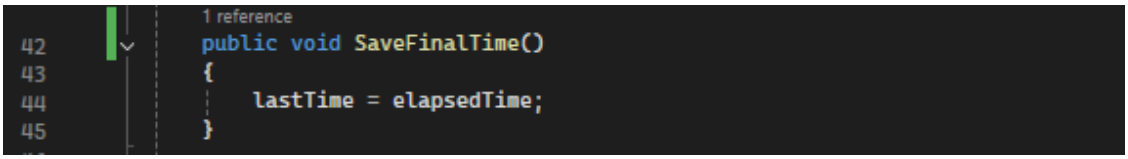


Ilustración 63. Captura del método SaveFinalTime().

De esta forma es posible recuperar el tiempo de supervivencia del jugador incluso después de cambiar de escena.

Luego, añadí la lógica del menú de pausa. Para esto, creé un GameObject vacío y le añadí un script con la lógica de la pausa del juego.

Empiezo por asignarle por arrastre la parte de la UI correspondiente con el menú de pausa y creando una variable para controlar la pausa.

```
[Header("Pause Panel (child del Canvas)")]
public GameObject pausePanel;

private bool isPaused = false;

Unity Message | 0 references
void Start()
{
    if (pausePanel != null)
        pausePanel.SetActive(false);
}
```

Ilustración 64. Captura de las variables de PauseManager.

Después, en la función de Update, gestiono si el jugador presiona la tecla “P” y el estado actual del juego. Si está pausado el juego, entonces se reanuda poniendo el Time.timescale a 1f, se oculta el menú de pausa y se modifica la variable del estado del juego. En caso de no estar pausado el juego, pasa lo opuesto: se muestra el menú de pausa, se establece el valor de Time.timescale a 0f y se modifica la variable del estado de juego.

```
Unity Message | 0 references
void Update()
{
    if (Input.GetKeyDown(KeyCode.P))
    {
        if (isPaused)
            Resume();
        else
            Pause();
    }
}

1 reference
public void Pause()
{
    pausePanel.SetActive(true);

    Time.timeScale = 0f;

    isPaused = true;
}

1 reference
public void Resume()
{
    pausePanel.SetActive(false);

    Time.timeScale = 1f;

    isPaused = false;
}
```

Ilustración 65. Captura de las funciones Update(), Pause() y Resume() de PauseManager.

Lo último de la lógica del menú de pausa es la función ExitToMainMenu, que se ejecuta cuando el usuario presiona el botón de Exit del menú de pausa. Esta función restablece el flujo del juego y, mediante el SceneManager, cambia a la escena del menú principal del juego.

```
0 references
public void ExitToMainMenu()
{
    Time.timeScale = 1f;

    SceneManager.LoadScene("Main menu");
}
```

Ilustración 66. Captura de la función ExitToMainMenu() de PauseManager.

Tras acabar la lógica del menú de pausa, me puse con añadir un sistema que permitiera al jugador recibir daño y perder vidas al ser alcanzado por los proyectiles enemigos. Esto lo implementé directamente dentro del script PlayerController, ya que está estrechamente relacionada con el comportamiento del jugador.

Para empezar, definí las variables necesarias para controlar la cantidad de vidas disponibles y el tiempo durante el cual el jugador será invulnerable tras recibir daño.

```
21
22      [Header("Health")]
23      public int maxLives = 5;
24      public float invulnerabilityDuration = 2f;
25      public float blinkInterval = 0.15f;
26
27      private int currentLives;
28      private bool isInvulnerable = false;
```

Ilustración 67. Captura del referencias para la vida del jugador en PlayerControler.cs.

La variable maxLives define el número máximo de vidas que tiene el jugador al comenzar la partida. Cuando el jugador recibe daño, este valor se reduce hasta llegar a cero. Asimismo, se establecen parámetros que controlan el tiempo de invulnerabilidad tras recibir daño y la velocidad del efecto visual de parpadeo.

El método encargado de gestionar la recepción de daño es TakeDamage(), que se ejecuta cuando el jugador es alcanzado por un proyectil enemigo.

```
        1 reference
91      public void TakeDamage(int damage)
92      {
93          if (isInvulnerable)
94              return;
95
96          currentLives -= damage;
97
98          UpdateLivesUI();
99
100         if (currentLives <= 0)
101         {
102             Die();
103         }
104         else
105         {
106             StartCoroutine(InvulnerabilityCoroutine());
107         }
108     }
```

Ilustración 68. Captura de la función TakeDamage().

Este método reduce el número de vidas disponibles y actualiza la interfaz de usuario. Además, se comprueba si el jugador se encuentra en estado de invulnerabilidad, evitando que pueda recibir daño repetidamente en un intervalo muy corto de tiempo.

Cuando el jugador recibe daño pero aún dispone de vidas restantes, se inicia una corrutina que activa temporalmente un periodo de invulnerabilidad acompañado de un efecto visual de parpadeo.

```
1 reference
IEnumerator InvulnerabilityCoroutine()
{
    isInvulnerable = true;

    float timer = 0f;

    while (timer < invulnerabilityDuration)
    {
        SetSpritesVisible(false);

        yield return new WaitForSeconds(blinkInterval);

        SetSpritesVisible(true);

        yield return new WaitForSeconds(blinkInterval);

        timer += blinkInterval * 2f;
    }

    SetSpritesVisible(true);

    isInvulnerable = false;
}
```

Ilustración 69. Captura del método InvulnerabilityCoroutine().

Durante este periodo, los sprites del jugador se activan y desactivan repetidamente, generando un efecto visual que indica al jugador que se encuentra temporalmente protegido contra el daño.

Para implementar este efecto, utilicé un método auxiliar que activa o desactiva todos los SpriteRenderer asociados al jugador y a sus objetos hijos.

```
3 references
void SetSpritesVisible(bool visible)
{
    foreach (SpriteRenderer sr in spriteRenderers)
    {
        sr.enabled = visible;
    }
}
```

Ilustración 70. Captura del método SetSpritesVisible().

Esto permite aplicar el efecto de parpadeo a todos los elementos visuales del jugador, incluyendo componentes adicionales como el motor de la nave.

Finalmente, para que el jugador pueda visualizar el número de vidas restantes, añadí un elemento de texto dentro de la interfaz de usuario que se actualiza cada vez que el jugador recibe daño.

```
2 references
void UpdateLivesUI()
{
    if (livesText != null)
    {
        livesText.text = "X " + currentLives;
    }
}
```

Ilustración 71. Captura del método UpdateLivesUI().

Además del texto, añadí un sprite junto al contador de vidas dentro de la interfaz para facilitar la identificación visual de este indicador, completando así la lógica del temporizador y vidas

del jugador.

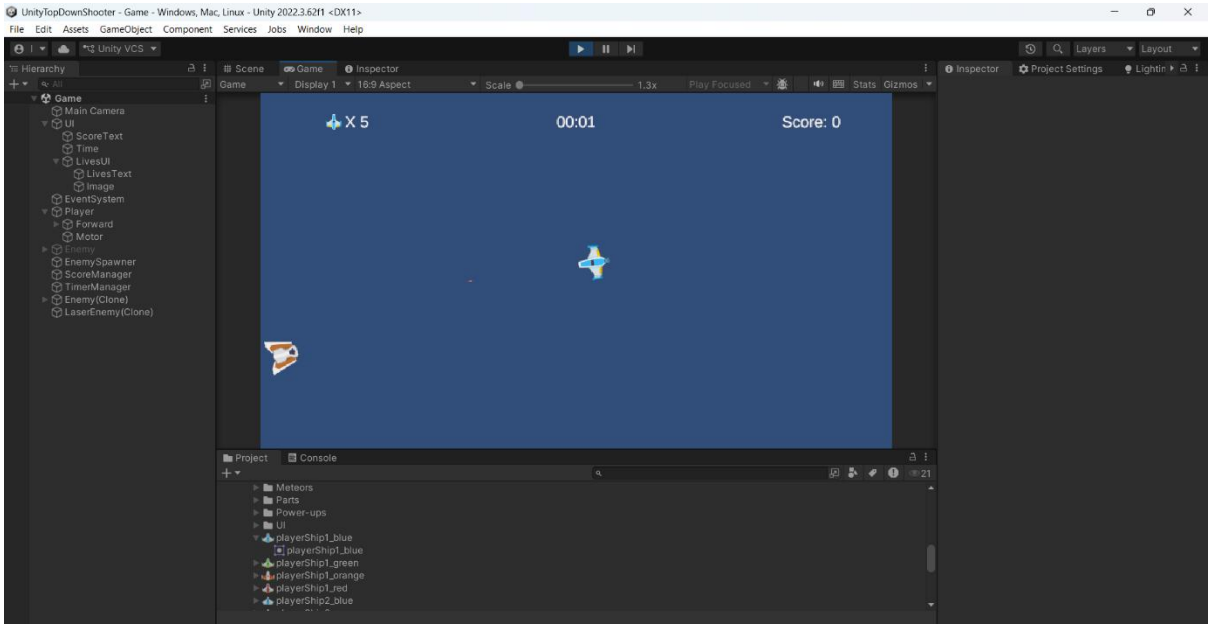


Ilustración 72. Captura de la lógica de las vidas del player, timer y menú de pausa en Unity.

3.1.6 Audio, fondo y power-ups

Habiendo completado el game loop principal, procedí a mejorar la experiencia del jugador mediante la incorporación de efectos de sonido, música de fondo y elementos visuales de escenario. Estos sistemas no afectan directamente a la jugabilidad, pero son fundamentales para reforzar la retroalimentación y la inmersión.

Comencé utilizando el sistema de audio de Unity basado en los componentes AudioSource y AudioClip. Este sistema permite asociar sonidos a objetos concretos dentro de la escena y reproducirlos en momentos específicos, como al disparar o al generar eventos. En el caso del disparo, integré la reproducción de sonido directamente en el script del arma, aprovechando que Unity permite reproducir clips de audio mediante el método PlayOneShot() sin necesidad de gestionar manualmente múltiples instancias de sonido.

```
17
18      [Header("Audio")]
19      public AudioSource audioSource;
20      public AudioClip laserSound;
```

Ilustración 73. Captura de referencias de audio.

Para la reproducción del sonido, uso AudioSource.PlayOneShot para reproducir el sonido deseado.

```
if (audioSource != null && laserSound != null)
{
    audioSource.PlayOneShot(laserSound);
}
```

Ilustración 74. Captura de lógica de reproducción de audio.

Aparte de los efectos de sonido, añadí música de fondo mediante un AudioSource configurado en bucle, lo que permite reproducir una pista continua durante toda la partida sin necesidad de intervención adicional en el código.

En cuanto al entorno visual, implementé un fondo utilizando un Sprite2D colocado detrás de todos los elementos de la escena. Este sprite actúa como imagen de fondo estática y se posiciona mediante el sistema de sorting layers de Unity para garantizar que siempre se renderice por debajo del resto de objetos.

Adicionalmente, utilicé un Tilemap para distribuir elementos del escenario, como asteroides, de forma estructurada. El sistema de tilemaps de Unity permite pintar directamente sobre una cuadrícula utilizando sprites previamente definidos, lo que facilita la creación de escenarios repetitivos o modulares sin necesidad de posicionar manualmente cada elemento.

Este sistema se basa en componentes como Grid y Tilemap, que trabajan conjuntamente para gestionar la disposición espacial de los tiles.

Unity permite crear fácilmente tilemaps a partir de diversas imágenes sueltas, por lo que lo empleo para el desarrollo en este motor; al no ser este el caso tan fácilmente para los otros motores, simplemente añadiré varias instancias de meteoritos para hacer el fondo.

Para acabar con este apartado, añadí una mecánica adicional mediante la implementación de power-ups, que pueden ser generados por los enemigos al morir con una determinada probabilidad. En este caso, el power-up implementado aumenta temporalmente la cadencia de disparo del jugador, lo que introduce una capa adicional de estrategia y recompensa dentro del juego.

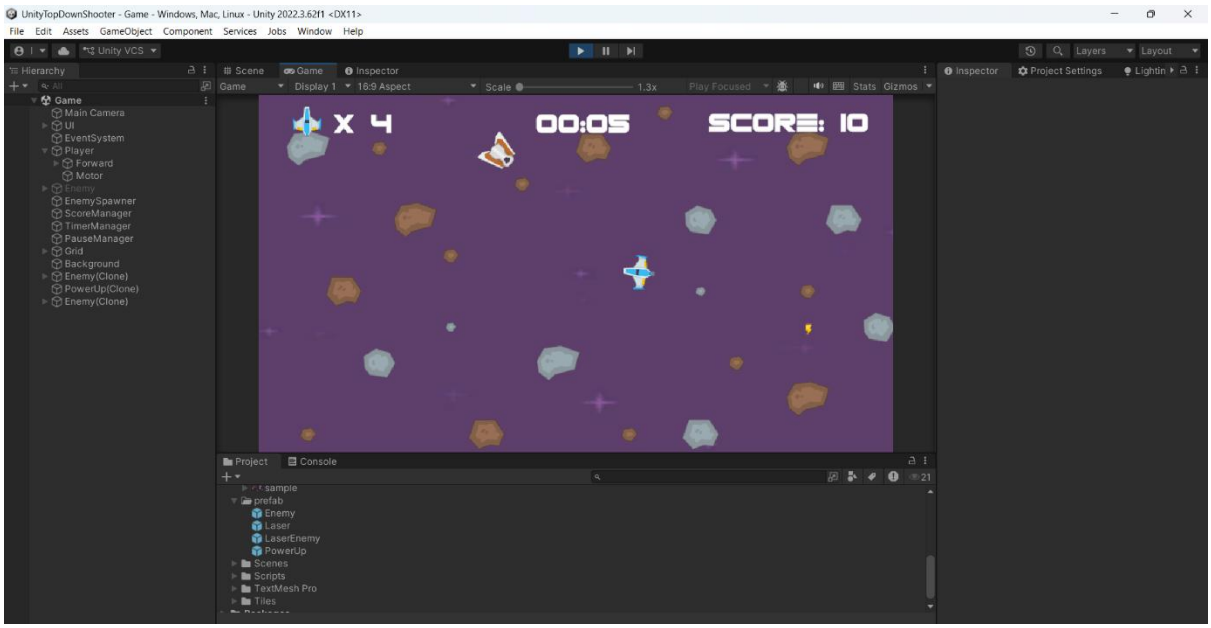


Ilustración 75. Captura de la escena game acabada en Unity.

3.1.7 Escenas game over y main menu

Tras haber finalizado la lógica principal del juego y completado la escena Game, procedí a desarrollar las escenas adicionales necesarias para estructurar el flujo completo de la aplicación: el menú principal y la pantalla de Game Over. Para ambas escenas reutilicé el

mismo fondo visual que en la escena principal, pero diferencié la experiencia mediante el uso de una música distinta, reforzando así la identidad de cada estado del juego.

En Unity, la gestión de escenas se realiza mediante el sistema de Scene Management, que permite cargar y cambiar entre distintas escenas utilizando la clase SceneManager. Este sistema facilita la separación de la lógica del juego en diferentes estados, mejorando la organización del proyecto.

En la escena de Game Over, implementé un script llamado GameOverManager, encargado de mostrar información relevante sobre la partida finalizada, concretamente la puntuación obtenida y el tiempo sobrevivido. Estos valores se recuperan a través de variables estáticas previamente almacenadas en los sistemas de puntuación y temporizador.

```
1 reference
void DisplayFinalScore()
{
    scoreText.text = "Score: " + ScoreManager.lastScore;
}

1 reference
void DisplayFinalTime()
{
    float time = TimerManager.lastTime;

    int minutes = Mathf.FloorToInt(time / 60f);
    int seconds = Mathf.FloorToInt(time % 60f);

    timeText.text = string.Format("Time: {0:00}:{1:00}", minutes, seconds);
}
```

Ilustración 76. Captura de las funciones DisplayFinalScore() y DisplayFinalTime().

Este tipo de implementación permite mantener los datos entre escenas sin necesidad de persistir objetos complejos, aprovechando variables estáticas accesibles globalmente.

Además, implementé botones en la interfaz que permiten al jugador reiniciar la partida o volver al menú principal. Estas acciones se gestionan mediante la carga de escenas específicas.

```
0 references
public void RestartGame()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene("Game");
}

0 references
public void GoToMainMenu()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene("Main menu");
}
}
```

Ilustración 77. Captura de las funciones RestartGame() y GoToMainMenu().

Tras probar las escenas, descubrí que es importante restablecer el valor de Time.timeScale antes de cambiar de escena, especialmente si el juego ha sido pausado previamente, ya que este valor afecta globalmente al tiempo del juego.

Sobre el menú principal, implementé una interfaz sencilla basada en botones que permiten iniciar la partida o salir de la aplicación. Unity facilita la creación de interfaces mediante su sistema de UI basado en Canvas, donde los elementos como textos y botones pueden configurarse visualmente y vincularse a funciones mediante el Inspector, sin necesidad de código adicional complejo.

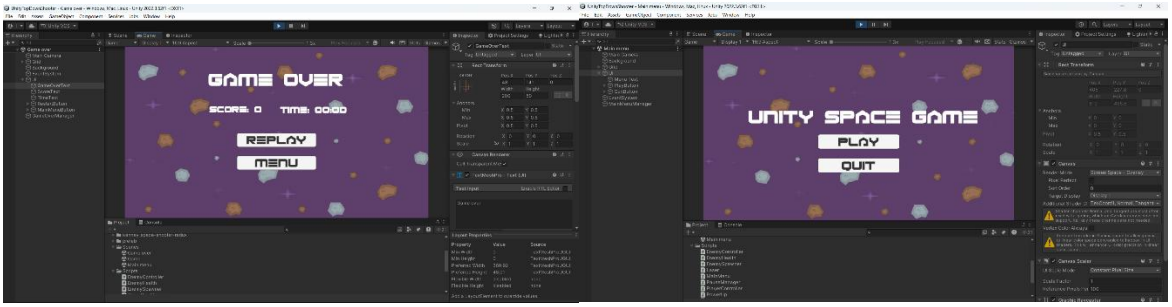


Ilustración 78. Capturas de las escenas game over y main menu acabadas en Unity.

3.1.8 Build del juego

Habiendo acabado la lógica del juego, el siguiente paso fue generar una build del proyecto utilizando el sistema de exportación de Unity. Este proceso se realiza a través del Build Settings, donde se selecciona la plataforma de destino, en este caso Windows, y las escenas que formarán parte del ejecutable final. Unity compila automáticamente todos los assets, scripts y dependencias en un único paquete ejecutable o distribuible, optimizando los recursos según la plataforma elegida. De esta manera, permite generar versiones del juego para múltiples plataformas con relativa facilidad, aunque el proceso puede resultar más pesado en comparación con otros motores debido al tamaño de las builds y al tiempo de compilación.

El sistema de exportación de Unity es un proceso robusto y exigente en recursos; ofrece una mayor capacidad de configuración y compatibilidad multiplataforma, lo que lo convierte en una herramienta más potente para proyectos de mayor escala.

3.1.9 Sobre los shaders

Aunque en este proyecto no he hecho uso de shaders, considero relevante mencionar su funcionamiento dentro del motor Unity debido a su importancia en el desarrollo de videojuegos modernos. Los shaders son pequeños programas que se ejecutan en la GPU y que permiten controlar cómo se renderizan los objetos en pantalla, afectando aspectos como el color, la iluminación, las transparencias o los efectos visuales avanzados. En Unity, los shaders pueden implementarse mediante código (HLSL) o utilizando herramientas visuales como Shader Graph, que facilita su creación sin necesidad de programación directa. Este sistema ofrece un alto grado de personalización, permitiendo desarrollar efectos complejos como distorsiones, iluminación dinámica o materiales personalizados.

Unity destaca por ofrecer un equilibrio entre potencia y accesibilidad, especialmente gracias a herramientas como Shader Graph, aunque en este proyecto no fue necesario su uso debido a la simplicidad visual del juego desarrollado.

3.2 Desarrollo en Godot

3.2.1 Diagrama de clases

Al igual que con el desarrollo en Unity, he hecho un diagrama de clases para el juego en Godot. En el diagráma de clases de Godot la estructura se organiza mediante un sistema de nodos jerárquicos dentro de escenas, donde cada nodo puede contener lógica propia mediante scripts.

El sistema está compuesto por un conjunto de clases que representan tanto entidades del juego como sistemas de gestión global. Esta separación permite mantener un diseño modular, en el que cada script cumple una función específica dentro de la escena o como parte de un sistema global.

PlayerController controla el comportamiento del jugador dentro de la escena. Gestiona el movimiento en el espacio 2D, la rotación hacia la posición del cursor y la acción de disparo mediante un nodo o recurso asociado que representa el arma. Además, se encarga de la lógica de vida del jugador, incluyendo la recepción de daño, la activación de un periodo de invulnerabilidad tras impactos y la verificación de la condición de muerte. Cuando las vidas llegan a cero, se desencadena la transición hacia la escena de fin de partida.

La clase EnemyController, define el comportamiento de los enemigos. Su lógica incluye el movimiento hacia el jugador, manteniendo cierta distancia, así como variaciones en el desplazamiento para generar un comportamiento más dinámico. Adicionalmente, se ocupa de orientar al enemigo hacia el jugador y coordinar el sistema de disparo mediante su arma asociada. La comunicación con el jugador se realiza a través de referencias directas a nodos dentro de la escena.

Por su parte, EnemyHealth actúa como un componente encargado de gestionar la vida de los enemigos. Este script, recibe el daño de los proyectiles y evalúa si la vida ha llegado a cero. En caso de ocurrir, el nodo enemigo se elimina de la escena y se interactúa con el sistema de puntuación para incrementar la puntuación del jugador.

La clase Weapon la implemento como un nodo que encapsula la lógica de disparo. Su función principal es instanciar proyectiles a partir de una escena (PackedScene) y aplicarles velocidad en la dirección correspondiente. Este sistema permite modificar el parámetro de la cadencia de disparo.

Los proyectiles, representados por la clase Laser, se implementan mediante nodos RigidBody2D. Su comportamiento se centra en detectar colisiones con otros nodos del entorno mediante señales y, en función del tipo de objeto impactado, aplicar daño y autodestruirse. También gestionan su propia vida útil cuando salen de los límites de la pantalla, garantizando así la limpieza de instancias en escena.

La clase EnemySpawner se encarga de la generación dinámica de enemigos. En Godot, esta funcionalidad es implementada habitualmente mediante la instancia de escenas predefinidas (PackedScene) en posiciones determinadas fuera de la vista del jugador. El sistema puede

utilizar temporizadores o lógica en _process para controlar la frecuencia de aparición, permitiendo ajustar la dificultad de forma progresiva durante la partida.

La clase PowerUp, implementada como un nodo Area2D, representa los objetos de mejora que el jugador puede recoger. Cuando el jugador entra en su área de colisión, se aplica un efecto temporal que modifica parámetros del juego, como la velocidad de disparo. Luego, el nodo se elimina de la escena, manteniendo así la coherencia del sistema.

Los sistemas de gestión global, como ScoreManager y TimerManager, los implemento en Godot mediante nodos autoload o singletons. Esto permite acceder a una única instancia desde cualquier script del proyecto. El ScoreManager se encarga de actualizar la puntuación cuando el jugador elimina enemigos y de reflejarla en la interfaz de usuario, mientras que el TimerManager controla el tiempo transcurrido durante la partida, actualizando un contador que se muestra en pantalla y almacenando el tiempo final al terminar la partida.

PauseMenu es una clase que está asociada a un nodo Control, esta gestiona el sistema de pausa. Su funcionamiento se basa en la propiedad de pausa del árbol de escenas (get_tree().paused), lo que detiene la ejecución de la mayoría de los nodos del juego. Adicionalmente, controla la visibilidad del panel de pausa y permite reanudar la partida o cambiar de escena, manteniendo un buen flujo entro las escenas y los estados del juego.

Por otro lado, GameOverManager se encarga de la escena de fin de partida, mostrando al jugador la puntuación final y el tiempo transcurrido. Esta clase accede a los datos almacenados en los autoloads o singletons para mostrar la información persistente y proporciona opciones para reiniciar la partida o volver al menú principal.

Finalmente, la clase MainMenu, también basada en un nodo Control, actúa como punto de entrada del juego. Su función principal es gestionar la transición hacia la escena de juego y permitir al usuario cerrar la aplicación, actuando como interfaz inicial del proyecto.

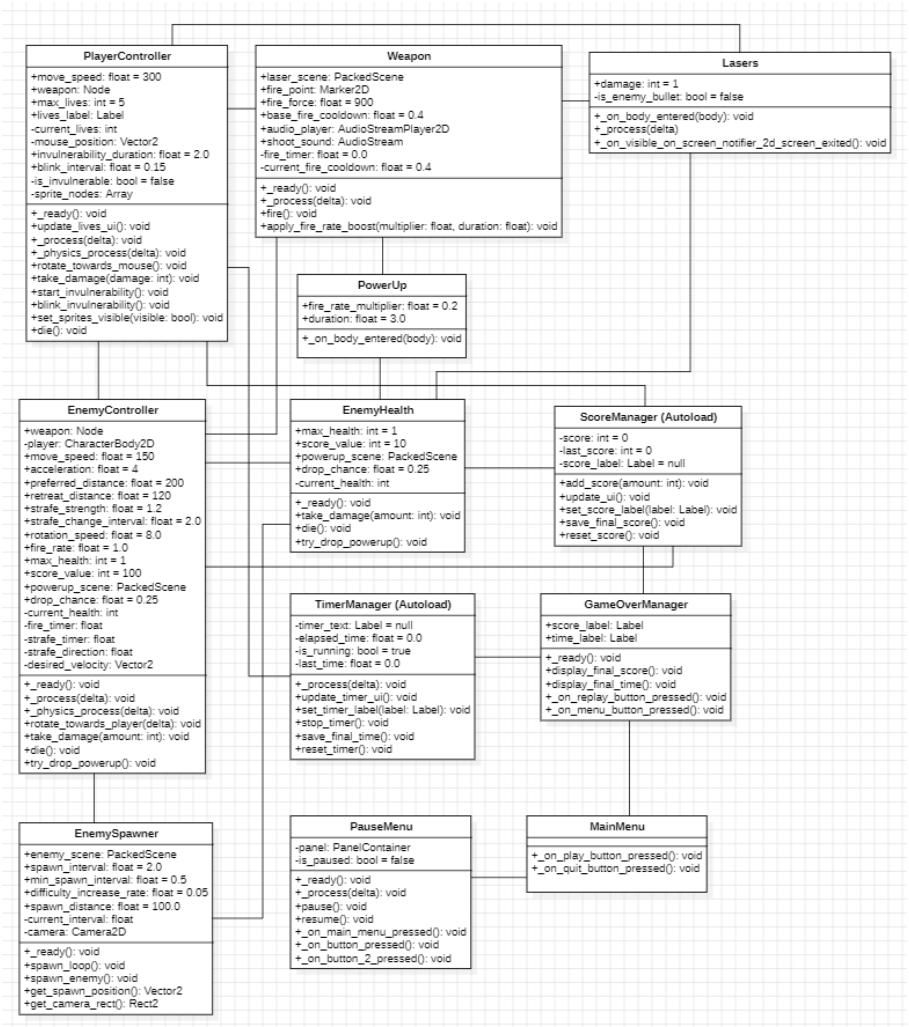


Ilustración 79. Diagrama de clases del proyecto en Godot.

3.2.2 Lógica del jugador

Empezando el desarrollo en Godot, creé una escena en dos dimensiones en blanco y comencé con la lógica del jugador. La lógica del movimiento del jugador en Godot, al igual que el resto de la lógica en este motor, la he implementado siguiendo un enfoque basado en nodos y scripts. En este caso, el jugador lo modelo como un nodo CharacterBody2D, el cual proporciona un sistema de movimiento propio gestionado directamente por código. Esta característica permite un control más preciso del desplazamiento del personaje.

La lógica principal del movimiento se basa en la captura de entradas mediante el sistema de acciones (InputMap) y la posterior construcción de un vector de dirección normalizado. Este vector lo utilizo para actualizar la propiedad velocity del CharacterBody2D, siendo finalmente aplicado mediante el método move_and_slide(), que gestiona automáticamente las colisiones. Este comportamiento lo implemento en el método _physics_process.

```gdscript
func _physics_process(delta):

    var input_dir = Vector2(
        Input.get_action_strength("move_right") - Input.get_action_strength("move_left"),
        Input.get_action_strength("move_down") - Input.get_action_strength("move_up")
    ).normalized()

    velocity = input_dir * move_speed
    move_and_slide()

    rotate_towards_mouse()
```

Ilustración 80. Captura de la función _physics_process.

En Godot, el control del movimiento lo realizo directamente sobre la velocidad del cuerpo, lo que evita efectos secundarios no deseados y facilita la implementación de mecánicas arcade como las de un shooter top-down.

La rotación del jugador la calculo en función de la posición del cursor en el espacio global, obtenida mediante get_global_mouse_position(). A partir de esta posición, se calcula un vector dirección cuyo ángulo determina la orientación del personaje; esta lógica se ve implementada en la función rotate_towards_mouse.

```gdscript
func rotate_towards_mouse():

    var direction = mouse_position - global_position
    rotation = direction.angle() + deg_to_rad(90)
```

Ilustración 81. Captura de la función rotate_towards_mouse.

El ajuste de 90 grados es necesario debido a la orientación del sprite, lo cual es una consideración habitual en el desarrollo en 2D. Esta forma de gestionar la rotación resulta especialmente directa en Godot, ya que el motor permite trabajar con ángulos en radianes de manera nativa.

En cuanto al sistema de disparo, he optado por una arquitectura modular en la que el nodo jugador contiene como hijo un nodo Weapon, encargado exclusivamente de la lógica de disparo. A su vez, este nodo contiene un Marker2D denominado fire_point, que define la posición y orientación desde la que se generan los proyectiles. Esta jerarquía permite desacoplar la lógica del jugador de la del arma, favoreciendo la reutilización del sistema tanto en el jugador como en los enemigos. La activación del disparo se realiza en el método _process.

```
29  func _process(delta):
30
31      if Input.is_action_just_pressed("shoot"):
32          weapon.fire()
33
34      mouse_position = get_global_mouse_position()
```

Ilustración 82. Captura del método _process.

El nodo Weapon implementa la lógica de instanciación del proyectil mediante el uso de PackedScene, equivalente conceptual a los prefabs. En Godot este sistema está completamente integrado en el flujo de escenas, lo que facilita su uso y organización. El método fire() se encarga de controlar la cadencia de disparo, reproducir efectos de sonido y generar el proyectil en la posición del fire_point, aplicándole una fuerza en la dirección adecuada.

```
19  func fire():
20
21      if fire_timer > 0:
22          return
23
24      if shoot_sound:
25          audio_player.stream = shoot_sound
26          audio_player.play()
27
28      var laser : RigidBody2D = laser_scene.instantiate()
29      get_tree().current_scene.add_child(laser)
30
31      laser.global_position = fire_point.global_position
32      laser.global_rotation = fire_point.global_rotation
33
34      laser.is_enemy_bullet = get_parent().is_in_group("enemy")
35
36      var direction = fire_point.global_transform.y
37      laser.apply_impulse(direction * fire_force)
38
39      fire_timer = current_fire_cooldown
```

Ilustración 83. Captura del método fire().

En este caso, resulta especialmente relevante el uso de global_transform.y, ya que en Godot este vector representa la dirección “hacia delante” del nodo, lo que simplifica el cálculo de la trayectoria del proyectil.

El proyectil Laser, lo he implementado como un RigidBody2D, aprovechando el sistema de físicas para simular su movimiento una vez instanciado. Su comportamiento se basa en la detección de colisiones mediante señales, aplicando daño únicamente a los objetivos válidos en función de si se trata de un disparo enemigo o del jugador.

```gdscript
func _on_body_entered(body):

    if is_enemy_bullet and body.is_in_group("player"):
        body.take_damage(damage)
        queue_free()
        return

    if not is_enemy_bullet and body.is_in_group("enemy"):
        body.take_damage(damage)
        queue_free()
        return

    queue_free()
```

Ilustración 84. Captura de la función _on_body_entered.

Asimismo, el proyectil se destruye automáticamente cuando abandona la pantalla, utilizando un nodo VisibleOnScreenNotifier2D que emite una señal al salir del área visible.

```gdscript
func _on_visible_on_screen_notifier_2d_screen_exited():
    queue_free()
```

Ilustración 85. Captura de la función _on_visible_on_screen_notifier_2d_screen_exited.

La gestión del estado del jugador incluye un sistema de invulnerabilidad temporal tras recibir daño, acompañado de un efecto visual de parpadeo. Esta lógica la implemento utilizando corrutinas con await, una característica propia de GDScript que simplifica la gestión de eventos temporales sin necesidad de estructuras adicionales.

```gdscript
func start_invulnerability():

    is_invulnerable = true

    await blink_invulnerability()

    is_invulnerable = false
```

Ilustración 86. Captura del método start_invulnerability().

Tras implementar toda esta lógica, ya tenía las mecánicas básicas del jugador. El jugador podía moverse, rotar y disparar.

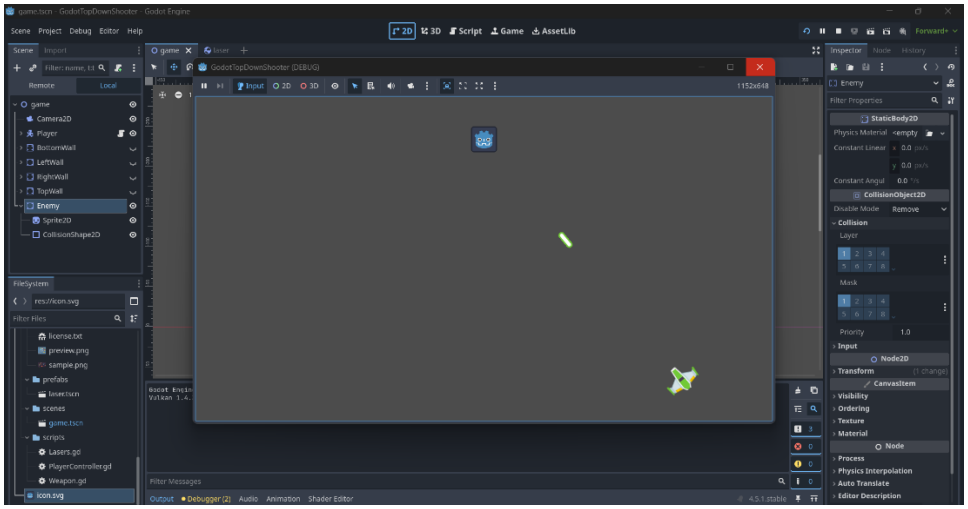


Ilustración 87. Captura de la lógica básica del jugador.

3.2.3 Lógica del enemigo y power-up

Lo siguiente fue la implementación del comportamiento de los enemigos en Godot. La lógica del enemigo sigue un enfoque basado en nodos del motor, reutilizando gran parte de la arquitectura definida para el jugador. Esta reutilización se puede ver en el uso compartido del sistema de disparo Weapon y del script de proyectiles Laser.

El enemigo lo implemento como un CharacterBody2D, al igual que el jugador, lo que permite reutilizar el mismo sistema de movimiento basado en velocidad y move_and_slide(). Sin embargo, en este caso la dirección de movimiento no depende de la entrada del usuario, sino de una lógica autónoma que tiene en cuenta la posición del jugador. Esta referencia es obtenida mediante el sistema de grupos de Godot (get_first_node_in_group("player")), lo que evita la necesidad de asignaciones manuales desde el editor. La inicialización de esta referencia y de otros parámetros internos la realizo en el método _ready.

```
35 func _ready():
36
37     current_health = max_health
38
39     fire_timer = fire_rate
40
41     player = get_tree().get_first_node_in_group("player")
42
43     strafe_direction = 1 if randf() > 0.5 else -1
44     strafe_timer = strafe_change_interval
45
```

Ilustración 88. Captura de la función _ready dentro de EnemyController.gd.

El movimiento del enemigo lo baso en un comportamiento combinado de persecución, mantenimiento de distancia y desplazamiento lateral, lo que genera trayectorias más dinámicas. Este comportamiento se calcula en _process, donde se determina la dirección hacia el jugador, la distancia relativa y un vector perpendicular que introduce variación en el movimiento. En Godot se puede centralizar gran parte del cálculo en _process, dejando la aplicación física para _physics_process.



```gdscript
func _process(delta):

    if player == null:
        return

    var enemy_position = global_position
    var player_position = player.global_position

    var to_player = player_position - enemy_position
    var distance = to_player.length()
    var dir_to_player = to_player.normalized()

    strafe_timer -= delta
    if strafe_timer <= 0:
        strafe_direction *= -1
        strafe_timer = strafe_change_interval

    var strafe = Vector2(-dir_to_player.y, dir_to_player.x) * strafe_direction

    var distance_movement = Vector2.ZERO

    if distance > preferred_distance:
        distance_movement = dir_to_player
    elif distance < retreat_distance:
        distance_movement = -dir_to_player

    var final_direction = distance_movement + strafe * strafe_strength
    final_direction = final_direction.normalized()

    desired_velocity = final_direction * move_speed

    fire_timer -= delta
    if fire_timer <= 0:
        weapon.fire()
        fire_timer = fire_rate
```

Ilustración 89. Captura de la función _process dentro de EnemyController.gd.

Una vez calculada la velocidad deseada, esta se aplica de forma progresiva mediante interpolación (lerp) en _physics_process, lo que permite suavizar el movimiento del enemigo. Esta lógica proporcionando un control más directo sobre la respuesta del movimiento.

```gdscript
func _physics_process(delta):

    if player == null:
        return

    velocity = velocity.lerp(desired_velocity, acceleration * delta)

    move_and_slide()

    rotate_towards_player(delta)
```

Ilustración 90. Captura del método _physics_process dentro de EnemyController.gd.

La rotación del enemigo sigue un principio similar al del jugador, orientándose constantemente hacia la posición del jugador. Esto lo realizo mediante interpolación angular (lerp_angle), lo que evita cambios bruscos de orientación y genera un comportamiento más natural.

```
95
96  func rotate_towards_player(delta):
97
98      var direction = player.global_position - global_position
99      var target_angle = direction.angle() + deg_to_rad(90)
100
101     rotation = lerp_angle(rotation, target_angle, rotation_speed * delta)
```

Ilustración 91. Captura del método rotate_towards_player dentro de EnemyController.gd.

En cuanto al sistema de disparo, el enemigo utiliza exactamente el mismo nodo Weapon que el jugador. La única diferencia relevante es el uso de un prefab de láser distinto, lo que permite variar el comportamiento o la apariencia de los proyectiles sin modificar la lógica. Además, se establece una propiedad (is_enemy_bullet) en el proyectil para diferenciar su origen, lo que permite que el mismo script Laser gestione correctamente las colisiones tanto para disparos del jugador como de enemigos.

El sistema de vida del enemigo lo he integrado directamente en el mismo script EnemyController, aunque también se presenta una versión separada EnemyHealth. En la implementación final, la lógica de daño la gestiono mediante el método take_damage, que reduce la vida y evalúa la condición de muerte.

```
104 func take_damage(amount:int):
105
106     current_health -= amount
107
108     if current_health <= 0:
109         die()
110
```

Ilustración 92. Captura de la función take_damage dentro de EnemyController.gd.

Cuando el enemigo muere, se añade puntuación al sistema global ScoreManager y se evalúa la posibilidad de generar un power-up, este proceso lo implemento en el método die.

```
112 func die():
113
114     ScoreManager.add_score(score_value)
115
116     try_drop_powerup()
117
118     queue_free()
```

Ilustración 93. Captura del método die() dentro de EnemyController.gd.

La generación de power-ups introduce un comportamiento probabilístico, utilizando randf() para determinar si se instancia un objeto de mejora. Es importante destacar el uso de call_deferred al añadir el nodo a la escena, lo cual evita errores relacionados con el sistema de físicas en Godot.

```gdscript
func try_drop_powerup():

    if powerup_scene == null:
        return

    if randf() <= drop_chance:

        var powerup = powerup_scene.instantiate()

        get_tree().current_scene.call_deferred("add_child", powerup)
        powerup.global_position = global_position
```

Ilustración 94. Captura del método try_drop_powerup() dentro de EnemyController.gd.

Los power-ups los implemento como nodos Area2D, utilizando detección de colisiones mediante señales. Cuando el jugador entra en contacto con uno de estos objetos, se aplica un efecto temporal sobre su arma, concretamente una mejora en la cadencia de disparo.

```gdscript
func _on_body_entered(body):

    if body.is_in_group("player"):

        var weapon = body.get_node_or_null("Weapon")

        if weapon != null:
            weapon.apply_fire_rate_boost(fire_rate_multiplier, duration)

        queue_free()
```

Ilustración 95. Captura de la función _on_body_entered dentro de power_up.gd.

Este enfoque resulta especialmente flexible en Godot, ya que el acceso a nodos hijos es directo y no requiere componentes intermedios. Adicionalmente, el uso de grupos simplifica la identificación de entidades, reduciendo la necesidad de etiquetas o comprobaciones adicionales.

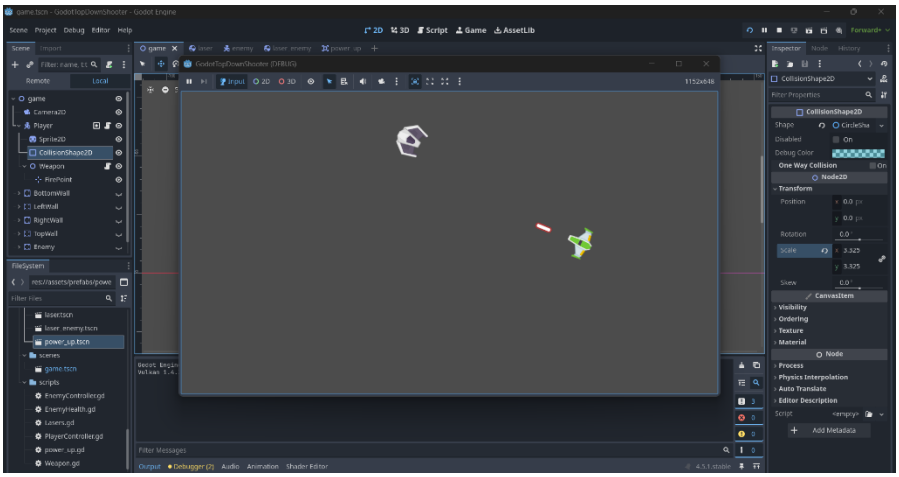


Ilustración 96. Captura de la lógica del enemigo y el power-up implementada.

3.2.4 Lógica del timer, vidas del jugador y puntuación

La implementación de los sistemas de puntuación, temporizador y gestión de vidas los abordé mediante el uso de nodos autoload o singletons, lo que me permite centralizar la lógica y acceder a estos sistemas desde cualquier parte del proyecto. La integración entre lógica y escena presenta particularidades que han condicionado tanto el diseño como la resolución de ciertos problemas durante el desarrollo.

El sistema de puntuación es un nodo global ScoreManager, encargado de almacenar el valor actual de la puntuación, actualizar la interfaz y conservar el resultado final entre escenas. La lógica de incremento de puntuación se encuentra en el método add_score, el cual modifica el valor interno y actualiza la interfaz.

```
8
9  func add_score(amount:int):
10
11     score += amount
12     print("Score actual:", score)
13
14     update_ui()
15
```

Ilustración 97. Captura de la función add_score.

La actualización de la interfaz la realizo en un método separado, update_ui, que asigna el texto al nodo visual asociado. En Godot surgió un problema importante: los nodos autoload no tienen acceso directo a los nodos de la escena activa. Esto implica que la referencia al Label no puede establecerse automáticamente al inicio. Este problema lo resolví mediante un método explícito de conexión desde el propio nodo de la interfaz.

```
17 func update_ui():
18
19     if score_label == null:
20         return
21
22     score_label.text = "Score: " + str(score)
23
```

Ilustración 98. Captura del método update_ui.

Para solucionar la ausencia de referencia, he implementado el método set_score_label, que permite asignar el nodo Label dinámicamente una vez que la escena ha sido cargada.

```
24
25 func set_score_label(label: Label):
26
27     score_label = label
28     print("Score conectado:", label)
29
30     update_ui()
31
```

Ilustración 99. Captura de la función set_score_label.

La conexión se realiza desde el propio nodo de la interfaz, lo que invierte el flujo habitual de dependencia; aquí es la UI la que se registra en el gestor.

```
1   extends Label
2
3   func _ready():
4       print("ScoreLabel READY")
5       ScoreManager.set_score_label(self)
```

Ilustración 100. Captura de la función _ready dentro de score_text.gd.

Este cambio fue necesario debido a que, en Godot, los nodos autoload se inicializan antes que las escenas, lo que provocaba que el Label fuese null en los primeros intentos de actualización. Este fue uno de los principales problemas que encontré durante la implementación, manifestándose en que la puntuación se actualizaba internamente pero no se reflejaba en pantalla. La solución consistió en establecer explícitamente la conexión una vez el nodo de UI estaba listo.

El sistema de temporizador sigue una lógica similar, gestionando el tiempo transcurrido mediante una variable acumulativa que se incrementa en cada frame dentro del método _process, integrada directamente en el ciclo de vida del nodo.

```
11  func _process(delta):
12
13      if not is_running:
14          return
15
16      elapsed_time += delta
17
18      update_timer_ui()
```

Ilustración 101. Captura de la función _process.

La actualización del texto la realizo en el método update_timer_ui, donde se convierte el tiempo acumulado a formato minutos y segundos.

```
21  func update_timer_ui():
22
23      if timer_text == null:
24          return
25
26      var minutes = int(elapsed_time / 60)
27      var seconds = int(elapsed_time) % 60
28
29      timer_text.text = "%02d:%02d" % [minutes, seconds]
30
```

Ilustración 102. Captura de la función update_timer_ui().

Al igual que en el sistema de puntuación, me surgió el mismo problema relacionado con la referencia al nodo de interfaz. Inicialmente, el temporizador no se mostraba en pantalla debido a que timer_text era null. Este problema lo resolví utilizando un método de asignación similar al del ScoreManager.

```
32  func set_timer_label(label: Label):
33      timer_text = label
34      print("Timer conectado:", label)
35
```

Ilustración 103. Captura del método set_timer_label.

Y su correspondiente llamada desde el nodo de interfaz.

```
1   extends Label
2
3
4   func _ready():
5       TimerManager.set_timer_label(self)
```

Ilustración 104. Captura de la función _ready dentro de time_label.gd.

Otro problema relevante me apareció en la gestión del tiempo final mostrado en la pantalla de Game Over. Al principio, el valor mostrado era siempre cero, lo que se debía a que la variable last_time no se estaba actualizando correctamente antes de cambiar de escena. Lo solucioné asegurando que el método save_final_time se llamase explícitamente antes de finalizar la partida.

```
41  func save_final_time():
42      print("Tiempo guardado:", last_time)
43      last_time = elapsed_time
44
```

Ilustración 105. Captura de la función save_final_time dentro de timer_manager.gd.

En Godot, aunque los autoloads también mantienen su estado, es necesario controlar cuidadosamente el momento en el que se actualizan los valores para evitar inconsistencias.

En cuanto al sistema de vidas del jugador, este lo integro directamente dentro del controlador del jugador, actualizando un Label asociado cada vez que se modifica el número de vidas. A diferencia del sistema de puntuación y tiempo, en este caso la referencia al nodo de interfaz sí se realiza directamente mediante una variable exportada, lo que simplifica su uso al estar dentro de la misma escena.

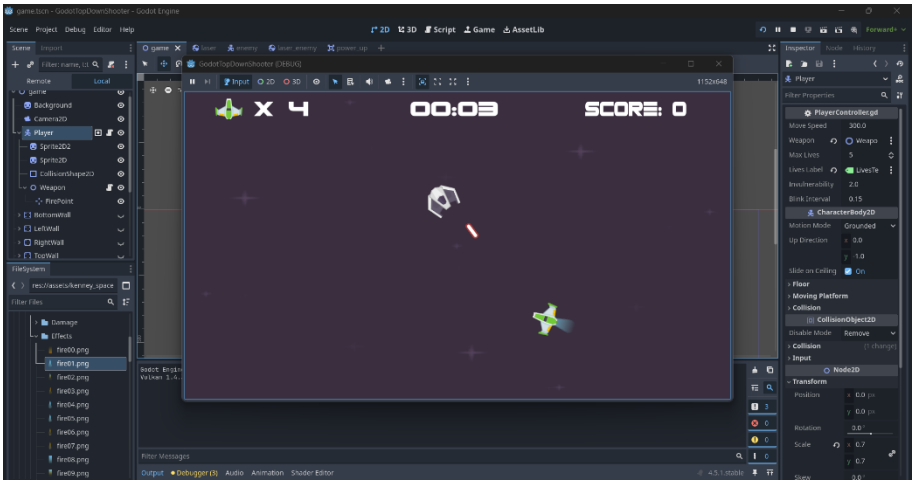


Ilustración 106. Captura de la lógica implementada del tiempo, puntuación y vidas.

3.2.5 Lógica del spawner, menú de pausa y música

El sistema de generación de enemigos en Godot lo he realizado siguiendo una lógica similar a la desarrollada anteriormente. En este caso, el comportamiento se encapsula en un nodo independiente que actúa como gestor del flujo de enemigos durante la partida.

La inicialización del sistema la realizo en el método _ready(), donde se obtiene una referencia a la cámara activa mediante get_viewport().get_camera_2d() y se establece el intervalo inicial de aparición.

```
15  func _ready():
16
17      camera = get_viewport().get_camera_2d()
18      current_interval = spawn_interval
19
20      spawn_loop()
```

Ilustración 107. Captura de la función _ready() dentro de enemy_spawner.gd.

En Godot la repetición temporal la implemento mediante una función asíncrona que utiliza await junto con temporizadores del árbol de escena. La función spawn_loop() representa el núcleo del sistema, generando enemigos de forma continua y ajustando progresivamente la dificultad.

```
22  func spawn_loop() -> void:
23
24      while true:
25
26          spawn_enemy()
27
28          current_interval = max(min_spawn_interval, current_interval - difficulty_increase_rate)
29
30          await get_tree().create_timer(current_interval, false).timeout
31
```

Ilustración 108. Captura de la función spawn_loop() dentro de enemy_spawner.gd.

De esta manera integro la asincronía directamente en el lenguaje GDScript. Asimismo, el control de dificultad lo realizo modificando el intervalo de aparición, lo que produce un

incremento gradual en la frecuencia de enemigos, replicando el comportamiento original pero con una sintaxis más concisa.

La instanciación de enemigos la realizo mediante PackedScene. Este comportamiento se describe en el método spawn_enemy().

```
32  func spawn_enemy():
33
34      var spawn_position = get_spawn_position()
35
36      var enemy = enemy_scene.instantiate()
37
38      get_tree().current_scene.add_child(enemy)
39
40      enemy.global_position = spawn_position
```

Ilustración 109. Captura de la función spawn_enemy() dentro de enemy_spawner.gd.

Aquí utilizo instantiate() y el nodo resultante se inserta manualmente en el árbol de escena. Esto exhibe la naturaleza jerárquica del sistema de Godot, donde toda instancia debe formar parte explícita del árbol para existir en la simulación.

La lógica de posicionamiento de los enemigos la calculo manualmente como un rectángulo basado en la posición de la cámara y el tamaño del viewport.

```
74  func get_camera_rect() -> Rect2:
75
76      var viewport_size = get_viewport_rect().size
77      var camera_pos = camera.global_position
78
79      var half_size = viewport_size / 2
80
81      return Rect2(camera_pos - half_size, viewport_size)
```

Ilustración 110. Captura de la función get_camera_rect() dentro de enemy_spawner.gd.

A partir de este rectángulo, se generan posiciones aleatorias fuera de los límites visibles, garantizando que los enemigos aparezcan progresivamente en pantalla. La selección del lado de aparición se realiza mediante una estructura match, equivalente al switch.

```gdscript
func get_spawn_position() -> Vector2:

    var rect = get_camera_rect()

    var side = randi() % 4

    match side:

        0: #arriba
            return Vector2(
                randf_range(rect.position.x, rect.end.x),
                rect.position.y - spawn_distance
            )

        1: #abajo
            return Vector2(
                randf_range(rect.position.x, rect.end.x),
                rect.end.y + spawn_distance
            )

        2: #izquierda
            return Vector2(
                rect.position.x - spawn_distance,
                randf_range(rect.position.y, rect.end.y)
            )

        _: #derecha
            return Vector2(
                rect.end.x + spawn_distance,
                randf_range(rect.position.y, rect.end.y)
            )
```

Ilustración 111. Captura de la función get_spawn_position() dentro de enemy_spawner.gd.

Sobre el apartado del sonido, la implementación en Godot la he basado en el uso de nodos AudioStreamPlayer2D, lo que permite integrar tanto música ambiental como efectos de sonido directamente en la escena. Para la música de fondo, he añadido un nodo de este tipo configurado en modo loop, lo que permite su reproducción continua sin necesidad de lógica adicional.

Para los efectos de disparo, el sistema lo he integrado dentro del propio script de Weapon..

```gdscript
    if shoot_sound:
        audio_player.stream = shoot_sound
        audio_player.play()
```

Ilustración 112. Captura de la lógica de efectos de sonido dentro de Weapon.gd.

Esto permite que tanto el jugador como los enemigos compartan la misma lógica de sonido, dado que ambos utilizan el mismo script de arma, diferenciándose únicamente en los recursos asignados.

Por último, para el sistema de pausa, he optado por una escena independiente basada en un nodo Control, lo que favorece la reutilización y modularidad. La jerarquía compuesta por

PanelContainer y VBoxContainer permite organizar automáticamente los elementos de interfaz, eliminando la necesidad de ajustes manuales de posición.

La lógica de pausa la implemento mediante la propiedad global get_tree().paused, que detiene la simulación completa del juego. Este comportamiento se define en los métodos pause() y resume().

```
22  func pause():
23
24      panel.visible = true
25      get_tree().paused = true
26      is_paused = true
27
28
29  func resume():
30
31      panel.visible = false
32      get_tree().paused = false
33      is_paused = false
```

Ilustración 113. Captura de la lógica de pausa y reanudar dentro de pause_menu.gd.

La pausa afecta directamente al árbol de nodos, lo que simplifica la implementación, pero requiere configurar adecuadamente qué nodos continúan procesando, nodos como la interfaz.

En cuanto al diseño visual de los botones y elementos de interfaz, he utilizado recursos .tres, propios de Godot, para definir estilos reutilizables. Estos archivos permiten configurar propiedades como colores, tipografías o estados de los botones sin necesidad de programación adicional.

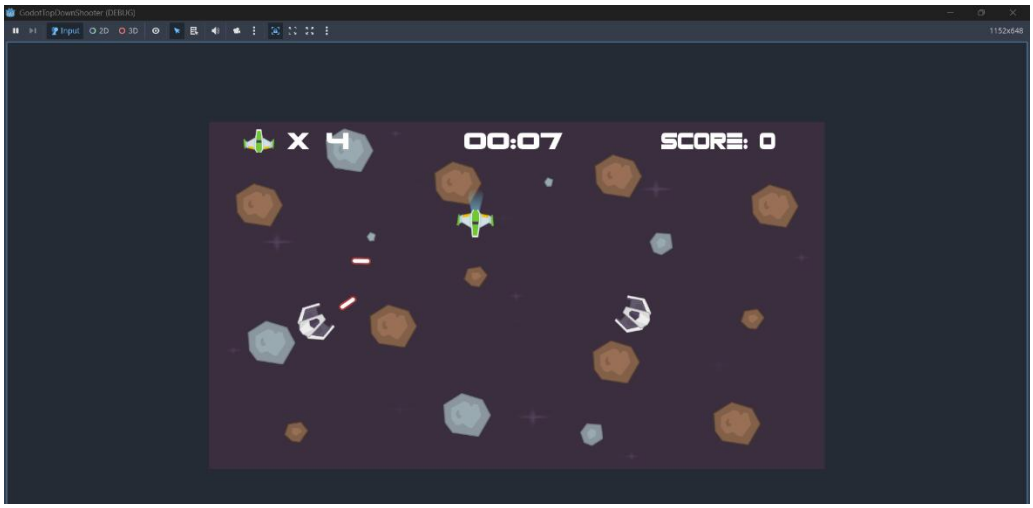


Ilustración 114. Captura de la lógica del spawner, menú de pausa y música.

3.2.6 Escenas game over y main menu

La lógica de los menús sigue un estilo similar al resto del proyecto, reutilizando tanto elementos visuales como estructuras de escena ya empleadas en el juego principal. En lugar de construir interfaces completamente aisladas, he optado por mantener una base común compuesta por la cámara, el fondo y varios sprites decorativos de meteoritos, sobre la cual se superpone un nodo Control encargado de gestionar la interfaz. Así logro mantener una coherencia estética entre escenas sin necesidad de duplicar lógica ni recursos.

En el caso del menú principal, me limité a gestionar la navegación entre escenas y la salida de la aplicación. El nodo raíz Control actúa como contenedor de los elementos interactivos, incluyendo un Label para el título del juego y dos botones. La funcionalidad la implementa directamente en los callbacks asociados a estos botones.

```
1   extends Control
2
3
4   func _on_play_button_pressed() -> void:
5       get_tree().change_scene_to_file("res://assets/scenes/game.tscn")
6       pass
7
8
9
10  func _on_quit_button_pressed() -> void:
11      get_tree().quit()
12      pass
```

Ilustración 115. Captura de la lógica de los botones de mainMenu.gd.

El acceso al árbol de escenas (SceneTree) centraliza este tipo de operaciones, simplificando considerablemente la lógica.

Por otra parte, la inclusión de música de fondo la he realizado mediante un nodo AudioStreamPlayer2D configurado en bucle, lo que permite mantener una ambientación sonora constante sin necesidad de control manual desde código.

El menú de Game Over introduce una capa adicional de complejidad, ya que no solo presenta opciones de navegación, sino que también debe mostrar información persistente de la partida, como la puntuación final y el tiempo sobrevivido. Esta información se obtiene a través de los sistemas globales ScoreManager y TimerManager configurados como nodos autoloads.

La lógica de inicialización de la interfaz la concentro en el método _ready(), donde llamo a las funciones encargadas de actualizar los textos.

```
1   extends Control
2
3   @export var score_label : Label
4   @export var time_label : Label
5
6
7   func _ready():
8       display_final_score()
9       display_final_time()
```

Ilustración 116. Captura de la función _ready() dentro de game_over.gd.

La puntuación se obtiene directamente del gestor global y se asigna al Label correspondiente.

```
11  func display_final_score():
12
13      score_label.text = "Score: " + str(ScoreManager.last_score)
```

Ilustración 117. Captura de la función display_final_score() dentro de game_over.gd.

Del mismo modo, el tiempo se formatea en minutos y segundos antes de mostrarse.

```
15  func display_final_time():
16
17      var time = TimerManager.last_time
18
19      var minutes = int(time / 60)
20      var seconds = int(time) % 60
21
22      time_label.text = "Time: %02d:%02d" % [minutes, seconds]
```

Ilustración 118. Captura del método display_final_time() dentro de game_over.gd.

Durante el desarrollo de este apartado me surgieron varios problemas relevantes. Uno de los errores principales fue la imposibilidad de acceder a TimerManager.last_time, lo que generaba errores de compilación. Este problema se debía a que en Godot es necesario que el script esté correctamente registrado como autoload para que sea accesible globalmente. La solución consistió en configurar TimerManager como singleton en los ajustes del proyecto.

Otro problema importante fue que el tiempo mostrado en la pantalla de Game Over siempre era cero. Esto se debía a que el valor de last_time no se estaba actualizando correctamente antes de cambiar de escena. En concreto, el método save_final_time() no se ejecutaba en el momento adecuado dentro de la lógica del jugador. Lo solucioné asegurando que dicho método se llamase justo antes de eliminar al jugador o cambiar de escena, garantizando así que el valor persistiera correctamente.

También detecté errores relacionados con la interfaz, como asignar texto a nodos Label nulos. Este problema pasa en Godot cuando las referencias a nodos no están correctamente enlazadas mediante @export o no existen en la jerarquía esperada. En Godot es frecuente depender de rutas de nodos o asignaciones dinámicas, lo que puede provocar este tipo de errores si no se validan adecuadamente.

La navegación desde el menú de Game Over es similar al menú principal, pero incluyendo la reinicialización del estado del juego.

```gdscript
func _on_replay_button_pressed() -> void:
	ScoreManager.reset_score()
	TimerManager.reset_timer()
	get_tree().change_scene_to_file("res://assets/scenes/game.tscn")
	pass


func _on_menu_button_pressed() -> void:
	ScoreManager.reset_score()
	TimerManager.reset_timer()
	get_tree().change_scene_to_file("res://assets/scenes/menu.tscn")
	pass
```

Ilustración 119. Captura de la lógica de los botones dentro de game_over.gd.

Este paso adicional es fundamental cuando se utilizan singletons, ya que estos persisten entre escenas y mantienen su estado si no se reinician manualmente.



Ilustración 120. Capturas de las escenas game over y main menu acabadas en Godot.

3.2.7 Build del juego

Sobre la generación de un ejecutable final en Godot, esta la he realizado mediante el sistema de exportación integrado en el propio editor, lo que me permitió transformar el proyecto en una aplicación distribuible sin necesidad de herramientas externas adicionales. Para ello, he utilizado la opción Export dentro del menú del proyecto, donde previamente he configurado un perfil de exportación para Windows. Este perfil define aspectos como la arquitectura objetivo, el nombre del ejecutable y la inclusión de los recursos del proyecto.

Una vez configurado, el proceso de exportación genera un único archivo ejecutable .exe junto con, en algunos casos, un archivo .pck que contiene los recursos del juego. Sin embargo, Godot permite integrar directamente estos recursos dentro del ejecutable, produciendo así un único archivo autónomo, ofreciendo una solución más compacta y sencilla de distribuir, especialmente en proyectos pequeños o prototipos.

Esta diferencia tiene implicaciones directas en la distribución del juego. En Godot, el hecho de poder disponer de un único ejecutable simplifica tanto la entrega como la ejecución en sistemas externos, evitando errores derivados de rutas o archivos faltantes. Aunque hay que tener en cuenta que, si uno no tiene una firma digital en Godot, el ejecutable lanzará un mensaje de alerta cuando se intente ejecutar por primera vez.

3.2.8 Sobre los shaders

En relación con el uso de shaders, aunque en este proyecto no los he implementado, es relevante hablar sobre como funcionan en Godot. En Godot, los shaders se escriben utilizando un lenguaje propio, denominado Godot Shading Language, que presenta una sintaxis similar a GLSL, pero adaptada a la estructura del motor. Estos shaders se aplican directamente a nodos mediante recursos de tipo ShaderMaterial, lo que permite modificar el comportamiento visual de sprites, luces o superficies sin necesidad de componentes adicionales.

Una de las características más distintivas de Godot es la simplicidad con la que se pueden integrar shaders en la escena. Al estar completamente integrados en el sistema de nodos, es posible asignarlos y modificarlos en tiempo real desde el editor, observando los cambios de forma inmediata. Adicionalmente, Godot unifica el uso de shaders 2D y 3D dentro de un mismo lenguaje.

En Godot no se depende de pipelines de renderizado configurables, lo que simplifica el proceso de creación de efectos visuales, aunque a costa de una menor flexibilidad en proyectos de gran escala.

En resumen, aunque no haya utilizado shaders en este proyecto, su integración en Godot representa una herramienta potente y accesible que refuerza la filosofía del motor: ofrecer un entorno unificado, sencillo y altamente interactivo para el desarrollo de videojuegos.

3.3 Desarrollo en GameMaker

3.3.1 Diagrama de clases

Como en el caso del desarrollo en Unity y Godot, también he hecho un diagrama de clases para GameMaker. En este diagrama, utilizo un modelo más simplificado en el que la lógica del juego se organiza principalmente a través de objetos autocontenidos y scripts globales. Esto implica que el concepto de clase no se implementa de forma estricta, sino que se aproxima mediante objetos que encapsulan comportamiento y estado, junto con funciones reutilizables externas.

El diagrama está formado por un conjunto de objetos que representan las entidades principales del juego, así como de un estado global compartido modelado en el diagrama como GlobalState (Autoload). Esto sustituye a los sistemas de gestión global presentes en otros motores, permitiendo almacenar información persistente como la puntuación o el tiempo de partida sin necesidad de instanciar gestores específicos. Esta organización simplifica el acceso a los datos, aunque introduce una mayor dependencia de variables globales.

El objeto obj_player concentra toda la lógica relacionada con el jugador. Aquí se unifica en un único objeto que gestiona el movimiento, la rotación hacia el cursor y el sistema de disparo. También se encarga del control de vida, incluyendo la recepción de daño, la activación de un periodo de invulnerabilidad temporal y la gestión de la muerte del jugador. Este evento desencadena la transición al estado de fin de partida y la actualización de los valores globales correspondientes.

El comportamiento de los enemigos lo implemento en obj_enemy, que encapsula tanto la lógica de movimiento como la de ataque. Su funcionamiento se basa en calcular la dirección hacia el jugador y ajustar su posición para mantener una distancia determinada, incorporando además variaciones laterales para generar un movimiento más dinámico. Asimismo, el objeto gestiona su propia rotación y la generación de proyectiles mediante una función de disparo.

Los proyectiles los divido en dos objetos diferenciados, obj_laser y obj_laser_enemy, cada uno asociado a un tipo de entidad. Ambos gestionan su desplazamiento mediante propiedades internas de velocidad y dirección, y detectan colisiones con sus respectivos objetivos. Al producirse un impacto, aplican daño al objeto correspondiente y se destruyen. Adicionalmente, controlan su propia eliminación cuando abandonan los límites de la pantalla, garantizando la correcta gestión de memoria durante la ejecución.

El sistema de generación de enemigos lo implemento mediante obj_enemy_spawner, que se encarga de crear instancias de enemigos de forma periódica. Esta lógica se basa en temporizadores internos y en el cálculo de posiciones de aparición fuera de los límites visibles de la pantalla, lo que permite que los enemigos entren en escena de forma progresiva. La frecuencia de aparición puede ajustarse dinámicamente, facilitando la escalabilidad de la dificultad.

El objeto obj_powerup representa los elementos de mejora que el jugador puede recoger. Su funcionamiento se basa en la detección de colisiones con el jugador, tras lo cual aplica un

efecto temporal que modifica parámetros como la cadencia de disparo. Una vez aplicado el efecto, el objeto se destruye, manteniendo la coherencia del sistema.

El estado global del juego se agrupa en GlobalState (Autoload), que actúa como contenedor de variables compartidas como la puntuación, el tiempo transcurrido o el estado de pausa. Tanto los objetos de juego como los de interfaz dependen de este estado para actualizar y mostrar información relevante.

La interfaz de usuario la gestiono mediante objetos como obj_ui, que se encargan de dibujar en pantalla elementos como la puntuación, el temporizador o las vidas del jugador. En GameMaker esta se implementa directamente mediante eventos de dibujo, lo que otorga mayor control pero requiere una gestión manual de posicionamiento y formato.

El sistema de pausa lo integro dentro de la lógica global del juego, utilizando una variable compartida que controla si el juego está detenido. Los objetos verifican este estado antes de ejecutar su lógica, lo que permite congelar el comportamiento del juego sin necesidad de un sistema de pausa centralizado. La interfaz de pausa se dibuja mediante el mismo sistema de UI, incluyendo opciones para reanudar la partida o salir al menú.

La lógica del tiempo transcurrido la implemento en la clase obj_timer_manager, donde en su método create inicializo las variables y en su step ejecuto la lógica del tiempo transcurrido.

Por otro lado, el flujo entre escenas lo gestiono mediante objetos como obj_main_menu y obj_game_over. El primero actúa como punto de entrada al juego, permitiendo iniciar la partida o cerrar la aplicación. El segundo muestra la información final del jugador, accediendo a los valores almacenados en el estado global para presentar la puntuación y el tiempo alcanzado, además de ofrecer opciones para reiniciar o volver al menú principal.

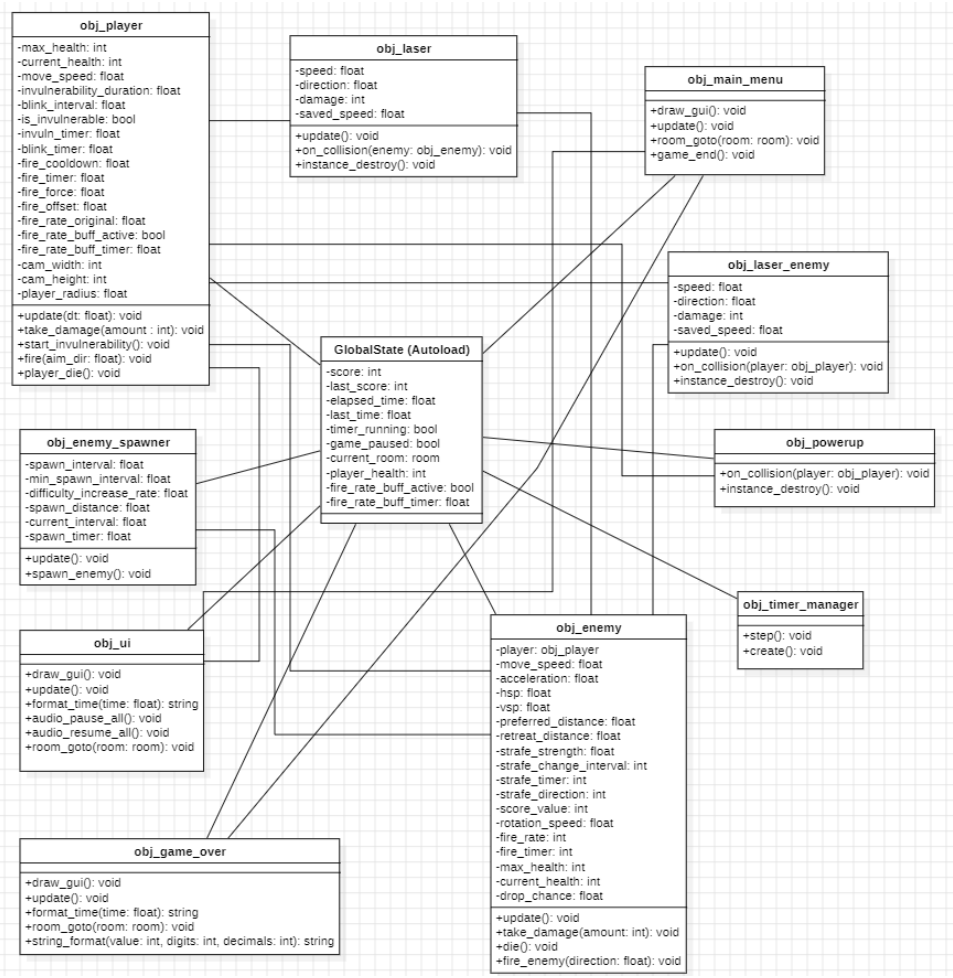
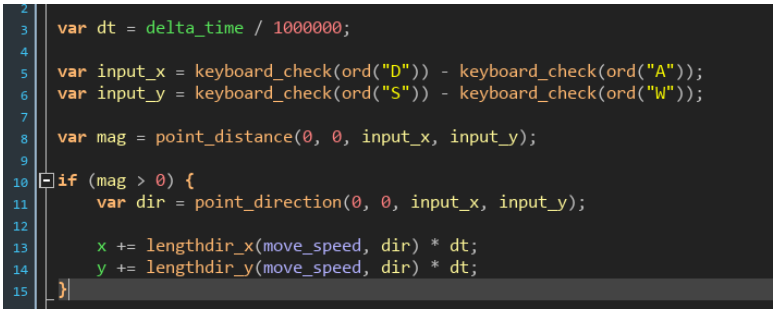


Ilustración 121. Diagrama de clases del proyecto en GameMaker.

3.3.2 Lógica del jugador

Empecé el desarrollo en GameMaker con la implementación del comportamiento del jugador. En GameMaker, la lógica la realizo directamente sobre los eventos del objeto, especialmente el evento Step, lo que implica que, al programar, gestiono manualmente tanto la actualización de la posición como la respuesta al input. Este modelo emplea un control más explícito del estado en cada frame.

La primera parte de la lógica del jugador fue el movimiento de este. El movimiento se basa en la lectura directa del teclado y en la transformación de dicho input en un desplazamiento sobre el plano cartesiano. Empleando funciones trigonométricas que permiten trabajar con direcciones y magnitudes de forma más directa. Este comportamiento lo implemento mediante el cálculo de un vector de entrada normalizado y su conversión a desplazamiento en los ejes X e Y.



```
var dt = delta_time / 1000000;

var input_x = keyboard_check(ord("D")) - keyboard_check(ord("A"));
var input_y = keyboard_check(ord("S")) - keyboard_check(ord("W"));

var mag = point_distance(0, 0, input_x, input_y);

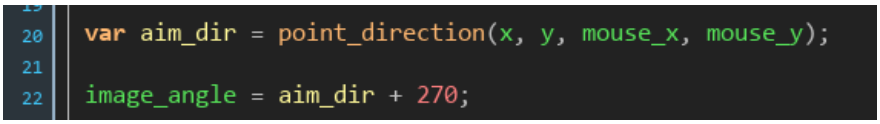
if (mag > 0) {
    var dir = point_direction(0, 0, input_x, input_y);

    x += lengthdir_x(move_speed, dir) * dt;
    y += lengthdir_y(move_speed, dir) * dt;
}
```

Ilustración 122. Captura de la lógica de movimiento dentro del step del obj_player.

Funciones como lengthdir_x y lengthdir_y permiten descomponer un vector de movimiento a partir de un ángulo sin necesidad de estructuras adicionales. Esto simplifica la sintaxis, pero traslada al desarrollador la responsabilidad de gestionar correctamente la dirección, la magnitud y la normalización del movimiento. El uso de delta_time garantiza que el desplazamiento sea independiente de la tasa de refresco, una consideración clave en ausencia de un sistema físico integrado.

La rotación del jugador es similar, calculándose directamente a partir de la posición del ratón. Aquí utilizo una función que devuelve el ángulo entre dos puntos en el espacio. Este valor se asigna directamente a la propiedad image_angle, que controla la orientación visual del sprite.



```
var aim_dir = point_direction(x, y, mouse_x, mouse_y);

image_angle = aim_dir + 270;
```

Ilustración 123. Captura de la lógica de rotación y orientación dentro del step del obj_player.

El ajuste de 270 grados es necesario para alinear la orientación del sprite con el sistema de coordenadas de GameMaker, lo que muestra cómo la representación visual y la lógica deben coordinarse manualmente.

El sistema de disparo lo he implementado mediante la creación dinámica de instancias de proyectiles. Esta funcionalidad la integro directamente en el objeto del jugador mediante una función que calcula la posición de aparición del proyectil en función de la dirección de disparo.

```
function fire(dir) {

    if (fire_timer > 0) return;

    audio_play_sound(sfx_laser1, 1, false);

    var forward_offset = 8;
    var lateral_offset = 55;
    var right_dir = dir - 90;

    var spawn_x = x;
    var spawn_y = y;

    spawn_x += lengthdir_x(forward_offset, dir);
    spawn_y += lengthdir_y(forward_offset, dir);

    spawn_x += lengthdir_x(lateral_offset, right_dir);
    spawn_y += lengthdir_y(lateral_offset, right_dir);

    var laser = instance_create_layer(spawn_x, spawn_y, "Instances", obj_laser);

    laser.direction = dir;
    laser.speed = fire_force;

    laser.image_angle = dir + 90;

    fire_timer = fire_cooldown;
}
```

Ilustración 124. Captura de la función de disparo en el script fire.

En este caso, la dirección del disparo la reutilizo tanto para el movimiento como para la orientación del proyectil, lo que simplifica la comunicación entre objetos. La función instance_create_layer la empleo para instanciar los lasers en la capa de “Instances”.

El objeto obj_laser encapsula la lógica del proyectil, delegando su movimiento en propiedades internas del motor como speed y direction. Esta característica permite que el desplazamiento lo gestione automáticamente sin necesidad de actualizar manualmente la posición en cada frame, lo que introduce una diferencia interesante respecto al control explícito aplicado al jugador.

```
speed = 12;
direction = 0;
damage = 1;

saved_speed = 0;
```

Ilustración 125. Captura del Create del obj_laser del player.

El comportamiento del proyectil ante colisiones lo implemento mediante eventos específicos, donde se verifica el tipo de objeto impactado y se aplica el daño correspondiente.

```
if (other.object_index == obj_enemy) {

    other.take_damage(damage);
    instance_destroy();
}
```

Ilustración 126. Captura de la lógica colisiones del proyectil dentro de obj_laser.

El ciclo de vida del proyectil se completa con su eliminación automática al salir de los límites de la pantalla, evitando la acumulación de instancias innecesarias. Este control lo implemento mediante una simple comprobación de posición.

```
14  if (x < -20 || x > 1300 || y < -20 || y > 740) {
15      instance_destroy();
16  }
```

Ilustración 127. Captura de la lógica fuera de pantalla del proyectil dentro de obj_laser.

La lógica del jugador y del sistema de disparo tiene un enfoque basado en el control directo y la simplicidad estructural. De esta manera, facilito la comprensión del comportamiento interno del juego y permite una implementación ágil de mecánicas 2D, aunque a costa de una mayor responsabilidad en la gestión manual de la lógica.

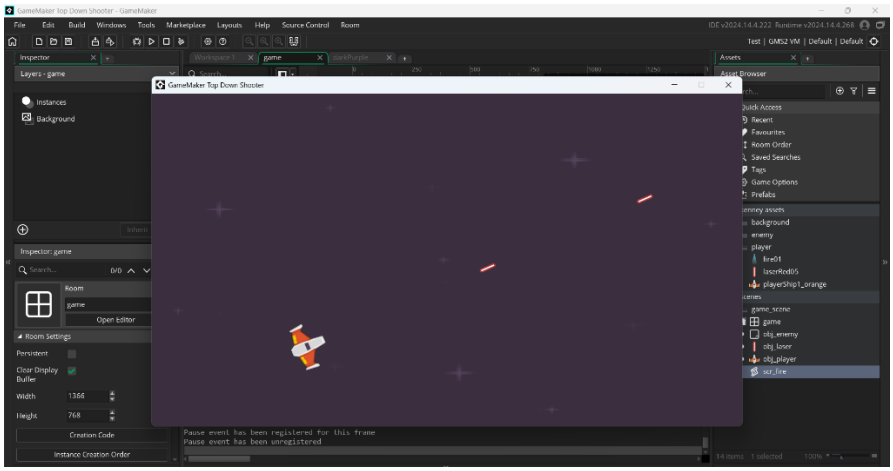


Ilustración 128. Captura de la lógica implementada del jugador en GameMaker.

3.3.3 Lógica del enemigo, spawner, power-up y puntuación

Sobre el comportamiento del enemigo, el sistema de power-ups y de la gestión de puntuación sigue un enfoque similar con la lógica previamente descrita para el jugador: un modelo centrado en objetos autónomos que gestionan su propio estado mediante eventos.

La implementación de la lógica del enemigo la hago directamente en el evento Step del objeto obj_enemy. Esto implica que tanto la toma de decisiones como la aplicación del movimiento ocurren en un mismo bloque de ejecución. El comportamiento resultante combina persecución, mantenimiento de distancia y desplazamiento lateral, lo que genera una IA sencilla pero dinámica.

```
strafe_timer  ;
if (strafe_timer <= 0) {
    strafe_direction *=  1;
    strafe_timer = strafe_change_interval;
}

var strafe_dir = dir_to_player + 90 * strafe_direction;

var move_dir = 0;

if (distance > preferred_distance) {
    move_dir = dir_to_player;
}
else if (distance < retreat_distance) {
    move_dir = dir_to_player + 180;
}
else {
    move_dir = strafe_dir;
}
```

Ilustración 129. Captura del Step del obj_enemy.

La dirección se expresa como un ángulo escalar, y el movimiento se descompone posteriormente en componentes X e Y. Esta conversión la realizo mediante las funciones trigonométricas propias del motor.

```
var target_vx = lengthdir_x(move_speed, move_dir);
var target_vy = lengthdir_y(move_speed, move_dir);

hsp = lerp(hsp, target_vx, acceleration);
vsp = lerp(vsp, target_vy, acceleration);

x += hsp * dt;
y += vsp * dt;
```

Ilustración 130. Captura lógica de dirección dentro del Step del obj_enemy.

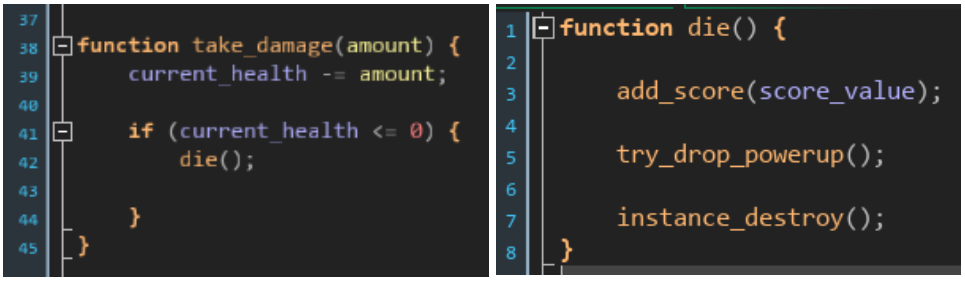
En GameMaker defino explícitamente las variables de velocidad y las inicializo correctamente en el evento Create. De hecho, uno de los problemas que he encontrado durante el desarrollo fue precisamente el error “variable not set before reading it”, causado por no haber inicializado estas variables antes de utilizarlas en la interpolación. La solución consistió en añadir su inicialización en el Create Event, lo que refleja cómo GameMaker exige un control más explícito del estado interno de los objetos.

La rotación del enemigo hacia el jugador sigue el mismo principio que en el jugador, adaptando el ángulo calculado al sprite. En este caso, utilizo una interpolación para suavizar el giro.

```
var target_angle = dir_to_player + 90;
image_angle = lerp(image_angle, target_angle, rotation_speed);
```

Ilustración 131. Captura lógica de rotación dentro del Step del obj_enemy.

En cuanto al sistema de daño y destrucción, la lógica la encapsulo en funciones definidas dentro del propio objeto enemigo. Así ambas responsabilidades las integro en un único objeto.



```
function take_damage(amount) {
    current_health -= amount;

    if (current_health <= 0) {
        die();

    }
}
```

```
function die() {

    add_score(score_value);

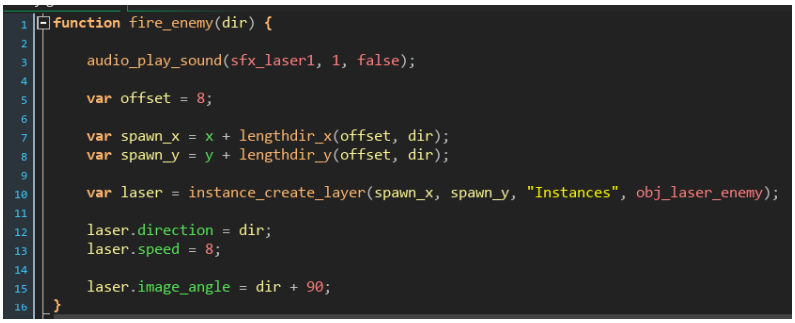
    try_drop_powerup();

    instance_destroy();
}
```

Ilustración 132. Capturas de las funciones take_damage y die().

En relación con el sistema de disparo del enemigo, he optado por replicar una lógica similar a la del jugador, pero desacoplada en un objeto independiente que actúa como proyectil enemigo, siendo este obj_laser_enemy.

El disparo del enemigo lo realizo mediante una función similar a la del jugador, que calcula la dirección hacia este y genera el proyectil con dicha orientación. Este comportamiento se encuentra en el Step, controlado por un temporizador (fire_timer) que regula la cadencia de disparo.



```
function fire_enemy(dir) {

    audio_play_sound(sfx_laser1, 1, false);

    var offset = 8;

    var spawn_x = x + lengthdir_x(offset, dir);
    var spawn_y = y + lengthdir_y(offset, dir);

    var laser = instance_create_layer(spawn_x, spawn_y, "Instances", obj_laser_enemy);

    laser.direction = dir;
    laser.speed = 8;

    laser.image_angle = dir + 90;
}
```

Ilustración 133. Captura de la función fire_enemy.

Este diseño reutiliza el mismo principio basado en ángulos que empleo en el jugador, lo que permite mantener coherencia en el sistema de disparo. No obstante, introduce una diferenciación clara a nivel del objeto obj_laser_enemy, lo que facilita la gestión de colisiones sin necesidad de flags.

El objeto obj_laser_enemy comparte estructura con el proyectil del jugador, pero está configurado para interactuar exclusivamente con este. Su comportamiento se basa en el uso de las propiedades internas speed y direction, lo que permite que el motor gestione automáticamente su movimiento, mientras que la lógica de colisión aplica daño directamente al jugador. De esta forma, cada objeto encapsula completamente su comportamiento sin necesidad de sistemas externos.

El sistema de power-ups lo implemento mediante la creación condicional de una nueva instancia tras la muerte del enemigo. La probabilidad de aparición la controlo mediante una comparación con un valor aleatorio.

```
function try_drop_powerup() {

    if (random(1) <= drop_chance) {

        instance_create_layer(x, y, "Instances", obj_powerup);
    }
}
```

Ilustración 134. Captura de la función try_drop_powerup().

Obj_powerup es un objeto que gestiona su efecto mediante una colisión directa con el jugador. En este objeto, recurro al evento de colisión, donde modifico directamente una propiedad del jugador.

```
var player = other;

player.fire_cooldown = player.fire_rate_original * 0.5;

player.fire_rate_buff_active = true;
player.fire_rate_buff_timer = 3.0;

instance_destroy();
```

Ilustración 135. Captura de la lógica del power-up.

Sin embargo, esta implementación inicial presentó una limitación importante, y es que el efecto era permanente. Para resolverlo, introduje un sistema temporal basado en contadores. Este comportamiento lo implemento manualmente mediante variables y control en el evento Step.

En la gestión de la puntuación, opto por variables globales agrupadas en un pseudo-singleton, como es el caso de global.score. Este valor lo incrementa directamente desde el enemigo al morir, eliminando la necesidad de referencias externas.

```
global.score = 0;

function add_score(amount) {
    global.score += amount;
}
```

Ilustración 136. Captura de la lógica de la puntuación global en la función add_score.

Durante el desarrollo, me resultó difícil implementar funciones externas, por lo que me llevó a replantear la arquitectura hacia un modelo basado en estado global. Esta solución, aunque menos formal que un singleton estructurado, la integro de forma natural con su sistema de variables globales.

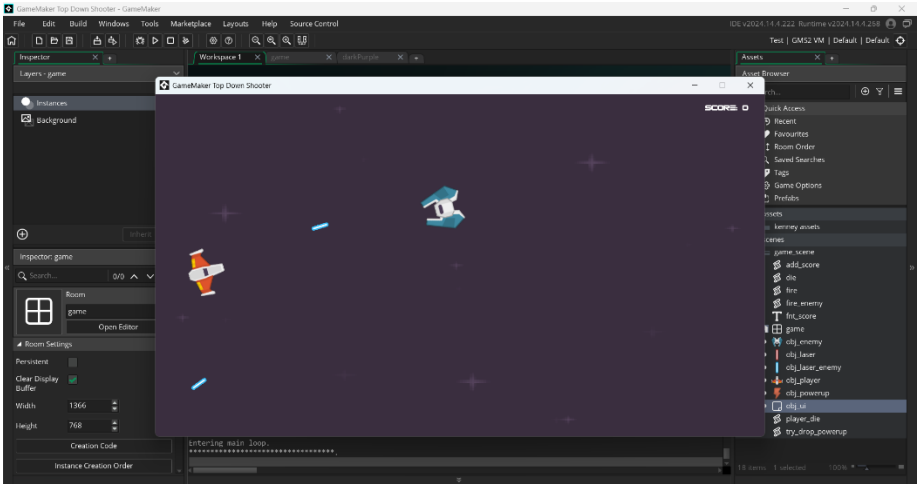


Ilustración 137. Captura de la lógica del enemigo, power-up y score en GameMaker.

Para hacer que se creen varias instancias del enemigo, tuve que crear la clase obj_enemy_spawner. En esta clase, inicializo las variables de la lógica en el método create de la clase; luego, en el step, compruebo si el juego está pausado. Si no lo está, entonces resto el tiempo transcurrido a la variable del temporizador de spawn. Cuando esta llega a ser menor o igual que cero, entonces ejecuto la función spawn_enemy(), donde creo una nueva instancia del obj_enemy en la capa de instance, calculo el nuevo intervalo de spawneo teniendo en cuenta el mínimo intervalo, el actual y la dificultad, y establezco el valor del nuevo intervalo al valor del tiempo de spawn.

```
var dt = delta_time / 1000000;

if (global.game_paused) exit;

spawn_timer -= dt;

if (spawn_timer <= 0) {

    spawn_enemy();

    current_interval = max(min_spawn_interval, current_interval - difficulty_increase_rate);

    spawn_timer = current_interval;
}
```

Ilustración 138. Captura de la lógica del método step del obj_enemy_spawner en GameMaker.

3.3.4 Lógica del timer, vidas del jugador y fondo de la escena

La lógica del temporizador, las vidas del jugador y los elementos decorativos del fondo continúan siguiendo la misma filosofía observada en los sistemas anteriores. Esta filosofía es una arquitectura basada en objetos independientes y en el uso de variables globales para compartir estado.

En el caso del temporizador, esta funcionalidad la traslado a un objeto específico, el objeto obj_timer_manager, o directamente a un sistema de variables globales. La actualización del tiempo la realizo en el evento Step, acumulando el tiempo transcurrido únicamente cuando el juego se encuentra en ejecución.

```
if (global.game_paused) exit;

if (global.timer_running) {

    var dt = delta_time / 1000000;
    global.elapsed_time += dt;
}
```

Ilustración 139. Captura de la lógica del timer.

En GameMaker no existe un sistema asociado a un nodo global accesible automáticamente, sino que es necesario definir explícitamente qué objeto gestiona el tiempo y cómo se actualiza. Asimismo, el formateo del tiempo para su representación en pantalla no se delega a un nodo de interfaz, sino que he de realizarlo en el propio sistema de UI mediante funciones auxiliares.

La lógica de las vidas del jugador la integro directamente dentro del objeto obj_player, eliminando la dependencia de nodos de interfaz como Label. En lugar de actualizar automáticamente un componente visual, opto por almacenar el estado current_health en el propio objeto y representarlo posteriormente en el evento Draw de un objeto de UI. La recepción de daño la controlo mediante una función que, además, incorpora un periodo de invulnerabilidad tras cada impacto.

```
function take_damage(amount) {

    if (is_invulnerable) return;

    current_health -= amount;

    if (current_health <= 0) {
        player_die();
    } else {
        start_invulnerability();
    }
}

function start_invulnerability() {

    is_invulnerable = true;
    invuln_timer = invulnerability_duration;
    blink_timer = 0;
}
```

Ilustración 140. Captura de la lógica de las vidas e invulnerabilidad.

Este sistema debo construirlo manualmente mediante variables de estado y contadores que se actualizan en cada frame. El efecto de parpadeo, por ejemplo, lo logro alternando la visibilidad del sprite en función de un temporizador.

Este enfoque también implica que la interfaz de usuario no está directamente vinculada al estado del jugador, sino que actúa como un sistema independiente que consulta variables globales o referencias a objetos.

En lo relativo a los elementos decorativos del fondo, he optado por una solución sencilla pero efectiva: la creación de múltiples objetos independientes, cada uno asociado a una sprite de meteorito distinta, colocados manualmente en una capa de instancias situada por debajo de los elementos jugables. Estos objetos no contienen lógica ni scripts, actuando exclusivamente como elementos visuales estáticos.

Debido a que las imágenes del asset pack venían por separado y el tileset proporcionado con todos los sprites no contaba con las dimensiones adecuadas, no fue posible crear un tileset cargando cada imagen de forma individual y añadiéndolas posteriormente, ya que GameMaker no dispone de esa funcionalidad. Tampoco fue viable generar un tileset de manera convencional, ya que el archivo original no respetaba las proporciones necesarias. Por este motivo, opté por trabajar con instancias colocadas en capas específicas, sin necesidad de componentes adicionales. Esta solución, además de simplificar el proceso, permite una iteración visual más rápida y flexible.

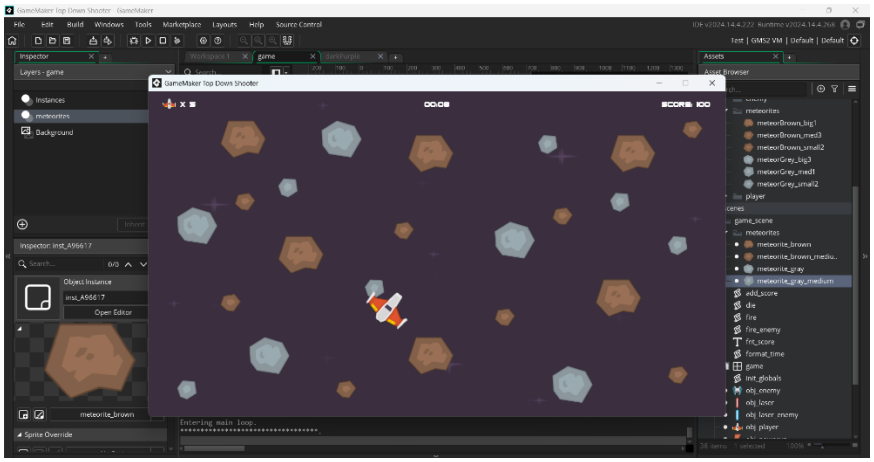


Ilustración 141. Captura de la lógica del timer, vidas y fondo en GameMaker.

3.3.5 Lógica del menú de pausa

En el menú de pausa, no existe un mecanismo interno que detenga automáticamente la ejecución de la lógica del juego; aquí debo construir uno manualmente mediante el control explícito del flujo de ejecución.

En este caso, la pausa la implemento a través de una variable global, la variable global.game_paused, que actúa como estado central del juego. Esta variable se alterna al pulsar la tecla “P” y condiciona la ejecución del resto de la lógica. Cada objeto debe comprobar manualmente si el juego está pausado antes de ejecutar su lógica en el evento Step. Este patrón introduce una responsabilidad adicional para el desarrollador, pero también ofrece un control más granular sobre qué elementos deben detenerse y cuáles no. El sistema de entrada que activa la pausa se integra directamente en el flujo de actualización.

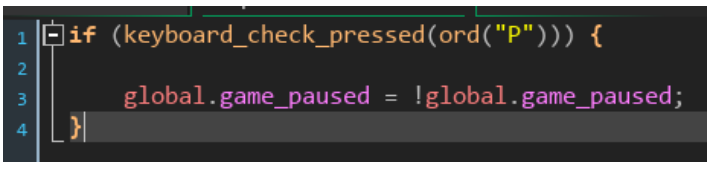


Ilustración 142. Captura de la lógica de pausado dentro de obj_game_manager.

Una vez activado este estado, el resto de objetos del juego, como el jugador, enemigos, proyectiles y temporizador, entre otros, deben incluir una comprobación temprana en su evento Step para evitar que continúen actualizándose.

```
if (global.game_paused) exit;

var dt = delta_time / 1000000;

var input_x = keyboard_check(ord("D")) - keyboard_check(ord("A"));
var input_y = keyboard_check(ord("S")) - keyboard_check(ord("W"));
```

Ilustración 143. Captura de ejemplo de la lógica de pausado dentro del Step del jugador.

En cuanto a la representación visual del menú de pausa, he optado por dibujar el menú directamente en el evento Draw GUI de un objeto, como es el caso con obj_ui. Este enfoque permite renderizar elementos de interfaz en coordenadas de pantalla independientemente de la cámara, pero requiere construir manualmente todos los componentes visuales, incluyendo texto, botones y estados de interacción.

```
if (global.game_paused) {

    audio_pause_all();

    var gui_w = display_get_gui_width();
    var gui_h = display_get_gui_height();

    var btn_w = 220;
    var btn_h = 60;

    var resume_x = (gui_w - btn_w) / 2;
    var resume_y = gui_h / 2;

    var quit_x = resume_x;
    var quit_y = resume_y + 80;

    if (mouse_check_button_pressed(mb_left)) {

        var mx = device_mouse_x_to_gui(0);
        var my = device_mouse_y_to_gui(0);

        if (point_in_rectangle(mx, my, resume_x, resume_y, resume_x + btn_w, resume_y + btn_h)) {
            global.game_paused = false;
            audio_resume_all();
        }

        if (point_in_rectangle(mx, my, quit_x, quit_y, quit_x + btn_w, quit_y + btn_h)) {
            audio_resume_all();
            room_goto(main_menu)

        }
    }
}
```

Ilustración 144. Captura del Step con la lógica del menú de pausa dentro de obj_ui.

La interacción con los botones resume y quit también la gestiono manualmente mediante la comprobación de la posición del ratón en coordenadas GUI.

Uno de los problemas más relevantes que he encontrado durante el desarrollo de esta sección fue precisamente que, tras implementar el menú de pausa, ciertos elementos como los proyectiles continuaban moviéndose. Esto demostraba que la pausa no afectaba automáticamente a todos los objetos. Lo solucioné introduciendo la comprobación if (global.game_paused) exit; en todos los eventos Step relevantes, asegurando que ninguna lógica se ejecutase mientras el juego estuviera pausado.

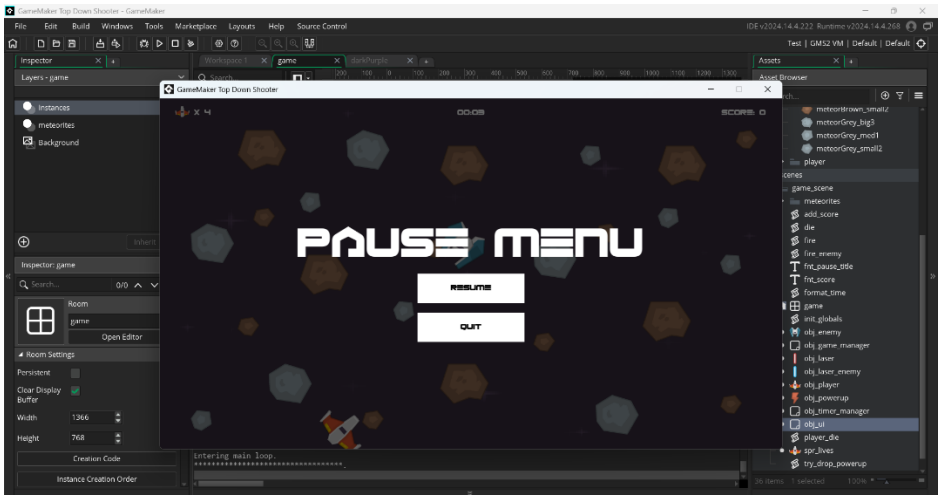


Ilustración 145. Captura de la lógica del menú de pausa en GameMaker.

3.3.6 Escena main menu y game over

El menú principal y el menú de fin de partida en GameMaker mantienen un formato similar al del resto del proyecto, adoptando un enfoque completamente manual tanto en la representación visual como en la gestión de la interacción.

En el caso del menú principal, no existe un sistema de UI declarativa. En su lugar, utilizo un objeto específico, el objeto obj_main_menu, que dibuja todos los elementos visuales en el evento Draw GUI y gestiona la interacción en el evento Step. Esto implica que tanto el renderizado del texto como la detección de clics los implemento manualmente, lo que otorga mayor control, pero también incrementa la complejidad.

El renderizado del menú principal se basa en dibujar el título y los botones centrados en pantalla, calculando dinámicamente sus posiciones.

```
var gui_w = display_get_gui_width();
var gui_h = display_get_gui_height();

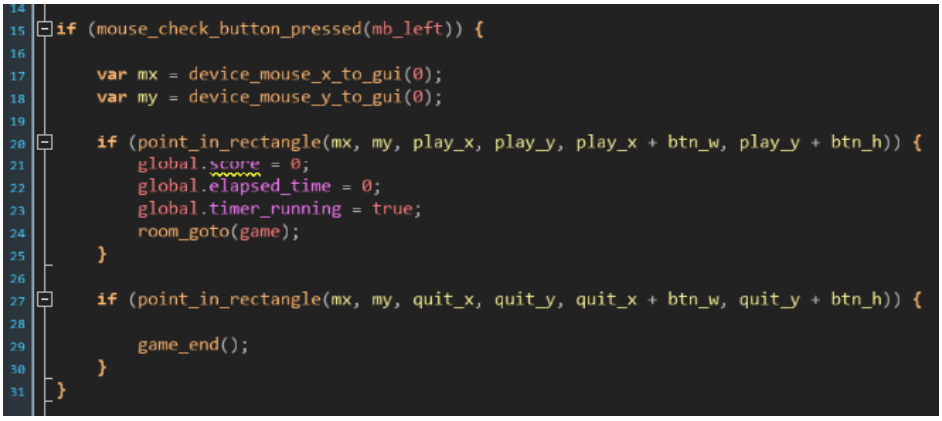
draw_set_font(fnt_title);
draw_set_color(c_white);

var title = "GAMEMAKER SPACE GAME";
var title_w = string_width(title);

draw_text((gui_w - title_w) / 2, gui_h / 3, title);
```

Ilustración 146. Captura de la lógica del renderizado del menú inicial.

La interacción con los botones la gestiono mediante la comprobación de si el cursor se encuentra dentro de un rectángulo y si se ha producido un clic del ratón.



```
if (mouse_check_button_pressed(mb_left)) {

    var mx = device_mouse_x_to_gui(0);
    var my = device_mouse_y_to_gui(0);

    if (point_in_rectangle(mx, my, play_x, play_y, play_x + btn_w, play_y + btn_h)) {
        global.score = 0;
        global.elapsed_time = 0;
        global.timer_running = true;
        room_goto(game);
    }

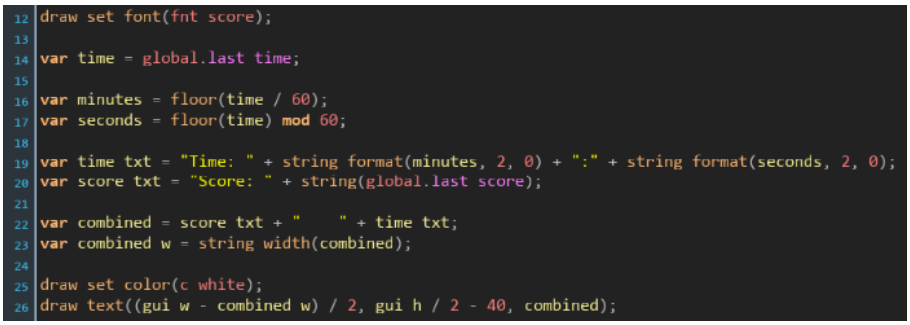
    if (point_in_rectangle(mx, my, quit_x, quit_y, quit_x + btn_w, quit_y + btn_h)) {

        game_end();
    }
}
```

Ilustración 147. Captura de la lógica de la interacción con los botones.

La interfaz no está compuesta por elementos reutilizables con comportamiento propio, sino que los construyo directamente mediante operaciones de dibujo y lógica condicional. Todo el sistema lo desarrollé desde cero en cada objeto de interfaz.

El menú de game over sigue una lógica muy similar, reutilizando el mismo sistema de renderizado y entrada, pero incorporando la visualización de datos persistentes como la puntuación final y el tiempo de partida. Accedo a estos valores directamente mediante variables globales como global.last_score y global.last_time. El formateo del tiempo y su representación en pantalla lo realizo manualmente antes de dibujar el texto.



```
draw set font(fnt score);

var time = global.last time;

var minutes = floor(time / 60);
var seconds = floor(time) mod 60;

var time txt = "Time: " + string format(minutes, 2, 0) + ":" + string format(seconds, 2, 0);
var score txt = "Score: " + string(global.last score);

var combined = score txt + "    " + time txt;
var combined w = string width(combined);

draw set color(c white);
draw text((gui w - combined w) / 2, gui h / 2 - 40, combined);
```

Ilustración 148. Captura de la lógica del formateo del tiempo y representación en pantalla.

La navegación entre escenas la gestiono mediante la función room_goto, donde las rooms actúan como contenedores completos del estado del juego.

Un aspecto relevante del menú de game over es la necesidad de reiniciar manualmente el estado global antes de iniciar una nueva partida o regresar al menú principal. Este comportamiento lo implemento mediante la reasignación de variables globales.

```
if (point_in_rectangle(mx, my, replay_x, replay_y, replay_x + btn_w, replay_y + btn_h)) {

    global.score = 0;
    global.elapsed_time = 0;
    global.timer_running = true;

    room_goto(game);
}


if (point_in_rectangle(mx, my, quit_x, quit_y, quit_x + btn_w, quit_y + btn_h)) {

    global.score = 0;
    global.elapsed_time = 0;
    global.timer_running = false;

    room_goto(main_menu);
}
```

Ilustración 149. Captura de la lógica del cambio de escenas en GameMaker.

Tanto el menú principal como el menú de fin de partida reflejan cómo es GameMaker; este es un sistema de interfaz construido de forma manual, donde el desarrollador controla directamente cada aspecto del renderizado y la interacción.

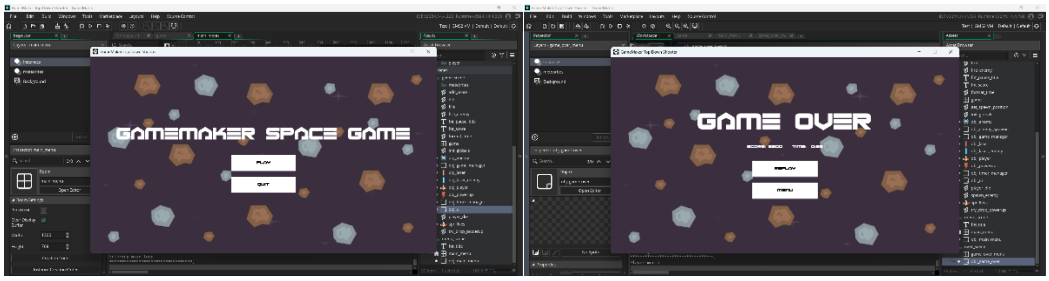


Ilustración 150. Capturas del menú principal y de game over en GameMaker.

3.3.7 Sobre los shaders

Al igual que el desarrollo en los otros motores, no he usado shaders para el desarrollo en GameMaker, aunque creo conveniente hablar de ellos. Los shaders permiten modificar la forma en la que se renderizan sprites, fondos o elementos de la interfaz gráfica utilizando programación en GPU mediante GLSL ES. Gracias a los shaders, los desarrolladores pueden crear todo tipo de efectos visuales. GameMaker trata los shaders de una manera distinta; el motor permite la integración de shaders a través de funciones específicas que permiten activarlos y enviar parámetros personalizados desde el código del juego, habilitando el control de los efectos visuales a tiempo real.

4 Conclusiones del Proyecto

Dando por concluido el desarrollo del mismo juego en los tres motores distintos, he de decir que he aprendido mucho sobre los motores, siendo GameMaker del que más he aprendido, ya que no tenía experiencia previa con este.

Para el veredicto sobre el mejor motor para el desarrollo de juegos en 2D, tendré en cuenta entre muchos otros aspectos como sus características técnicas, comunidad, documentación, precio, potencia y sencillez, entre muchos otros factores. Al fin y al cabo, el mejor dependerá en gran medida de lo que uno quiera hacer, pero trataré de nominar a uno de los motores como el mejor para el desarrollo de juegos 2D de manera general.

En primer lugar, es importante destacar que tanto Unity como Godot y GameMaker son herramientas sólidas, con documentación completa y capacidades suficientes para desarrollar juegos en 2D de manera profesional. Sin embargo, presentan diferencias significativas tanto en su enfoque como en la experiencia de desarrollo que ofrecen.

Desde el punto de vista del aprendizaje y accesibilidad, Godot se posiciona como la mejor opción de los tres. Su facilidad de uso, junto con un lenguaje de programación como GDScript, más sencillo e intuitivo que C#, permite que nuevos desarrolladores puedan empezar a trabajar de manera rápida y eficiente. Adicionalmente, aunque Unity cuenta con una mayor cantidad de tutoriales debido a su antigüedad y popularidad, la curva de aprendizaje en este motor es más pronunciada. Esto se debe principalmente a la complejidad de su lenguaje de programación y a su estructura basada en GameObjects y componentes, que puede resultar un poco más difícil al comenzar que la estructura basada en nodos de Godot.

Por otro lado, GameMaker, que en teoría debería ser el motor más accesible, presenta ciertas contradicciones en la práctica. Si bien es cierto que en el desarrollo del proyecto ha requerido menos lógica programada en comparación con los otros motores, la forma en la que se estructura dicha lógica puede resultar confusa. Su sistema, junto con el uso de GML y la posibilidad de emplear visual scripting, puede ser útil para proyectos pequeños o prototipos rápidos, pero a medida que aumenta la complejidad del proyecto, este enfoque tiende a volverse desorganizado y difícil de mantener. El visual scripting, aunque accesible, escala mal y puede generar estructuras difíciles de gestionar.

En cuanto a la comunidad y recursos disponibles, tanto Unity como Godot destacan por contar con comunidades activas que facilitan el aprendizaje y la resolución de problemas. Unity dispone de una comunidad más amplia y consolidada, lo que se traduce en una mayor cantidad de recursos, foros y contenido generado por usuarios. No obstante, Godot ha experimentado un crecimiento significativo en los últimos años, ofreciendo también una comunidad muy participativa y en constante expansión. GameMaker, en comparación, cuenta con una comunidad más reducida y menor cantidad de recursos formativos, lo que puede suponer una desventaja para desarrolladores que dependan en gran medida de tutoriales y apoyo externo. Por otro lado, los tres motores disponen de documentaciones muy buenas; yo diría que están a la par en este aspecto. Las documentaciones son intuitivas, están disponibles en varios idiomas, son muy completas y cuentan con una función de búsqueda que hace que sea muy fácil encontrar lo que buscas.

Otro factor determinante es el modelo de negocio de cada motor. Godot destaca claramente en este aspecto al ser completamente gratuito y de código abierto, lo que supone una gran ventaja para desarrolladores independientes, ya que no existen costes ni comisiones asociadas a la publicación de juegos. Unity, aunque gratuito en sus versiones iniciales, introduce comisiones una vez se superan ciertos umbrales de ingresos, lo que puede afectar a la rentabilidad del proyecto. Por su parte, GameMaker ofrece un modelo intermedio, siendo gratuito en su uso básico, pero requiriendo un pago único para tener licencia comercial y exportar a múltiples plataformas, y un modelo de suscripción mensual en el caso de consolas. Estos factores económicos pueden influir de manera significativa en la decisión final del motor a utilizar.

Desde un punto de vista técnico, también existen diferencias importantes en la manera de trabajar con cada motor. Unity se caracteriza por su arquitectura basada en componentes y su uso de C#, lo que ofrece una gran flexibilidad y potencia, pero a costa de una mayor complejidad. También, Unity cuenta con una gran cantidad de herramientas listas para usar que superan a las de los otros engines. Dentro de estas herramientas se encuentran: el Sprite Editor, que permite cortar spritesheets, configurar pivotes y generar animaciones de manera visual y rápida; las Sorting Layers y el Order in Layer, que son fundamentales para controlar el orden de renderizado de personajes, fondos y efectos en escena; sistemas como Tilemap y Grid, que son ideales para construir niveles en juegos de diferentes géneros sin necesidad de programar cada elemento manualmente; y un sistema de animación robusto con Animator y Animation Clips, facilitando transiciones fluidas y comportamientos complejos para personajes 2D. Unity también cuenta con herramientas para gráficas y de efectos visuales, Shader Graph, que permite crear shaders personalizados sin necesidad alguna de código, y el motor incorpora soporte para el Universal Render Pipeline (URP), que mejora el rendimiento y habilita luces 2D, sombras y materiales avanzados.

Godot, en el aspecto técnico, utiliza un sistema de nodos más modular y estructurado, muy similar a Unity, que facilita la organización del proyecto y permite una comprensión más clara de la lógica del juego. No posee tantas herramientas listas para usar como Unity, pero cuenta con cosas como su propio sistema de nodos que hace que cada elemento sea fácilmente reutilizable. Otras herramientas de las que Godot dispone para el desarrollo 2D son los nodos como Sprite2D, AnimatedSprite2D y TileMap, que permiten desde mostrar imágenes simples hasta construir niveles complejos basados en cuadrículas de forma rápida y eficiente. Además, el sistema de AnimationPlayer permite crear animaciones detalladas de forma visual, controlando propiedades como posición, escala o frames de sprites sin necesidad de programación avanzada. Cabe mencionar el sistema de señales del que dispone Godot; este sistema permite conectar eventos entre nodos de forma flexible, reduciendo la necesidad de acoplamiento entre scripts y haciendo que la lógica del juego sea más fácil de mantener. Godot posee un potente sistema de shaders que, mediante su lenguaje de shaders propio, permite crear diferentes efectos visuales con un mayor grado de customización; aunque, si el usuario no quiere programar, puede hacer uso del editor de Visual Shaders, que facilita la creación de efectos visuales mediante nodos, sin escribir código.

Sobre el aspecto técnico de GameMaker, este adopta un enfoque más simplificado, pero menos estructurado, lo que puede derivar en problemas de escalabilidad en proyectos más

grandes. GameMaker es el motor más sencillo que requiere programar menos lógica que el resto; incluso mediante el uso de su función GML Visual permite crear la lógica sin necesidad de programar código alguno. Adicionalmente, GameMaker dispone de herramientas para editar sprites, hojas de sprites y animar por fotogramas. A esto se le suman herramientas como Room Editor, que permite crear niveles de manera visual utilizando un método de arrastrar y soltar, su sistema de tilesets y tilemaps, que facilitan la creación de niveles, y la herramienta Sequence Editor, que facilita la creación de cinemáticas y animaciones avanzadas.

Unity es el más potente de los motores con el mayor número de herramientas listas para usar, aunque esto viene a costa de tardar más tiempo en compilar, ser más pesado y no ser tan ligero como los otros motores. Godot dispone de varias herramientas listas para usar, aunque no al mismo nivel que Unity, pero Godot es más lightweight que el otro motor. GameMaker está más optimizado para 2D, no requiere programar tanto o nada en absoluto, es bastante lightweight y dispone de herramientas para simplificar el desarrollo de juegos en dos dimensiones, aunque es el más limitado de los tres.

Además, aunque no lo he tenido en cuenta como criterio principal en esta comparación, cabe mencionar que tanto Unity como Godot ofrecen soporte para tecnologías más avanzadas como 3D o realidad virtual, lo que los convierte en motores más versátiles a largo plazo. GameMaker, al estar centrado exclusivamente en 2D, limita sus posibilidades en este sentido.

En conclusión, tras analizar diversos factores como facilidad de uso, comunidad, documentación, modelo de negocio y aspectos técnicos. Puedo afirmar que Godot es la opción más equilibrada y recomendable generalmente para el desarrollo de juegos en 2D, especialmente para desarrolladores independientes o principiantes. Su accesibilidad, su modelo gratuito y su estructura de trabajo clara lo convierten en una herramienta muy eficiente. Si bien no cuenta con las herramientas listas para usar más potentes, dispone de un amplio surtido que facilita el desarrollo de juegos en dos dimensiones. Unity sigue siendo una opción muy potente, respaldada por una gran comunidad y con muchas herramientas de alto nivel listas para usar, pero su complejidad, modelo de costes y ser más pesado pueden suponer una barrera en ciertos casos. GameMaker, aunque útil para proyectos simples y para principiantes, presenta limitaciones en cuanto a escalabilidad y organización que lo hacen menos recomendable para desarrollos más complejos.

Por tanto, la elección del motor dependerá siempre del contexto y las necesidades del proyecto, pero, en términos generales, Godot se posiciona como la mejor opción dentro de esta comparativa para el desarrollo de juegos en 2D.

5 Uso de herramientas de Inteligencia Artificial

En este apartado se adjunta la declaración responsable sobre el uso de la IA en este TFG.

Gabriel Sánchez López, DECLARO BAJO MI RESPONSABILIDAD:

1. Que el presente Trabajo de Fin de Grado (TFG) es íntegramente original, de elaboración propia, y que no ha sido redactado total ni parcialmente por ninguna herramienta de inteligencia artificial generativa.

2. Que, en el desarrollo técnico del proyecto, he utilizado de forma puntual la herramienta ChatGPT gratuita, exclusivamente como recurso auxiliar de consulta técnica para la revisión y comprensión de fragmentos de código, corrección ortográfica y conceptos relacionados con la historia de los videojuegos y sus motores.

3. Que este uso se ha realizado con arreglo a las indicaciones contenidas en el documento: "Uso de herramientas de Inteligencia Artificial (IA) en el TFG" de la Universidad, documento institucional que promueve un uso responsable, ético y pedagógico de este tipo de herramientas.

4. Que en ningún caso se ha empleado la IA generativa para:

- Redactar apartados sustanciales del TFG.
- Generar contenido explicativo, resultados y conclusiones.
- Sustituir el proceso intelectual, analítico o crítico del TFG.

5. Que el contenido, código, resultados y conclusiones del presente TFG son fruto exclusivo del trabajo personal y reflejan un análisis comparativo original sin uso de IA para redacción o generación de contenido sustancial.

6. Que autorizo expresamente que el presente TFG sea sometido a sistemas de detección de plagio y análisis de originalidad que la Universidad estime oportunos, con el fin de verificar la originalidad del texto y el cumplimiento de las buenas prácticas recomendadas por la Universidad.

Y para que conste, firmo la presente declaración.



6 Bibliografía

[1] Historia de los videojuegos
https://es.wikipedia.org/wiki/Historia_de_los_videojuegos

[2] Motor de videojuego
https://es.wikipedia.org/wiki/Motor_de_videojuego

[3] Unity (motor de videojuego)
https://es.wikipedia.org/wiki/Unity_(motor_de_videojuego)

[4] Julián Kaper. (16 sept 2023). El escándalo de Unity: la tecnología de videojuegos que formó a la industria y ahora es odiada.
https://www.ambito.com/tecnologia/el-escandalo-unity-la-videojuegos-que-formo-la-industria-y-ahora-es-odiada-n5822425

[5] Godot
https://es.wikipedia.org/wiki/Godot

[6] Redot: Why We Forked Godot
https://www.youtube.com/watch?v=Ma87X6a0n88

[7] GameMaker Studio
https://es.wikipedia.org/wiki/GameMaker_Studio

7 Anexo

A continuación, dejaré varios enlaces donde se podrán descargar las builds de los juegos creados para este TFG desde la página web itch.io.

Versión de Unity:
https://depressedengineersl.itch.io/unity-space-game?secret=q7jxNm329qv4TaNzWhllcRQ18

Versión de Godot:
https://depressedengineersl.itch.io/godot-space-game?secret=TGq2WZ8PIqS34AMuNve0DKYoM4

Versión de GameMaker:
https://depressedengineersl.itch.io/gamemaker-space-game?secret=Lm0sjiqMAmrF0CNUjq6nGX6ixPA